



Entwicklung eines Algorithmus zur Hinderniserkennung mittels MIMO FMCW Radar

Bachelorarbeit

von

Shaofei Liu

Hochschule München

Fakultät für angewandte Naturwissenschaften und Mechatronik

Studiengang Bachelor Mechatronik/Feinwerktechnik

Matrikelnummer: 00037918

Referent: Prof. Dr.-Ing. Datong Wu

Korreferent: Prof. Dr. rer. nat. Alfred Kersch

Betreuerin: Dipl.-Ing. Katharina Burger, CPT Group GmbH

Tag der Einreichung: 27.09.2019

München 2019

Sperrvermerk

In dieser Abschlussarbeit sind Geschäftsgeheimnisse der CPT Group GmbH enthalten. Eine Weitergabe, Vervielfältigung oder Veröffentlichung der Arbeit sowie die Verwertung und Mitteilung ihres Inhalts ist ohne ausdrückliche schriftliche Genehmigung der CPT Group GmbH, Siemensstraße 12, 93055 Regensburg, nicht gestattet.

Name: Shaofei Liu

geb.: 19.10.1996

Matr.Nr.: 00037918

06 MFB im SS 2019

Erklärung

gemäß § 13 Abs. 5 RaPO

Hiermit erkläre ich, dass ich die Bachelorarbeit selbstständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Ort, Datum

Unterschrift

Kurzfassung

Seit einigen Jahren wird die Technologie des FMCW-Radars im zivilen Bereich intensiv entwickelt und vermehrt eingesetzt. Aufgrund seiner Messfähigkeiten im Ultranahbereich und seiner hohen Entfernungsmessgenauigkeit wird er zunehmend zur Hindernis- und Fahrbahnerkennung eingesetzt, insbesondere in der Automobilindustrie. Viele Automobilunternehmen haben mit FMCW-Radaren Fahrzeugprodukte entwickelt, wie Kollisionsalarme, Einbruchalarme und automatische Fahrsysteme. Wie andere Sensoren hat das FMCW-Radar jedoch auch Fehleranfälligkeiten. So ist es entscheidend, die Radarmessergebnisse nach der digitalen Signalverarbeitung mit einigen Methoden und Algorithmen zu verbessern. In dieser Arbeit wird basierend auf der Verwendung des MIMO-FMCW-Radars zur Hinderniserkennung vor der Autotür eine Methode zum Messen und zum Bestimmen der Position von Hindernissen im Ultranahbereich vorgestellt. Die mit dem MIMO-FMCW-Radar durchgeführten Messversuche werden in der Arbeit beschrieben und die Leistung des Radars wird anhand dieser Versuchsergebnisse bewertet. Zwei Algorithmen werden in dieser Arbeit vorgestellt, um die geringere Messintensität des Radars für Hindernisse bei großen Winkeln und Entfernungen zu kompensieren und die Radarmessung für Hindernisse mit schwachem Reflexionsvermögen zu verbessern. Das „Verstärkungsgitter“ verwendet die Radarmessungen des gleichen Hindernisses, um die Stärke von Radarsignalen zu kompensieren, die aus unterschiedlichen Richtungen und in unterschiedlichen Entfernungen reflektiert werden. Die „Leermessungsmethode“ generiert das Leermessungsspektrum, indem sie die Spektren mehrerer Messungen bei statischer Umgebung vergleicht, um die Peaks zu finden, die in den neuen Messungen auftreten.

Schlüsselwörter: FMCW-Radar, Hinderniserkennung, Radarmessung, MIMO

Abstract

In recent years, the technology of the FMCW radar in the civilian area has been intensively developed and used. Due to its measuring capabilities in the ultra-close range and its high distance measuring accuracy, it is increasingly used for obstacle detection and road detection, especially in the automotive industry. Many automotive companies have developed vehicle products with FMCW radars such as collision alarms, burglar alarms and automatic driving systems. However, like other sensors, the FMCW radar also has error susceptibility. So, it is crucial to improve the radar measurement results after digital signal processing with some methods and algorithms. In this paper, based on the use of the MIMO FMCW radar for obstacle detection in front of the car door, a method for measuring and determining the position of obstacles in the ultra-close range is presented. The measurements carried out with the MIMO FMCW radar are described in the paper and the performance of the radar is evaluated based on these test results. Two algorithms are presented in this paper to compensate for the lower measurement intensity of the radar for large-angle and long-range obstacles and to improve the radar measurement for low reflectivity obstacles. The “Gain Grid” uses the radar measurements of the same obstacle to compensate for the gain of radar signals reflected from different directions and different distances. The “Empty Measurement Calibration” generates the empty measurement spectrum by comparing the spectra of several measurements in the static environment to find the peaks that appear in the new measurements.

Key words: FMCW radar, obstacle detection, radar measurement, MIMO

Abkürzungsverzeichnis

A

Abb.	<i>Abbildung</i>
ADC	<i>Analog-to-Digital Converter</i>

C

CA-CFAR	<i>Cell-Averaging-CFAR</i>
CFAR	<i>Constant False Alarm Rate</i>
CUT	<i>Cell Under Test</i>

D

DCA	<i>Digital-to-Analog Converter</i>
DFT	<i>Discrete Fourier Transform</i>
DOA	<i>Direction of Arrival</i>

F

FMCW	<i>Frequency Modulated Continuous Wave</i>
FFT	<i>Fast Fourier Transform</i>

I

IF	<i>Intermediate Frequency</i>
----	-------------------------------

K

Kfz	<i>Kraftfahrzeug</i>
-----	----------------------

M

MIMO	<i>Multiple Input Multiple Output</i>
------	---------------------------------------

Abbildungsverzeichnis

Abb. 1.1: Funktionsprinzip „Power Door“	2
Abb. 2.1: Eine Radix-2-Zerlegungsberechnung für DFT.....	6
Abb. 2.2: Zerlegungsberechnung für DFT des 8-Punkt-Signals	6
Abb. 2.3: Zellenselektion von 1D-CFAR.....	8
Abb. 2.4: Beispiel für 1D-CFAR	9
Abb. 2.5: Zellenselektion von 2D-CFAR.....	10
Abb. 2.6: Zeitdomänensignal	11
Abb. 2.7: Spektrum ohne Zero-Padding.....	11
Abb. 2.8: Zeitdomänensignal mit Zero-Padding.....	12
Abb. 2.9: Spektrum mit Zero-Padding.....	12
Abb. 2.10: Messprinzip des FMCW-Radarmoduls	13
Abb. 2.11: Änderungen der Frequenzen des TX-Chirps und des RX-Chirps mit der Zeit	14
Abb. 2.12: Frequenz des IF-Signals über die Zeit.....	15
Abb. 2.13: Ein Satz TX-Chirps und ein Satz von einem Objekt reflektierten RX-Chirps	16
Abb. 2.14: Winkelbeziehung zwischen der Sendeantennen, dem Objekt und der Empfangsantennen	17
Abb. 3.1: Anordnung der Antennen des Radarmoduls „AWR1642BOOST-ODS“	19
Abb. 3.2: Die Verbindung zwischen DCA1000EVM und Radar	20
Abb. 3.3: Schematisches Diagramm des Radars.....	20
Abb. 3.4: Parametereinstellungen der Radarmessung.....	21
Abb. 3.5: Parametereinstellungen der Radarmessung.....	22
Abb. 3.6: Binärdateiformat des vom Host empfangenen Signals	22
Abb. 3.7: Headerformat der Binärdatei	23
Abb. 3.8: Datenmatrix aus Binärdatei	23
Abb. 3.9: Beispiel eines 1D-Spektrums	24
Abb. 3.10: Beispiel eines 2D-Spektrums	25
Abb. 3.11: Signalintensität des 2D-Range-Angle -Diagramms	26
Abb. 3.12: Positionsbeziehung zwischen Radarantennen	27
Abb. 3.13: Positionsbeziehung zwischen den Antennen auf dem Modul „AWR1642BOOST-ODS“.	27
Abb. 3.14: Positionsbeziehung zwischen den virtuellen Antennen auf dem Modul „AWR1642BOOST-ODS“	28
Abb. 3.15: Antennenstruktur des Moduls „AWR1642BOOST-ODS“ und verwendete Antennen für die Berechnung der „TX-Ebene“	28
Abb. 3.16: Antennenstruktur des Moduls „AWR1642BOOST-ODS “ und verwendete Antennen für die Berechnung der „RX-Ebene“	29
Abb. 3.17: Positionsbeziehung zwischen RX-Ebene, TX-Ebene, Radar und Hindernis	30
Abb. 3.18: Der Abstand und Winkel des Hindernisses zum Radar in der x-y-Ebene.....	31
Abb. 3.19: Der Abstand und Winkel des Hindernisses zum Radar in der y-z-Ebene.....	31
Abb. 4.1: Messbereich des Radars	33
Abb. 4.2: Aufbau der Radarmessung	34
Abb. 4.3: Positionen des Stifts in der Messung.....	34
Abb. 4.4: Ein verarbeitetes Signalspektrum.....	35
Abb. 4.5: Vergleich zwischen Spektren mit und ohne Hindernisse	36
Abb. 4.6: Vergleich zwischen Hindernisspektren bei unterschiedlichen Winkeln	36

Abb. 4.7: Messergebnisse des Radars in verschiedenen Bereichen in 360°-Messung.....	37
Abb. 4.8: Messaufbau in der Halle.....	38
Abb. 4.9: Standardhindernisse	39
Abb. 4.10: Messaufbau mit Standard-Hindernissen.....	39
Abb. 4.11: 1D-FFT-Spektrum einer Messung mit Standardhindernissen	40
Abb. 4.12: Vergleich zwischen Alupfostenspektrum und Leermessungsspektrum	40
Abb. 4.13: Vergleich zwischen Spektren der unterschiedlichen Pfosten	41
Abb. 4.14: Vergleich zwischen Alupfostenspektren bei unterschiedlichen Winkeln	41
Abb. 4.15: 2D-Range-Angle-Diagramm des Aluminiumpfostens bei 1m, 0°	42
Abb. 4.16: 2D-Range-Angle-Diagramm des Aluminiumpfostens bei 1m, 45°	43
Abb. 4.17: 2D-Range-Angle-Diagramm mit und ohne Aluminiumpfosten.....	43
Abb. 4.18: 2D-Range-Angle-Diagramm mit und ohne des Holzpfostens.....	44
Abb. 4.19: Erkennungsergebnis des Aluminiumpfostens bei 1,5m, 0° durch CFAR	44
Abb. 4.20: Erkennungsergebnis des Holzpfostens bei 1,5m, 45° durch CFAR.....	45
Abb. 5.1: Beispiel eines 15 × 15-Verstärkungsgitters in Halbkreisform	47
Abb. 5.2: Messungsaufbau für „Field of View“-Messungen	48
Abb. 5.3: Die Positionen der 70mm-Aluminiumpfosten bei der „Field of View“-Messung.....	49
Abb. 5.4: Messergebnisse desselben Hindernisses an verschiedenen Positionen unter denselben Messbedingungen.....	51
Abb. 5.5: Messergebnisse in der x-y-Ebene, nachdem den vier Ecken des Messbereichs und den Mittelpunkten der vier Seiten Werte zugewiesen werden.....	52
Abb. 5.6: Drei Datenpunkte im Raum ohne Oberflächenanpassung.....	53
Abb. 5.7: Oberflächenanpassung mit drei Datenpunkten im Raum	54
Abb. 5.8: Oberflächenanpassung mit 3 Datenpunkten im Raum	54
Abb. 5.9: Einige Punkte in der Ebene, die zur Parkettierung der Ebene mit Dreiecken verwendet werden	55
Abb. 5.10: Die ersten drei Schritte und Endergebnisse des „Webalgorithmus“	56
Abb. 5.11: Die x-y-Ebene des Messergebnisses, die durch den „Webalgorithmus“ in mehrere Dreiecke unterteilt wird	57
Abb. 5.12: Eine mit dem „Webalgorithmus“ erzeugte Oberfläche, die sich aus verschiedenen Messergebnissen zusammensetzt	58
Abb. 5.13: Eine mit der „Curve Fitting Toolbox“ in Matlab erstellte Oberfläche	59
Abb. 5.14: Das 50 × 31-Verstärkungsgitter	60
Abb. 5.15: Eine nicht durch das Verstärkungsgitter verarbeitete Messung (Das Hindernis an der Position des gepunkteten Kreises wird nicht erkannt.)	61
Abb. 5.16: Die durch das Verstärkungsgitter verarbeitete Messung.....	61
Abb. 6.1: Signalspektrum des Aluminiumpfostens bei 1500 mm, -45°.....	63
Abb. 6.2: Die Signalpunkte unter dem 0,8-fachen der Durchschnittsleistung werden auf das 0,8-fache der Durchschnittsleistung gesetzt.....	63
Abb. 6.3: Die zweite Messung nach der Verarbeitung analog zu Abb. 6.2	64
Abb. 6.4: Leermessung generiert aus zwei Messungen	65
Abb. 6.5: Leermessung generiert aus drei Messungen.....	65
Abb. 6.6: Vergleich der zweiten Messung mit Leermessung.....	66
Abb. 6.7: Messergebnisse des 10mm-Kunststoffpfostens bei 1000 mm, 45°	68
Abb. 6.8: Kunststoffpfostenmessung im Vergleich zur Leermessung	68

Inhaltsverzeichnis

Kurzfassung.....	i
Abkürzungsverzeichnis	iii
Abbildungsverzeichnis	iv
1. Einleitung	1
1.1 Technologischer Hintergrund.....	1
1.2 Motivation und Ziele der Arbeit.....	2
1.3 Inhaltlicher Aufbau	2
2. Theoretische Grundlagen	4
2.1 Wichtige Signalverarbeitungsalgorithmen	4
2.1.1 Fast Fourier Transform.....	4
2.1.2 Constant False Alarm Rate (CFAR).....	7
2.1.3 Zero-Padding.....	10
2.2 Radarmessprinzipien	13
2.2.1 Radarmessprinzip für Abstandmessung	13
2.2.2 Radarmessprinzip für Geschwindigkeitsmessung.....	16
2.2.3 Radarmessprinzip für Winkelmessung.....	17
3. Radarmessverfahren und Ermittlung der Hindernisposition	19
3.1 Aufbau und Messergebnisse des Radars	19
3.2 MIMO und virtuelle Antennen.....	26
3.3 Die Position des Hindernisses in dem 3D-Raum	29
4. Messprogramme für Hinderniserkennung.....	33
4.1 360°-Messung in der Radarebene	33
4.2 „Field of View“-Messungen mit Standard-Hindernissen.....	38
5. Signalverstärkungsgitter im Messbereich	46
5.1 Einschränkungen des FMCW Radars in praktischen Anwendungen.....	46
5.2 Aufbau eines Verstärkungsgitters	46
5.3 Beispiel für die Verwendung des Verstärkungsgitters	61
6. Algorithmus für Leermessung.....	62
6.1 Generierung des Spektrums ohne Hindernisse.....	62
6.2 Kompensation für das Ergebnis der Leermessung	66

6.3	Erkennung von schwach reflektierenden Materialien mit Algorithmus der Leermessung ...	68
6.4	Praktische Anwendung des Leermessung-Algorithmus.....	69
7.	Zusammenfassung und Ausblick.....	70
7.1	Zusammenfassung.....	70
7.2	Ausblick	71
	Literaturverzeichnis.....	72
	Danksagung.....	74
	Anhang	75
1.	Webalgorithmus	75
2.	Algorithmus zum Konstruieren von Oberflächen mit drei Punkten im Raum	79
3.	Algorithmus für Verstärkungsgitter	81
4.	Algorithmus für Leermessung.....	85
5.	Algorithmus für CFAR.....	87

1. Einleitung

In diesem Kapitel wird der technologische Hintergrund der Hinderniserkennung mit Radar in der Automobilindustrie kurz vorgestellt. Darüber hinaus werden die Motivation und Ziele dieser Arbeit vorgestellt. Am Ende dieses Kapitels werden die Hauptinhalte der anderen Kapitel kurz beschrieben.

1.1 Technologischer Hintergrund

In der Automobilindustrie werden verschiedene Sensoren verwendet, wie zum Beispiel Radar, Ultraschallsensoren, Kameras und Lidar. Radar ist ein häufig verwendeter Sensor in Kraftfahrzeugerkennungsanwendungen zum Erkennen und Verfolgen. Es verwendet elektromagnetische Wellen, um die Entfernung, Richtung, Dopplergeschwindigkeit, Höhe und den Winkel von Objekten zu bestimmen.^{[1] [2]} Im Vergleich zu anderen Sensoren hat Radar seine eigenen Vorteile, wie zum Beispiel funktionieren Radarsensoren bei extremen Witterungsverhältnissen sehr stabil und werden weniger durch Regen und Wind gestört.^[3]

Das Kfz-Radar für die Straßenverkehrssicherheit ist seit mehreren Jahrzehnten in der Entwicklung, und sein Ziel war von Anfang an die Vermeidung von Kollisionen. Jährlich kommt es weltweit im Straßenverkehr zu rund 1,2 Millionen tödlichen Unfällen. Allein in Europa ereignen sich 1,3 Millionen Unfälle, hierbei werden mehr als 40 000 Menschen getötet.^[4] Die ersten Autoradare mit einem aktiven Frequenzverdopplungsreflektor auf der Rückseite von Zielfahrzeugen mit einer Frequenz von 10 GHz wurden Anfang der 1960er Jahre in den USA gebaut.^[5] Aufgrund der Miniaturisierung von Antennen verwenden Radargeräte in der Automobilindustrie jetzt höhere Frequenzen wie 24 GHz und 77 GHz. Der 24 GHz-Bereich hat mittlerweile einen größeren Marktanteil erreicht, vor allem durch Mid-Range-Sensoren. Derzeit dominiert der 77 GHz-Bereich, der explizit für das Automotive-Radar geregelt wurde und weltweit zur Verfügung steht.^[6] Einer der Hauptvorteile von 77 GHz ist die große Bandbreite, die in diesem Band verfügbar ist. Die Verfügbarkeit einer großen Bandbreite verbessert die Entfernungsauflösung und -genauigkeit im Ultranahbereich erheblich.^[7] Daher werden alle in dieser Arbeit beschriebenen Messaufgaben, bei denen Radar zum Messen verwendet wird, vom 77-GHz-Radar ausgeführt.

Darüber hinaus ist das bei dieser Messung verwendete Radar ein FMCW-MIMO-Radar, und seine spezifischen Merkmale werden in den folgenden Kapiteln erläutert.

1.2 Motivation und Ziele der Arbeit

Ein Radarsensor für Autotüren befindet sich in der Entwicklungsphase. Das Ziel ist es, ein Radarsystem für die Autotür zu entwickeln, das Hindernisse in der Nähe der Tür erkennen kann. Die Autotür soll sich eigenständig öffnen und wieder schließen. Befindet sich ein Hindernis in der Nähe der Tür, wird das automatische Öffnen der Tür nicht fortgesetzt.

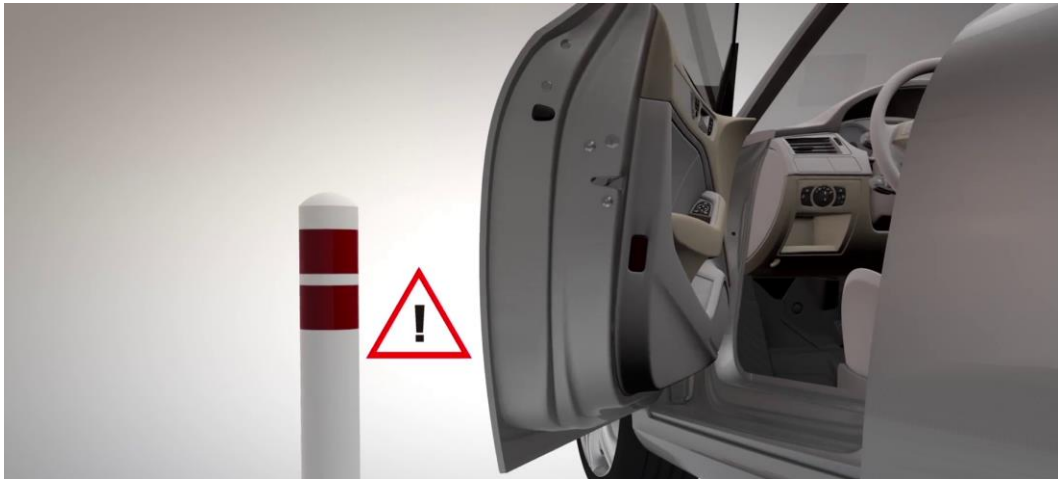


Abb. 1.1: Funktionsprinzip „Power Door“

Daher basiert dieser Artikel auf dem Radar, das für Autotüren und die Erkennung von Hindernissen in der Nähe der Tür verwendet wird. Das in den Messversuchen verwendete Radar ist das AWR1642BOOST-ODS von Texas Instruments mit zwei TX-Antennen und vier RX-Antennen. In diesem Artikel wird die Methode zur Ermittlung der spezifischen Position von Hindernissen im 3D-Raum mithilfe von Radarmesssignalen vorgeschlagen. Außerdem werden neue Algorithmen zur Verbesserung der Radarmessergebnisse vorgeschlagen.

1.3 Inhaltlicher Aufbau

Das zweite Kapitel dieser Arbeit enthält eine Einführung in die gängigen Algorithmen für die Signalverarbeitung sowie in die Methode zum Messen von Abstand, Geschwindigkeit und Winkel von Objekten mithilfe des FMCW-MIMO-Radars.

Das dritte Kapitel dieser Arbeit führt in den Prozess der Radarmessung und in die Verwendung der Messergebnisse zur Bestimmung der räumlichen Position von Hindernissen ein.

Das vierte Kapitel dieser Arbeit stellt die mit Radar durchgeführten Messversuche vor und analysiert die Messergebnisse des Radars in den Messversuchen.

Das fünfte Kapitel dieser Arbeit enthält eine Methode zur Verarbeitung von Radarmessergebnissen, um den Signalleistungsverlust des Radars bei der Fernmessung und der Messung mit großen Winkeln zu kompensieren und genauere Messergebnisse zu erhalten.

Das sechste Kapitel dieser Arbeit enthält eine weitere Methode für den Umgang mit Radarmessungen, um die Fähigkeit des Radars zu verbessern, Hindernisse mit schwachen Reflexionsfähigkeiten zu erkennen.

Das siebte Kapitel dieser Arbeit fasst den gesamten Artikel zusammen und gibt einen Ausblick für zukünftige Arbeiten vor.

2. Theoretische Grundlagen

In diesem Teil werden die theoretischen Grundlagen für die Erkennung von Hindernissen durch Radar erläutert, einschließlich der Basisalgorithmen der digitalen Signalverarbeitung und des Prinzips der Radarmessung.

2.1 Wichtige Signalverarbeitungsalgorithmen

Die Fast Fourier Transformation (FFT) ist einer der am häufigsten verwendeten digitalen Signalverarbeitungsalgorithmen und wandelt diskrete Zeitbereichssignale in diskrete Frequenzbereichssignale um. CFAR ist ein Algorithmus zum Erfassen gültiger Signale im Spektrum aus dem Rauschen nach der FFT. Zero-Padding ist eine Methode zur Erhöhung der spektralen Auflösung durch Hinzufügen von Nullpunkten vor und nach Zeitbereichssignale, wenn die Anzahl der Abtastpunkte zu gering ist. Im Folgenden werden diese Algorithmen besprochen.

2.1.1 Fast Fourier Transform

Fourier-Transform bedeutet, dass eine Funktion, die bestimmte Bedingungen erfüllt, als trigonometrische Funktionen (Sinus- und/oder Kosinus- Funktionen) oder als lineare Kombination ihrer Integrale ausgedrückt werden kann. Es drückt aus, aus welchen dreieckigen Funktionen verschiedener Frequenzen die ursprüngliche Funktion besteht, und was die Amplituden der trigonometrischen Funktionen sind, die diesen Frequenzen entsprechen.

Da die Daten im Bereich der Informationstechnologie normalerweise diskrete Daten sind, verwendet man hauptsächlich diskrete Fourier-Transformation (DFT), um verschiedene Signale mittels Computer zu verarbeiten. Das Spektrum, das durch diskrete Fourier-Transformation gewonnen wird, ist ein diskretes Spektrum, das die Amplituden sinusförmiger Signale mit unterschiedlichen Frequenzen darstellt, die im ursprünglichen diskreten Signal enthalten sind.

Die Formel der diskreten Fourier-Transformation lautet:^[8]

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{kn}, k = 0, 1, \dots, N-1 \quad (2.1)$$

In der Formel sind $W_N = e^{-j\frac{2\pi}{N}}$ und N die Länge des Probenahmeintervalls der Transformation. Die Eingangsdaten $x(n)$ sind N Stichprobenpunkte des ursprünglichen Signals. Der Wert von

$X(k)$, der jedem k im diskreten Spektrum entspricht, stellt die Amplitude eines trigonometrischen Funktionssignals einer bestimmten Frequenz dar.

Da bei der Berechnung der diskreten Fourier-Transformation viele komplexe Exponentialmultiplikationen benötigt werden, ist eine solche Rechenaufgabe für den Computer immer noch zu rechenintensiv. Daher wird eine einfachere Methode zur Berechnung der DFT, nämlich die schnelle Fourier -Transformation (FFT), geschaffen.

Einer der an den häufigsten verwendeten FFT-Algorithmen ist der Radix-2-FFT-Algorithmus basierend auf der Dezimierung der Rechenzeit. Diese FFT wird wie folgt berechnet:

Die Länge der bekannten Sequenz $x(n)$ ist N , und $N = 2^M$, wobei M eine positive Ganzzahl ist. Es seien $x_1(r) = x(2r)$, $x_2(r) = x(2r + 1)$ ($r = 0, 1, \dots, \frac{N}{2} - 1$). Aus der Formel der DFT werden die folgenden Gleichungen erhalten:

$$X_1(k) = DFT[x_1(r)] = \sum_{r=0}^{\frac{N}{2}-1} x_1(r) W_N^{rk} \quad (r = 0, 1, \dots, \frac{N}{2} - 1) \quad (2.2)$$

$$X_2(k) = DFT[x_2(r)] = \sum_{r=0}^{\frac{N}{2}-1} x_2(r) W_N^{rk} \quad (r = 0, 1, \dots, \frac{N}{2} - 1) \quad (2.3)$$

So erhält man auch:

$$\begin{aligned} X(k) &= DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{nk} \quad (n = 0, 1, \dots, N - 1) \\ &= \sum_{r=0}^{\frac{N}{2}-1} x(2r) W_N^{2rk} + \sum_{r=0}^{\frac{N}{2}-1} x(2r + 1) W_N^{(2r+1)k} \\ &= \sum_{r=0}^{\frac{N}{2}-1} x_1(r) W_N^{2rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x_2(r) W_N^{2rk} \\ &= \sum_{r=0}^{\frac{N}{2}-1} x_1(r) W_N^{rk} + W_N^k \sum_{r=0}^{\frac{N}{2}-1} x_2(r) W_N^{rk} \\ &= X_1(k) + W_N^k X_2(k) \quad (k = 0, 1, \dots, \frac{N}{2} - 1) \end{aligned} \quad (2.4)$$

Die erste Hälfte der DFT kann daher durch folgende Formel vereinfacht werden:

$$X(k) = X_1(k) + W_N^k X_2(k) \quad (k = 0, 1, \dots, \frac{N}{2} - 1) \quad (2.5)$$

Die zweite Hälfte der DFT kann ebenfalls durch folgende Formel vereinfacht werden:

$$X\left(k + \frac{N}{2}\right) = X_1(k) - W_N^k X_2(k) \quad (k = 0, 1, \dots, \frac{N}{2} - 1) \quad (2.6)$$

Eine N-Punkte-DFT wird dann in zwei N/2-Punkte-DFTs zerlegt. Die Berechnungsweise ist wie folgt:

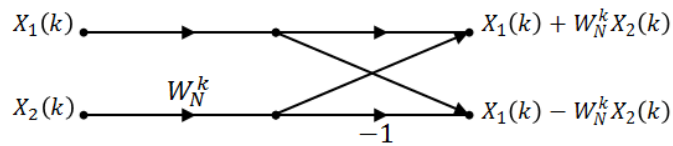


Abb. 2.1: Eine Radix-2-Zerlegungsberechnung für DFT

Diese Berechnungsmethode wird als Schmetterlingsgraph bezeichnet. Weil $N/2 = 2^{M-1}$, erfüllt die Berechnung der $X_1(k)$ und $X_2(k)$ immer noch die Nutzungsbedingungen des Schmetterlingsgraphs, also können die Zerlegungen andauernd weitergehen. Nach jeder Zerlegung reduziert sich der Wertebereich von k um die Hälfte, bis zum letzten $k = 0$. Im letzten Schritt ist $W_N^0 = 1$, deswegen ist keine komplexe Multiplikation mehr erforderlich. Der konkrete vereinfachte Prozess (FFT) der DFT-Berechnung für ein 8-Punkte-Signal wird in der folgenden Abbildung (Abb. 2.2) dargestellt. [9]

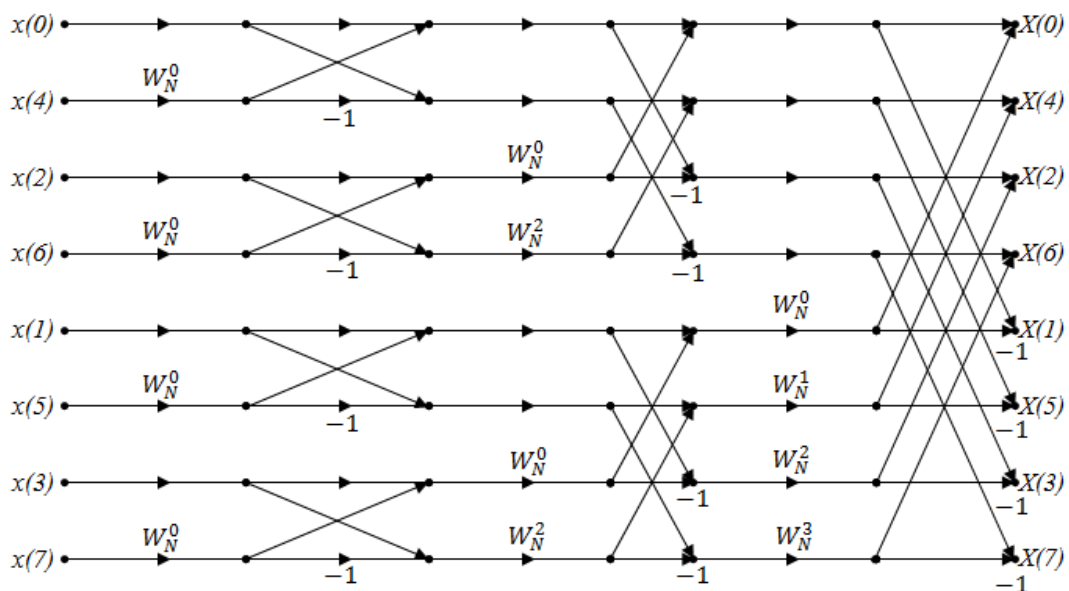


Abb. 2.2: Zerlegungsberechnung für DFT des 8-Punkt-Signals

Es werden neben der Radix-2-FFT oft auch andere Radix-FFTs verwendet. Hier zum Beispiel die Berechnungsmethode für Radix-4-FFT:

$$X(k) = DFT[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{kn} \quad (k = 0, 1, \dots, N-1) \quad (2.7)$$

Wenn $x_1(r) = x(4r)$, $x_2(r) = x(4r + 1)$, $x_3(r) = x(4r + 2)$, $x_4(r) = x(4r + 3)$,
 $r = 0, 1, \dots, \frac{N}{4} - 1$:

$$X(k) = X_1(k) + W_N^k X_2(k) + W_N^{2k} X_3(k) + W_N^{3k} X_4(k) \quad (2.8)$$

$$X\left(k + \frac{N}{4}\right) = X_1(k) - jW_N^k X_2(k) - W_N^{2k} X_3(k) + jW_N^{3k} X_4(k) \quad (2.9)$$

$$X\left(k + \frac{N}{2}\right) = X_1(k) - W_N^k X_2(k) + W_N^{2k} X_3(k) - W_N^{3k} X_4(k) \quad (2.10)$$

$$X\left(k + \frac{3N}{4}\right) = X_1(k) + jW_N^k X_2(k) - W_N^{2k} X_3(k) - jW_N^{3k} X_4(k) \quad (2.11)$$

$$\left(k = 0, 1, \dots, \frac{N}{4} - 1\right)$$

Alle obigen Beschreibungen betreffen eine 1D-DFT. Die zweidimensionale DFT wird anhand einer ähnlichen Methode berechnet, deren Definition lautet:

$$X(p + 1, q + 1) = \sum_{j=0}^{M-1} \sum_{k=0}^{N-1} W_M^{jp} W_N^{kq} x(j + 1, k + 1), j, k = 0, 1, \dots, N - 1 \quad (2.12)$$

In Formel sind $W_M = e^{-i\frac{2\pi}{M}}$ und $W_N = e^{-i\frac{2\pi}{N}}$. Aufgrund der Symmetrie der obigen Formel kann 2D-DFT in zwei 1D-DFT-Berechnungen zerlegt und die 1D-DFT kann auch durch FFT vereinfacht werden.

2.1.2 Constant False Alarm Rate (CFAR)

Wie oben erwähnt, kann man das Spektrum erhalten, indem man das Signal mit FFT verarbeitet, und ein Peak repräsentiert eine Signalfrequenz, die sich vom Rauschen unterscheidet. In praktischen Anwendungen kann der Computer jedoch einen „Peak“ nicht immer sicher identifizieren, und manchmal ist ein „Peak“ nicht offensichtlich, wenn die Rauschleistung und die Signalleistung ähnlich sind.

In diesem Fall verwendet man CFAR als Peak-Detektionsalgorithmus. Das Prinzip der CFAR-Erkennung selbst ist sehr einfach. Sie betrachtet andere Probenpunkte in der Nähe eines Probenpunktes als Rauschen und vergleicht den Wert dieses Probenpunktes mit dem Rauschen, um zu beurteilen, ob der Probenpunkt ein nützlicher Signalfunkt oder Rauschpunkt ist.

Einer der an den häufigsten verwendeten CFAR-Algorithmen ist „Cell-Averaging-CFAR“ (CA-CFAR). Die Rauschleistung in der Nähe des erkannten Punktes wird zunächst nach folgender Formel berechnet: ^[10]

$$P_n = \frac{1}{N} \sum_{m=1}^N x_m \quad (2.13)$$

Darunter sind N die Anzahl der in der Nähe des erkannten Punktes ausgewählten Stichprobenpunkte und x_m der Wert jedes Stichprobenpunktes.

Diese Leistung sollte nicht direkt als Schwellenwert betrachtet werden, sondern mit einem entsprechenden Koeffizienten multipliziert werden. Der Koeffizient wird als Schwellenfaktor bezeichnet und in α ausgedrückt. Seine Berechnungsformel ist:

$$\alpha = N(P_{fa}^{-\frac{1}{N}} - 1) \quad (2.14)$$

wobei P_{fa} die gewünschte Fehlalarmrate ist.

Der Schwellenwert der CFAR-Erkennung kann dann nach folgender Formel berechnet werden:

$$T = \alpha P_n \quad (2.15)$$

Bei der CFAR-Erkennung werden andere Probenpunkte für die Berechnung der Lärmleistung in der Nähe der erkannten Punkte nicht willkürlich ausgewählt. Diese Stichprobenpunkte, sogenannte Zellen, fallen in zwei Kategorien: Schutzzellen und Trainingszellen.



Abb. 2.3: Zellenselektion von 1D-CFAR

Die obige Abbildung (Abb. 2.3) zeigt die Verteilung der Zellen für CFAR-Erkennung eines eindimensionalen Signals. Zelle im Test (cell under test) ist der Probenpunkt, den man ermitteln will. Schutzzellen werden auf beiden Seiten des zu ermittelnden Punktes verteilt, um den Einfluss der Leistung der Probenahmestelle auf die Berechnung der Lärmleistung zu schützen. Trainingszellen sind ein Satz von Stichprobenpunkten, die für die Berechnung der Lärmleistung ausgewählt werden. Andere Probenpunkte sind nicht mit diesem Test verbunden.

Wenn die Probenpunkte ausgewählt werden, verwendet man den obigen Algorithmus, um die Rauschleistung der Probenpunkte in der Nähe des erkannten Punktes zu berechnen, den Schwellenwert zu generieren und ihn mit der Leistung des erkannten Punktes zu vergleichen. Wenn die Größe der Probenpunkte einen bestimmten Schwellenwert überschreitet, können die Probenpunkte als Ziel festgelegt werden. ^[11]

Wenn diese Erkennung abgeschlossen ist, wählt man den nächsten Punkt des aktuell erkannten Probenahmepunktes als den zu testenden Punkt und wiederholt den oben beschriebenen Prozess.

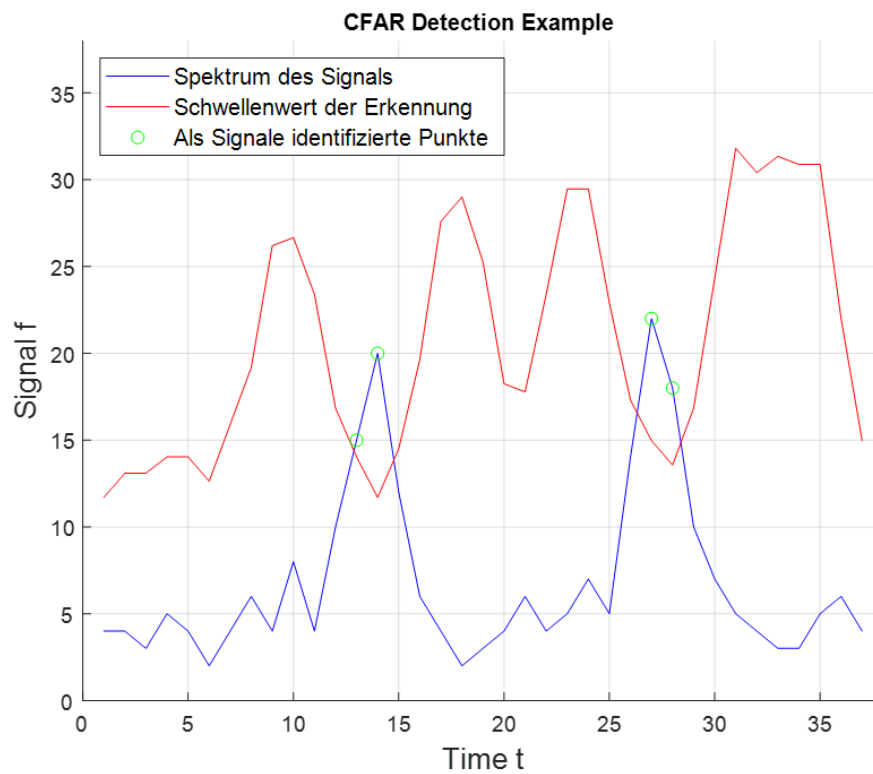


Abb. 2.4: Beispiel für 1D-CFAR

Die obige Abbildung (Abb. 2.4) ist ein Beispiel für die CFAR-Erkennung. Die blaue Linie ist das ursprüngliche Signal, die rote Linie ist die berechnete Rauschschwelle und die grünen Kreise sind die anerkannten effektiven Signalpunkte.

Die zweidimensionale CFAR-Erkennung ähnelt der eindimensionalen Methode. Der einzige Unterschied ist die Auswahl der Zellen. Im zweidimensionalen Fall umgeben Schutzzellen und Trainingszellen die Zelle im Test.

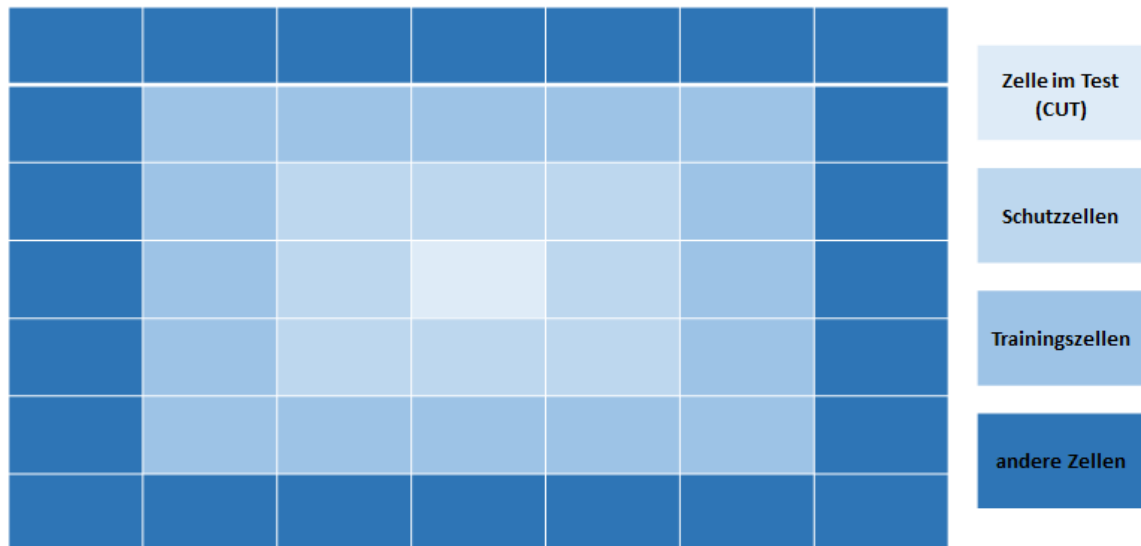


Abb. 2.5: Zellenselektion von 2D-CFAR

In diesem Fall werden alle Signalpunkte in diesem zweidimensionalen Bereich verwendet, um die Rauschleistung zu berechnen. Und die Rauschleistung wird verwendet, um zu bestimmen, ob der Signalpunkt in der Mitte ein nützlicher Signalpunkt ist.

2.1.3 Zero-Padding

Wenn die Anzahl der Abtastpunkte in dem Zeitbereich zu klein ist, sind die Spitze und die Veränderung des Signalspektrums nach der schnellen Fourier-Transformation nicht offensichtlich, so dass es schwierig ist, die genauen Informationen des Signals zu analysieren. In diesem Fall kann man nach dem Berechnungsgrundsatz der diskreten Fourier - Transformation die spektrale Auflösung verbessern, indem man nach den Signalpunkten einige Nullpunkten in dem Zeitbereich hinzufügt. ^[12]

Angenommen, es gibt ein Signal in dem Zeitbereich, dessen Funktionsausdruck

$$f(t) = \sin(200\pi t) + \sin(220\pi t) \quad (2.16)$$

ist. Wenn die Anzahl der ausgewählten Abtastpunkte gering ist (z. B. 15 Punkte mit Abtastfrequenz auf 1000Hz), wird das Signal im Zeitbereich wie unten (Abb. 2.6) angezeigt:

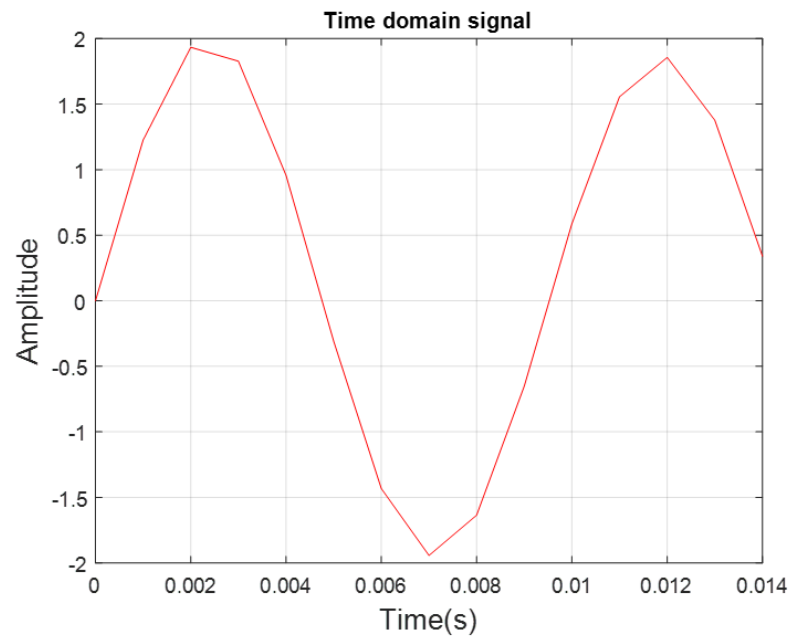


Abb. 2.6: Zeitdomänensignal

Wenn man auf diesen Signalverlauf eine FFT anwendet, sehen die Ergebnisse wie in der folgenden Abbildung (Abb. 2.7) aus:

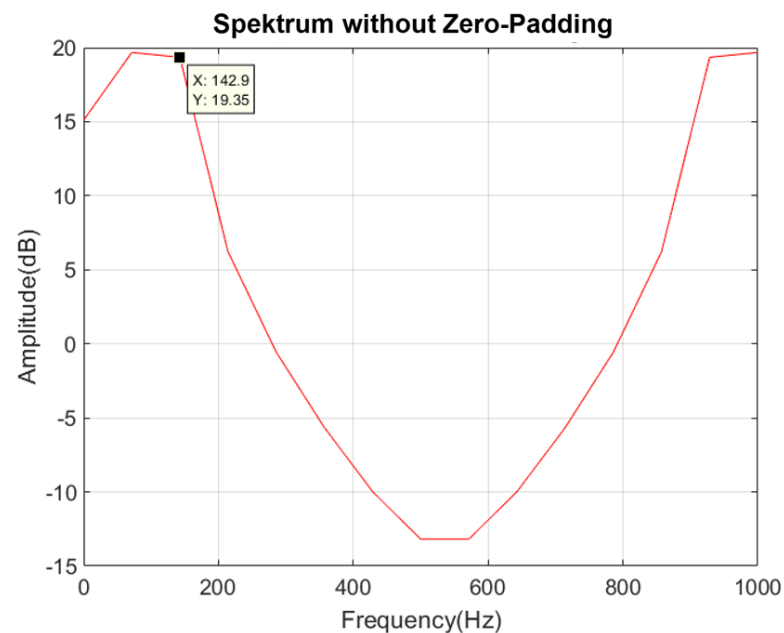


Abb. 2.7: Spektrum ohne Zero-Padding

Man kann erkennen, dass die Spitzenfrequenz des Spektrums bei ca. 100 Hz auftritt, aber die Auflösung ist sehr gering.

Als nächstes addiert man nach dem Prinzip des Zero-Paddings mehrere Nullpunkte (z. B. 45 Punkte) nach dem Zeitbereichssignal:

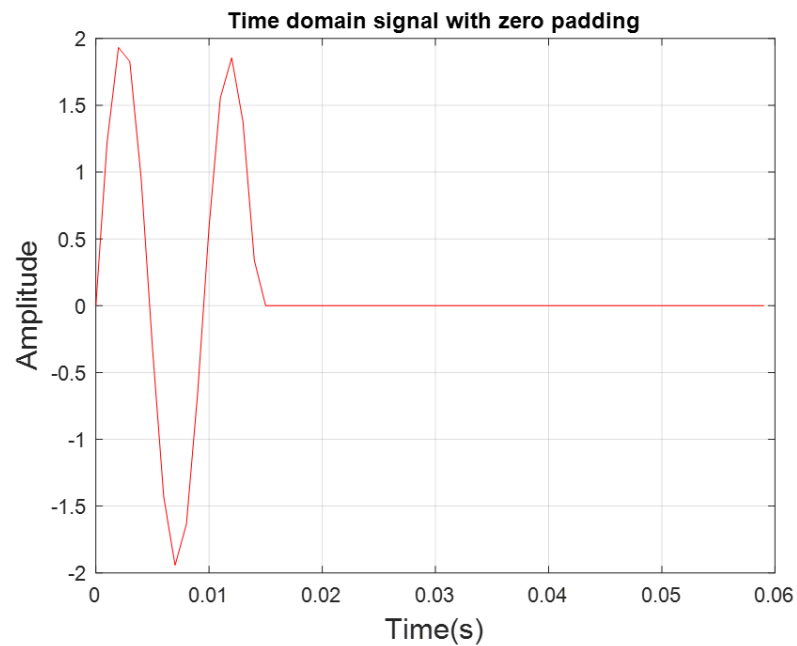


Abb. 2.8: Zeitdomänensignal mit Zero-Padding

In diesem Fall erhält man nach Durchführung einer FFT das folgende Spektrum:

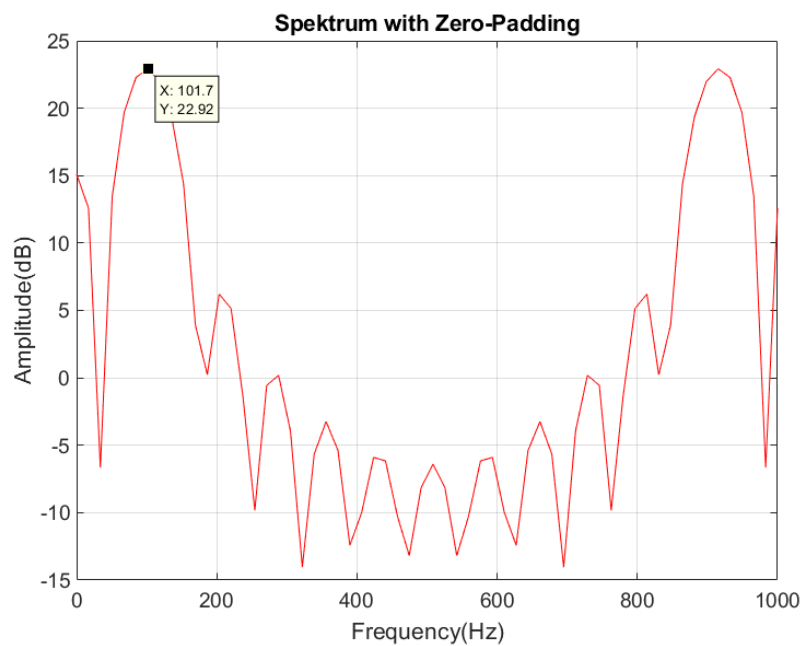


Abb. 2.9: Spektrum mit Zero-Padding

Im Gegensatz zum vorherigen Spektrum ist die Auflösung wesentlich höher. Es gibt mehr Datenpunkte über den dargestellten Frequenzbereich, sodass man den Peak des Spektrums besser ermitteln kann. In der Grafik liegt der Peak bei etwas über 100 Hz, was auch mit dem Ausdruck des Zeitbereichssignals übereinstimmt.

Bei dieser Methode treten jedoch einige falsche Peaks im Spektrum auf, da die neu hinzugefügten Nullpunkte und die ursprünglichen Signalkpunkte eine zusätzliche, nicht vorhandene Sinusfunktion generieren können.

2.2 Radarmessprinzipien

In den folgenden Abschnitten werden die Messprinzipien des FMCW-MIMO-Radars erläutert, einschließlich der Messung von Entfernung, Geschwindigkeit und Winkel des Objekts zum Radar.

2.2.1 Radarmessprinzip für Abstandmessung

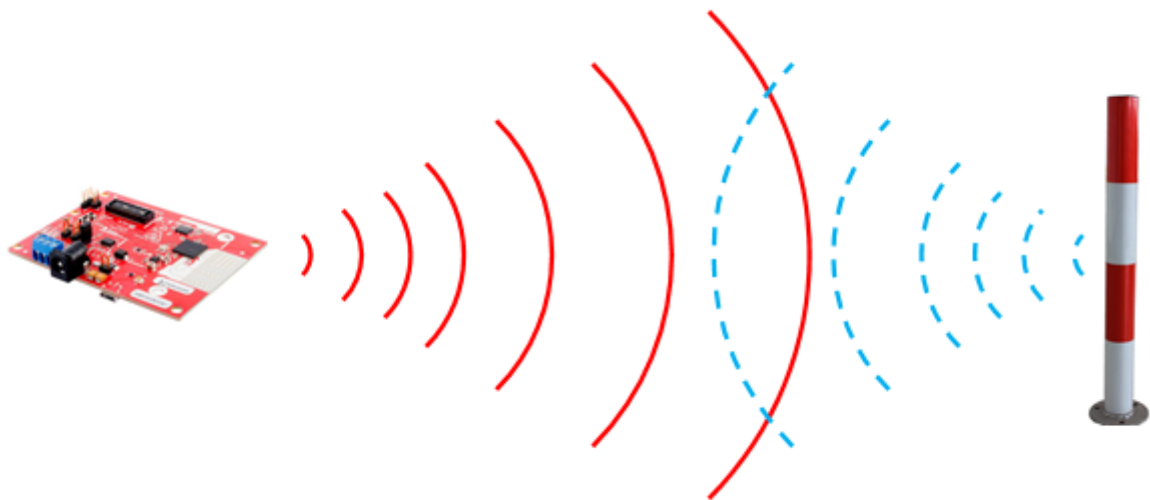


Abb. 2.10: Messprinzip des FMCW-Radarmoduls

Bei den Messungen wird ein FMCW-Radarmodul verwendet. Im grundlegenden Messverfahren sind alle Radarsysteme ähnlich, es wird ein Signal von der TX-Antenne des Radars übertragen, durch die Hindernisse reflektiert und von der RX-Antenne des Radars empfangen. Im Allgemeinen verwendet das Radar die Zeitdifferenz zwischen dem gesendeten und dem empfangenen Signal, um die Entfernung des Objekts zu berechnen. Aber eine direkte Laufzeitmessung ist aufwändig und einfacher ist eine indirekte Laufzeitmessung. Das

Verfahren ist als FMCW (Frequency Modulated Continuous Wave) bekannt. Statt des Vergleichs der Zeiten zwischen Sendesignal und Empfangsecho werden beim FMCW-Radar die Frequenzen zwischen dem Sendesignal und dem Empfangsecho verglichen. ^[13]

Charakteristisch für das FMCW-Radar ist, dass das übertragene Signal, das Chirp genannt wird, eine hochfrequente kontinuierliche Welle ist, deren Frequenz moduliert wird, z.B. linear mit der Zeit ansteigend (Dreieck-Modulation). ^[14] Die Frequenz der Rückwärtswelle folgt demselben Muster, aber mit einem Zeitversatz, der zur Berechnung der Objektdistanz verwendet werden kann.

Wenn das Signal zum Radar zurückkehrt, verarbeitet ein sogenannter Mixer das übertragene Signal und das empfangene Signal. Wenn das übertragene Signal in Form von $x_1 = \sin(\omega_1 t + \varphi_1)$ ist, ist das empfangene Signal auch in Form von $x_2 = \sin(\omega_2 t + \varphi_2)$. Das von Mixer verarbeitete Ausgangssignal ist wie folgt:

$$x_{out} = \sin[(\omega_1 - \omega_2)t + (\varphi_1 - \varphi_2)] \quad (2.17)$$

ω_1 und ω_2 sind die momentanen Frequenzen von Übertragungssignalen und Empfangssignalen. Dieses verarbeitete Signal wird IF-Signal („intermediate frequency“) genannt. Da die Frequenzen der Übertragungs- und Empfangssignal mit der gleichen Steigung linear zunehmen, ist das IF-Signal ein Signal mit konstanter Frequenz. Der oben beschriebene Prozess lässt sich anhand des folgenden Diagramms (Abb. 2.11, Abb. 2.12) veranschaulichen:

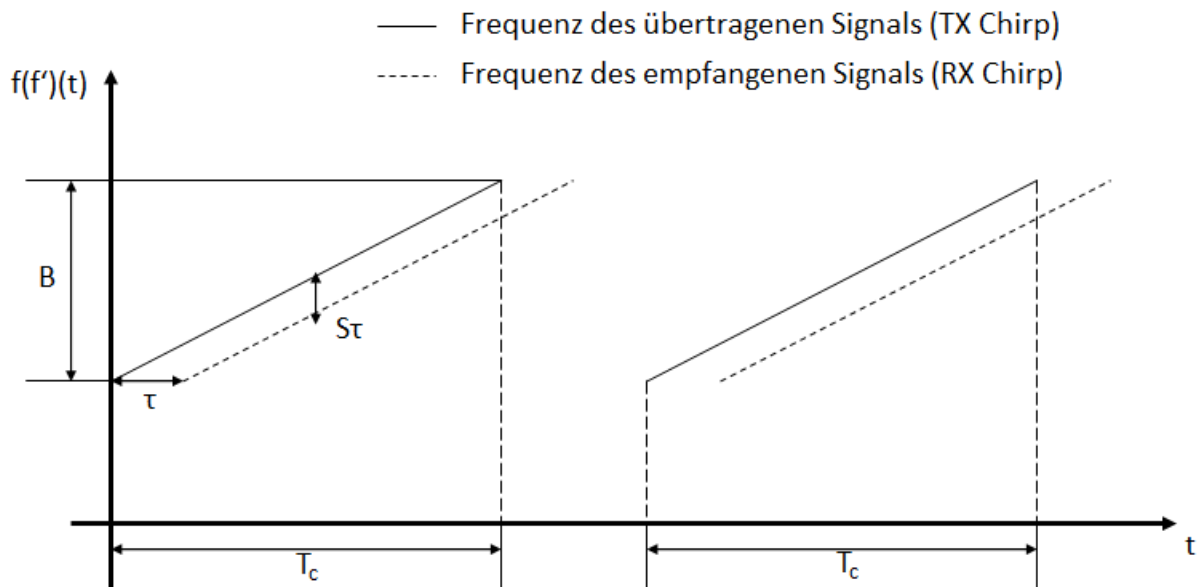


Abb. 2.11: Änderungen der Frequenzen des TX-Chirps und des RX-Chirps mit der Zeit

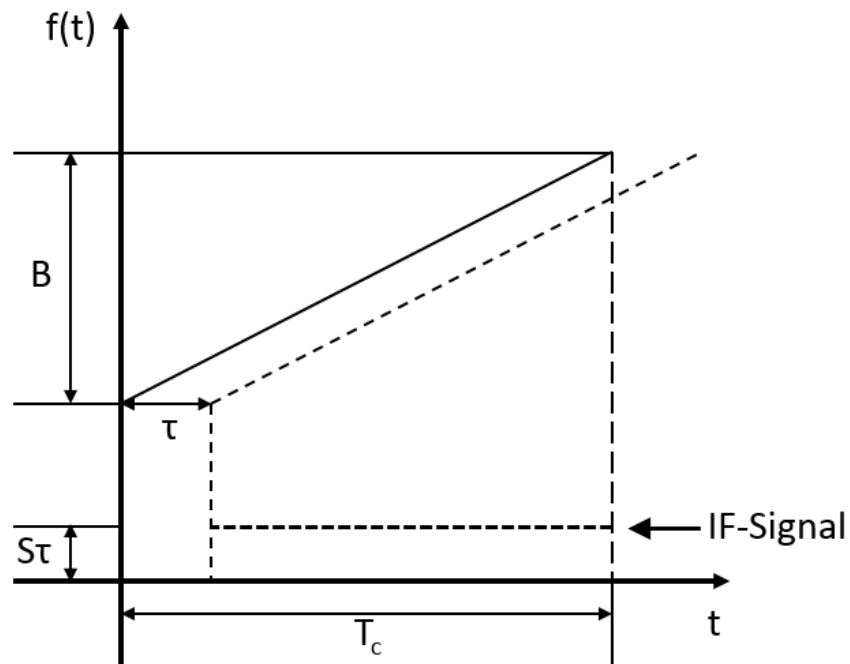


Abb. 2.12: Frequenz des IF-Signals über die Zeit

Da das IF-Signal wiederum eine sinusförmige Welle ist, kann man die Frequenz des IF-Signals ($\omega_1 - \omega_2$) durch Fourier-Transformation berechnen. Anhand der Eigenschaften der dreieckigen Welle kann der Abstand vom Objekt zum Radar nur durch den Frequenzunterschied ($\omega_1 - \omega_2$) bestimmt werden.

In der Praxis muss das zurückgekommene Signal zuerst von einem ADC verarbeitet werden, der darauffolgend angewandte Algorithmus zur Frequenzanalyse ist der FFT-Algorithmus. Das Spektrum nach der Bearbeitung der FFT wird Range-FFT genannt.

Wenn es eine Spitze am n-ten Punkt des Spektrums gibt, bedeutet dies, dass es ein Objekt in der entsprechenden Entfernung gibt, und die Entfernung zwischen dem Objekt und dem Radar folgende ist: ^[15]

$$d = \frac{nF_s C}{2SN} \quad (2.18)$$

wobei F_s die Abtastfrequenz, C die Lichtgeschwindigkeit, S die Steigung der Chirpfrequenz und N die Anzahl der Abtaststellen sind.

2.2.2 Radarmessprinzip für Geschwindigkeitsmessung

Da das von Mixer verarbeitete IF-Signal ein sinusförmiges Signal ist, enthält es nicht nur die Frequenzinformationen, sondern auch die Phaseninformationen. Wie oben erwähnt, kann man die Frequenzinformationen verwenden, um den Abstand zwischen dem Objekt und dem Radar zu berechnen. Um die Geschwindigkeit des Objekts gegenüber dem Radar zu berechnen, werden die Phaseninformationen verwendet.

Angenommen, dass man viele Chirps mit gleicher Anfangsphase (z.B. Phasenwinkel ist 0) in gleichen Intervallen über eine TX-Antenne sendet. Wenn sich das Objekt mit einer bestimmten Geschwindigkeit gegenüber dem Radar bewegt, ändert sich der Abstand zwischen dem Objekt und dem Radar in jedem Zeitintervall leicht. Diese Änderung ist zu klein, um eine Änderung der Frequenz des empfangenen Signals zu verursachen, aber genug, um eine Änderung in der Phase des empfangenen Signals zu verursachen. Wenn man kontinuierlich n Chirps übermittelt, erhält man eine Reihe von sinusförmigen Signalen entsprechend dem gleichen Objekt mit der gleichen Frequenz und der gleichen Phasendifferenz $\Delta\varphi$ zwischen zwei benachbarten Chirps.

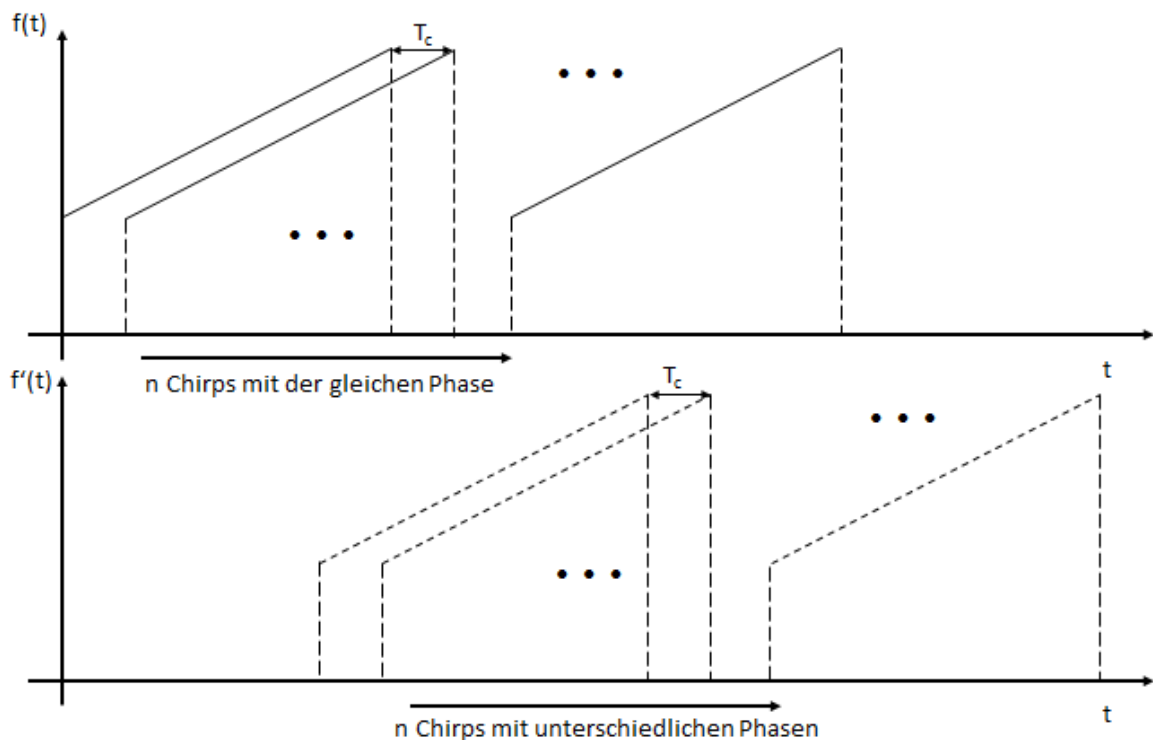


Abb. 2.13: Ein Satz TX-Chirps und ein Satz von einem Objekt reflektierten RX-Chirps

Man kann diesen Signalsatz auch mit FFT verarbeiten, da der Phasenunterschied zwischen zwei benachbarten Chirps konstant ist. Wenn zu einem Zeitpunkt das empfangene Signal $f(1) = A\sin(\varphi_0)$ dem ersten Chirp entspricht, so entsprechen die direkt darauf folgenden Signale

$f(2) = A\sin(\varphi_0 + \Delta\varphi)$, $f(3) = A\sin(\varphi_0 + 2\Delta\varphi)$, ... , $f(n) = A\sin[\varphi_0 + (n - 1)\Delta\varphi]$
jeweils den 2-ten bis n-ten Chirp.

Weil alle empfangenen Signale oben in Form von $f(i) = A\sin[\varphi_0 + (i - 1)\Delta\varphi]$ sind, kann man deren Datenpunkte, die zeitlich direkt aufeinander folgen, auch mithilfe einer FFT analysieren. Nach der FFT entspricht ein Peak im Spektrum dieser $\Delta\varphi$, die mit der Geschwindigkeit des Objekts zusammenhängt.

Wenn es eine Spitze am n-ten Punkt dieses Spektrums gibt, bedeutet dies, dass sich ein Objekt in einer entsprechenden Geschwindigkeit gegenüber dem Radar bewegt, und die Geschwindigkeit Folgende ist: ^[15]

$$v = \frac{nC}{2fT_cN} \quad (2.19)$$

wobei C die Lichtgeschwindigkeit, f die Signalfrequenz, T_c das Intervall zwischen zwei Chirps und N die Anzahl der Chirps sind.

2.2.3 Radarmessprinzip für Winkelmessung

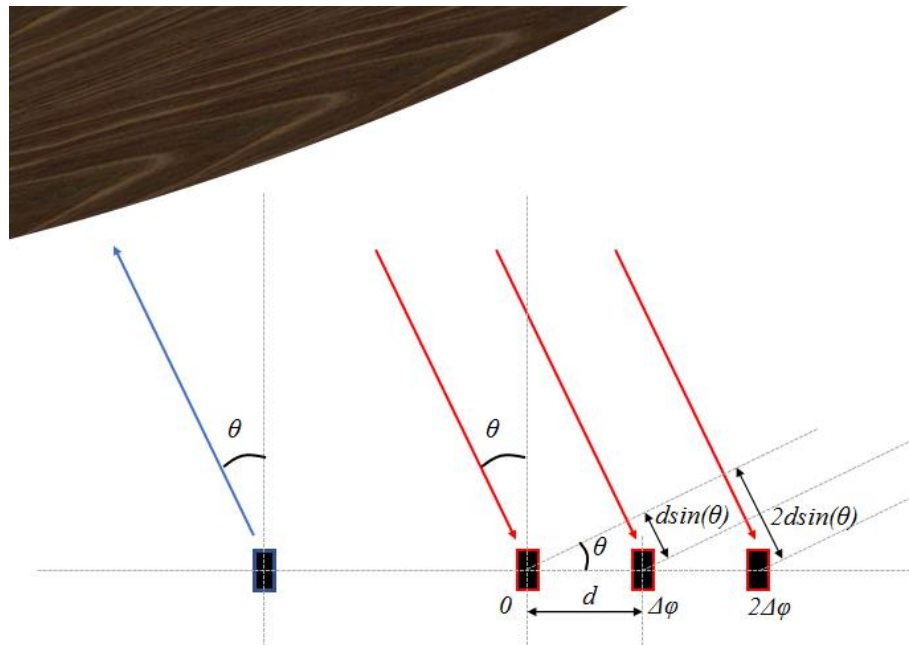


Abb. 2.14: Winkelbeziehung zwischen der Sendeantennen, dem Objekt und der Empfangsantennen

Angenommen, der Abstand zwischen zwei RX-Antennen ist d. Die Signale, die von der gleichen TX-Antenne übertragen werden, benötigen eine unterschiedliche Zeit, um die RX-

Antennen zu erreichen, nachdem sie vom Objekt reflektiert wurden. Diese Zeitdifferenz wird für genügend weit entfernte Objekte durch $\Delta t = \frac{d \sin(\theta)}{c}$ berechnet.

Da dieser Zeitunterschied sehr klein ist, darf man annehmen, dass die Signalfrequenzen, die bei verschiedenen RX-Antennen ankommen, gleich sind. Das bedeutet, dass die Chirps von verschiedenen Antennen als sinusförmige Signale mit nur einem kleinen Phasenunterschied in diesem kleinen Zeitraum betrachtet werden.

Wenn die Frequenz des Chirps zu diesem Zeitpunkt f ist, wird der Phasenunterschied zwischen den Signalen, die an verschiedenen RX-Antennen ankommen, so berechnet: ^[15]

$$\Delta\varphi = 2\pi f \Delta t = \frac{2\pi d \sin(\theta)}{\lambda} \quad (2.20)$$

Genau wie bei der Geschwindigkeitsmessung kann man die diskrete Fourier -Transformation aller Signale (mit FFT-Funktionen in Matlab) berechnen, nachdem man die N Probenpunkte aller Antennen erhalten hat. Die Spitze der Kurve kann benutzt werden, um den Phasenunterschied $\Delta\varphi$ zu finden. Nach dem Erhalt der $\Delta\varphi$ kann man mit der Formel den Winkel θ berechnen: ^[15]

$$\theta = \sin^{-1}\left(\frac{\lambda \Delta\varphi}{2\pi d}\right) \quad (2.21)$$

Angenommen, N ist die Anzahl der RX-Antennen und die Phasendifferenz $\Delta\varphi$ entspricht dem i -ten Punkt im Spektrum, dann ist $\Delta\varphi = i2\pi fT = \frac{i2\pi}{N}$, was bedeutet, dass $\theta = \sin^{-1}\left(\frac{\lambda \Delta\varphi}{2\pi d}\right) = \sin^{-1}\left(\frac{\lambda i}{dN}\right)$ ist. Die Abtastauflösung des Winkels beträgt also:

$$\Delta\theta = \sin^{-1}\left(\frac{\lambda}{dN}\right) \quad (2.22)$$

Dies bedeutet, dass die Winkelauflösung des Radars umso höher ist, je mehr RX-Antennen vorhanden sind.

3. Radarmessverfahren und Ermittlung der Hindernisposition

Im vorigen Kapitel werden die Prinzipien der Radarmessung und die Algorithmen der Signalverarbeitung vorgestellt. Dieses Kapitel beschreibt den Messprozess des Radars und die Verwendung der Messungen zur Bestimmung der Position von Hindernissen.

3.1 Aufbau und Messergebnisse des Radars

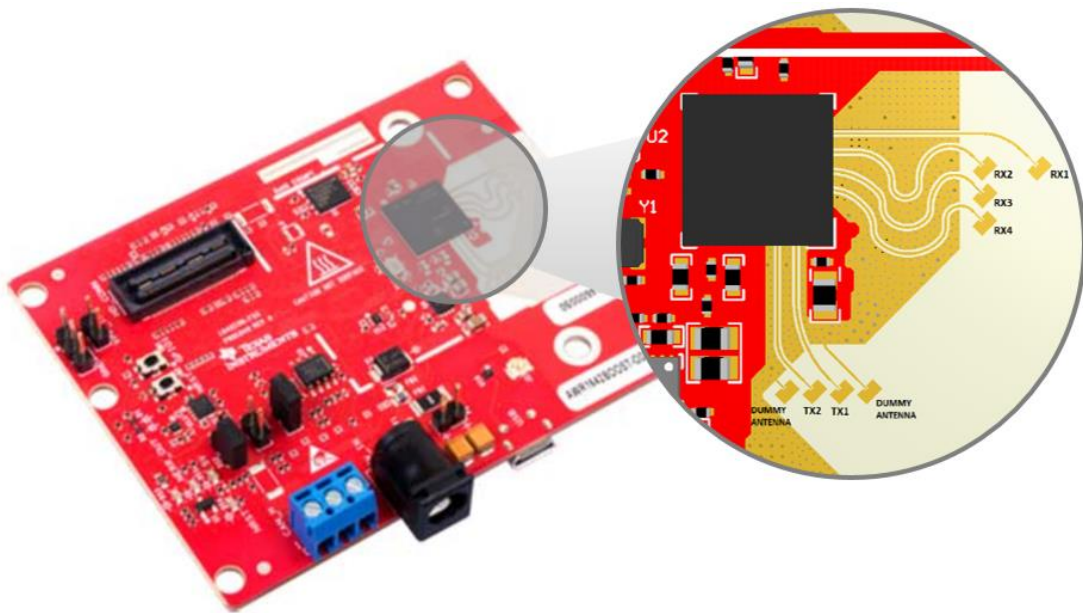


Abb. 3.1: Anordnung der Antennen des Radarmoduls „AWR1642BOOST-ODS“

Das obige Bild (Abb. 3.1) ^[16] zeigt das Radar Entwicklungsmodul, das bei der Erkennung von Hindernissen eingesetzt wird. Er besteht aus 2 Sendeantennen (TX1, TX2), 4 Empfangsantennen (RX1, RX2, RX3, RX4), Chips und anderen Schaltungselementen. Damit das Radar richtig funktioniert und die Rohdaten an den Computer übermittelt, braucht man einen DCA1000EVM-Datenerfassungsstecker, ein Stromkabel, zwei USB-Kabel und ein Netzkabel. Der DCA1000EVM kann Daten in Echtzeit erfassen und streamen, sodass Daten über Netzkabel vom Radar zum Host übertragen werden können. ^[17] Die Verbindung zwischen den beiden und dem Host ist wie folgt:

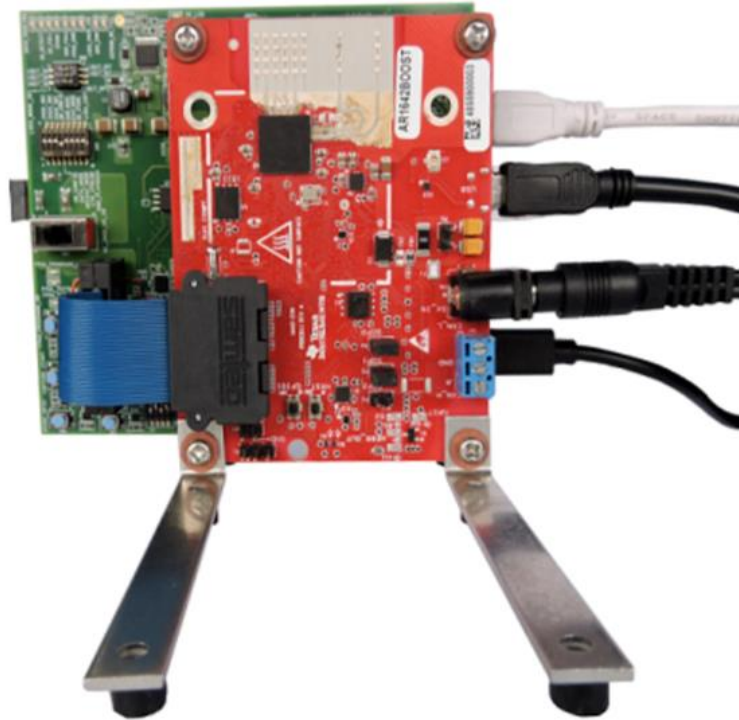


Abb. 3.2: Die Verbindung zwischen DCA1000EVM und Radar

Das folgende (Abb. 3.3) ist ein schematisches Diagramm des Radars. ^[16]

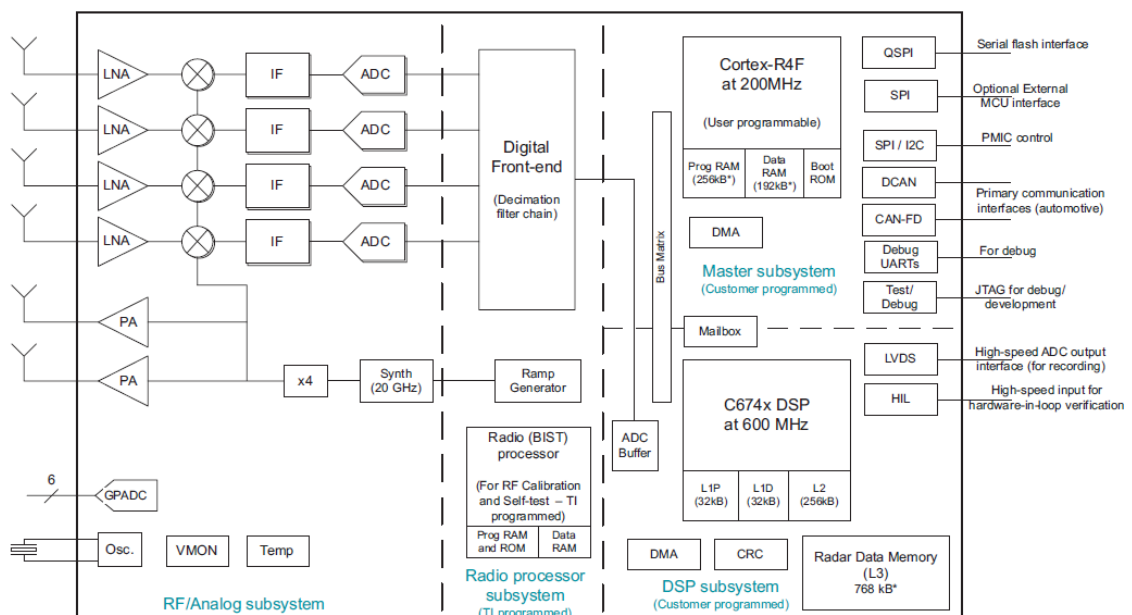


Abb. 3.3: Schematisches Diagramm des Radars

Eine Software namens mmWaveStudio wird für die Messung verwendet. Über diese Software kann man Befehle vom Host an das Radar senden, um den Radarmessprozess zu steuern. In

dieser Software kann man steuern, welche Antennen Signale senden, wie viele Chirps bei jeder Messung gesendet werden usw.

Für die Interpretation der Messergebnisse muss man die vom Radar übertragenen Signale kennen. Wie in der Abbildung unten (Abb. 3.4) gezeigt, beginnt die Sendefrequenz des Radarsignals bei 77 GHz und steigt mit einer Geschwindigkeit von 36,355 MHz/μs. Die Messung besteht aus acht Frames, ein Frame aus 128 Chirps. Bei dieser Messung hat jeder Chirp in einem Frame die gleichen Eigenschaften. Während eines Zyklus eines Chirps beginnt die Probenahme um 6ms und dauert bis 110ms mit insgesamt 1024 Probenahmestellen. Es wird auch gezeigt, dass die Anfangsphase aller Signale, die von TX-Antennen übertragen werden, 0 ist.

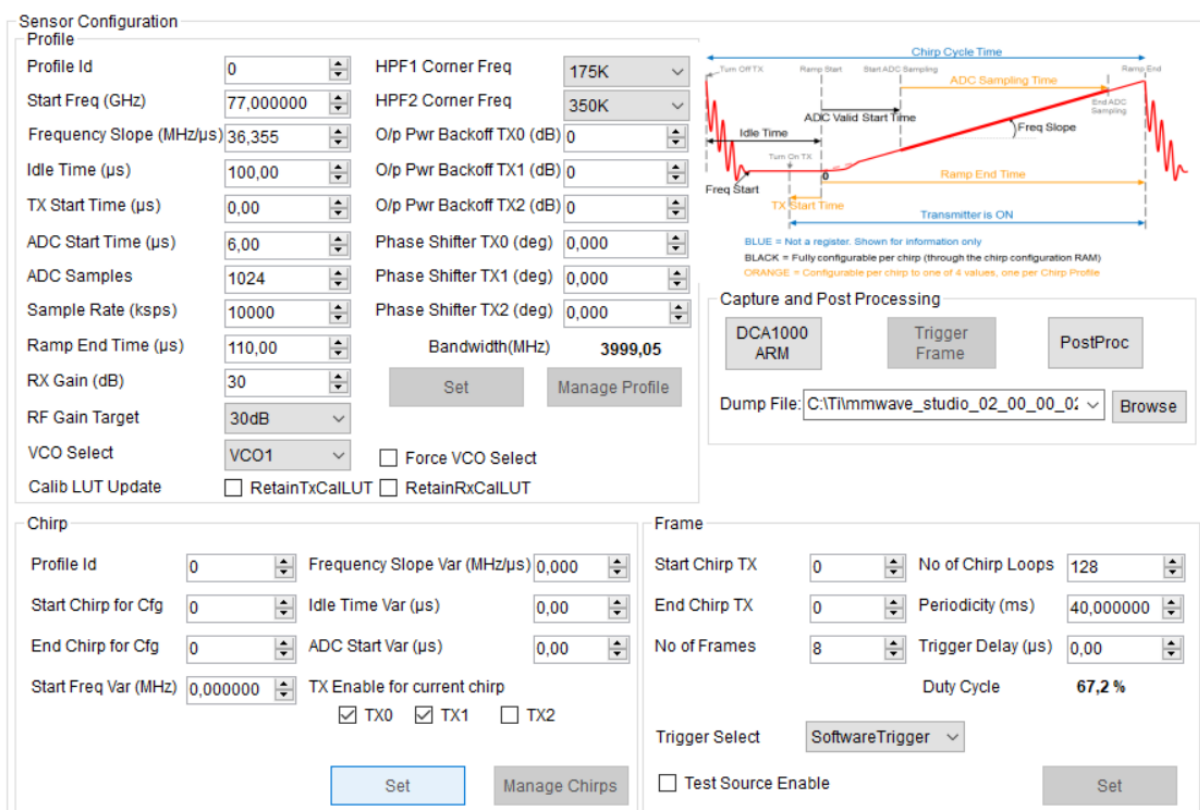


Abb. 3.4: Parametereinstellungen der Radarmessung

Das spezifische Timing eines Chirpsyklus ist in der folgenden Abbildung (Abb. 3.5) dargestellt:

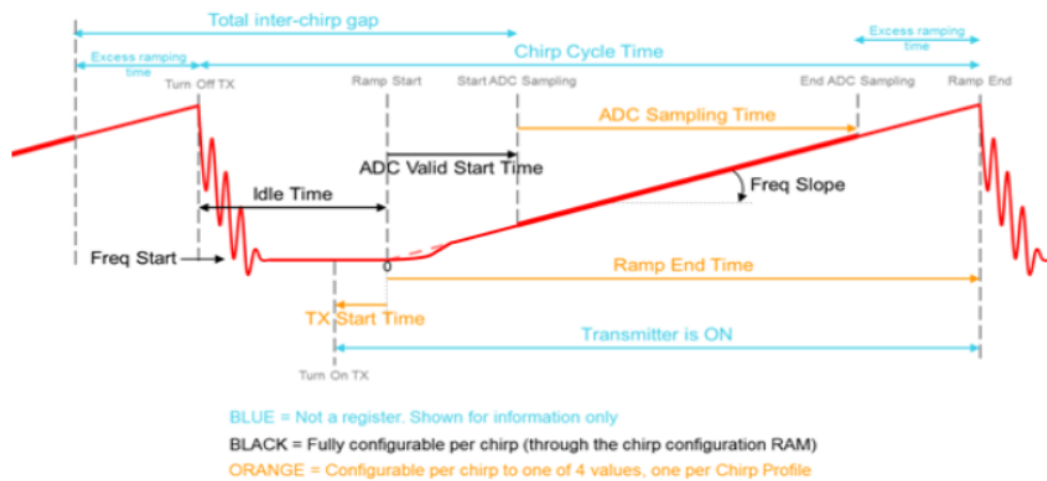


Abb. 3.5: Timing eines Chirps

Eine Chirp-Periode besteht hauptsächlich aus zwei Teilen, der „Idle Time“ und der Arbeitszeit der TX-Antennen (in diesem Beispiel als „Ramp End Time“ bezeichnet). Zwischen der „Idle Time“ und der „Ramp End Time“ müssen die TX-Antennen und der AD-Wandler betriebsbereit sein. Nach dem Beginn der Signalübertragung steigt die Chirp-Frequenz während der „ADC Sampling Time“ linear mit einer bestimmten „Slope“ an, und der AD-Wandler führt mehrere Abtastungen durch.

Nachdem alle Parameter eingestellt sind, können die Radarsignale übertragen und die zurückgegebenen Daten verarbeitet werden. Nach der Verarbeitung durch das digitale Frontend des Radars sind die vom Host-Computer empfangenen Datenpakete in folgendem Format:

Datenstruktur	Header	Daten	Header	Daten	...	Header	Daten	Header	Daten
Datenmenge	7	728	7	728	(7+728)	7	728	7	Reste Daten

Abb. 3.6: Binärdateiformat des vom Host empfangenen Signals

Wie in der Abbildung gezeigt, werden die übertragenen Daten auf 728 Bytes verteilt mit 7 Bytes Header davor übertragen, bis der letzte Teil der Datenübertragung beendet ist. Für das k-te Header haben seine sieben Datenbytes das folgende Format ([x] ist die nach unten integrierte Funktion von x):

Bytes des Headers	1	2	3	4	5	6	7
Daten	k	0	1456	0	$1456(k-1) - \left\lfloor \frac{1456(k-1)}{65536} \right\rfloor \times 65536$	$\left\lfloor \frac{1456(k-1)}{65536} \right\rfloor$	0

Abb. 3.7: Headerformat der Binärdatei

Dieser Header hat die Funktion, die Anzahl der Pakete zu zählen und die empfangenen Daten zu überprüfen. Header müssen vor der Datenverarbeitung aus den Daten entfernt werden.

Sobald man die Rohdaten hat, kann man sie mit der Funktion in Matlab in die Form umwandeln, die man benötigt. Um die Datenverarbeitung zu erleichtern, speichert man die Daten in Form des folgenden Diagramms in einer Matrix:

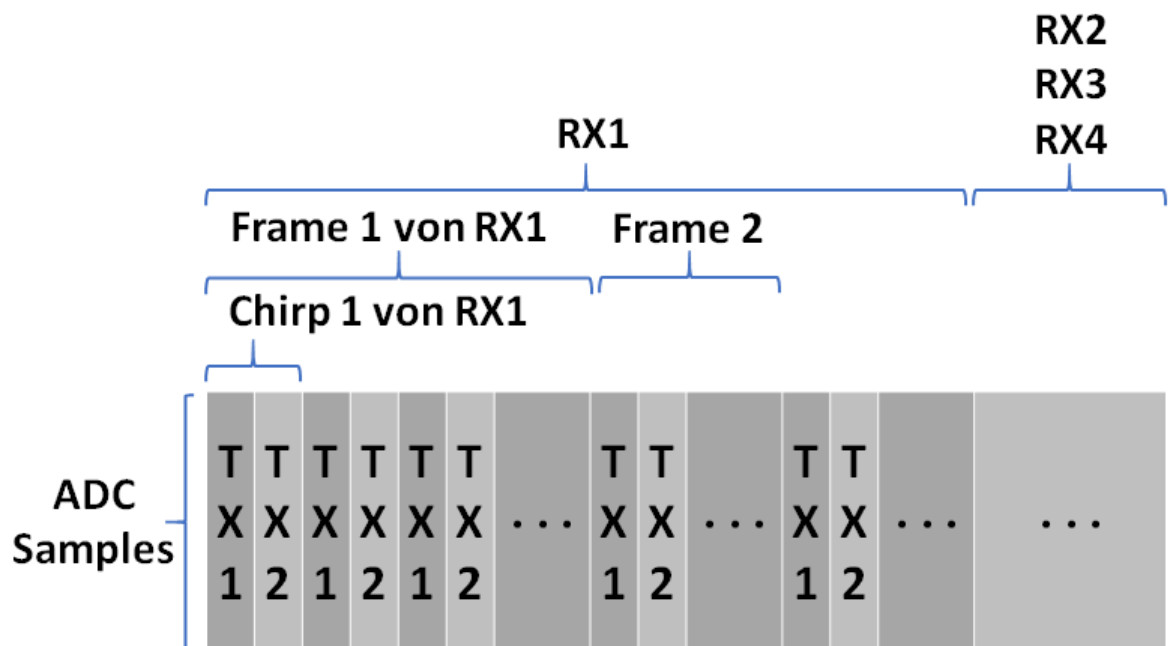


Abb. 3.8: Datenmatrix aus Binärdatei

Wie in der Abbildung dargestellt, sind in jeder Spalte der Matrix die Daten von 1024 Probenahmestellen eines Chirps. Diese Chirps sind in der Reihenfolge TX, Chirps, Frames und RX angeordnet. Es werden weitere Formeln verwendet, um die Messdaten gemäß dieser definierten Reihenfolge zu verarbeiten und zu analysieren.

Man weiß nach der obigen Diskussion, dass die Auflösungen von Distanz, Geschwindigkeit und Winkelachse sind:

$$\Delta d = \frac{F_s c}{2SN} \quad (3.1)$$

$$\Delta v = \frac{c}{2fT_c N} \quad (3.2)$$

$$\Delta \theta = \sin^{-1}\left(\frac{\lambda}{dN}\right) \quad (3.3)$$

Um den Abstand zum Hindernis zu bestimmen, muss man FFT auf einem (oder mehreren) Chirps durchführen, und die über CFAR detektierten Distanzkoordinaten sind dann Positionen von Hindernissen.

Im eigentlichen Berechnungsprozess werden Fensterfunktionen auf das Signal im Zeitbereich angewandt, um den Leck-Effekt zu vermeiden, der in der FFT auftreten kann. Darüber hinaus nimmt man auch den Logarithmus der FFT des erhaltenen Signals und multipliziert ihn mit 20, wodurch man Dezibel als Frequenzeinheit verwendet erhält.

Die folgende Abbildung (Abb. 3.9) zeigt beispielhaft ein Spektrum nach Anwenden einer FFT.

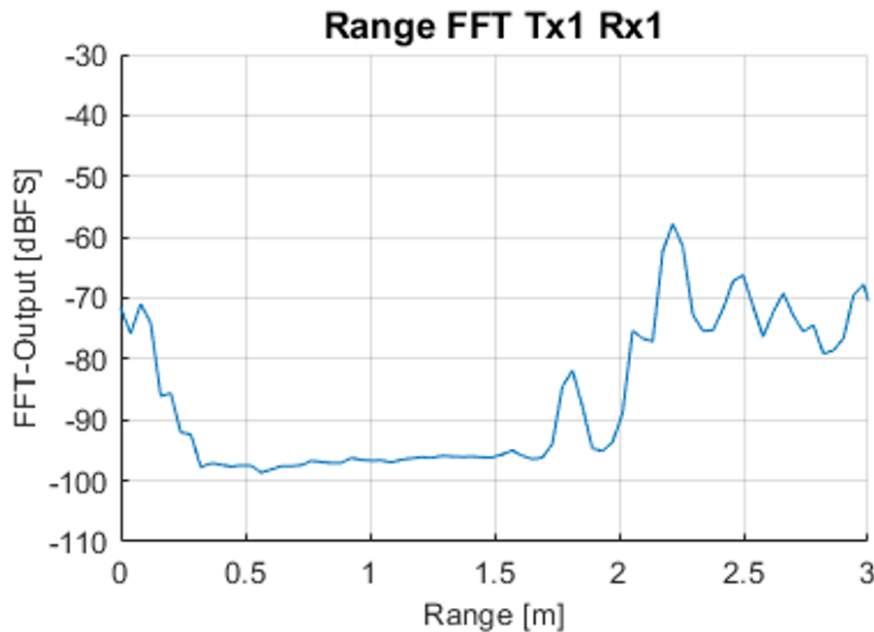


Abb. 3.9: Beispiel eines 1D-Spektrums

In dem Spektrum ist die Einheit der Ordinate dBFS, die wie folgt berechnet wird:

$$P_{out(dBFS)} = P_{out(fft)} - correction_{factor} \quad (3.4)$$

$$correction_{factor} = 20\log_{10} \frac{2^{b_{adc}-1}}{1} + 20\log_{10} N - 20\log_{10} \sqrt{2} \quad (3.5)$$

Wobei b_{adc} die Anzahl der Binärdatenbits ist (bei der Messung ist $b_{adc} = 16$), N die Anzahl der Abtastpunkte ist und $correction_{factor}$ der Parameter ist, der für die Kompensation der FFT-Verstärkung verwendet wird. ^[18]

Für die Radarapplikation ist nicht nur die Abstandsinformation, sondern auch die Winkelinformation des Hindernisses sehr wichtig. Wie in der vorherigen Beschreibung von Fast Fourier Transform erwähnt, kann man FFT verwenden, um Winkelinformationen zu erhalten, indem man die Signale der verschiedenen RX-Antennen vergleicht. Die Ergebnisse dieses Vergleichs können auch in Grafiken gezeigt werden, die 2D-FFT oder Range-Angle-FFT genannt werden. Zunächst wird die Fensterfunktion verwendet, um das Signal im Zeitbereich zu verarbeiten, dann werden die Daten des Abstands und der Winkel durch FFT verarbeitet. Danach soll das Koordinatensystem von zylindrischen in kartesische Koordinaten umgewandelt werden. Das Ergebnis ist in der folgenden Abbildung (Abb. 3.10) zu sehen:

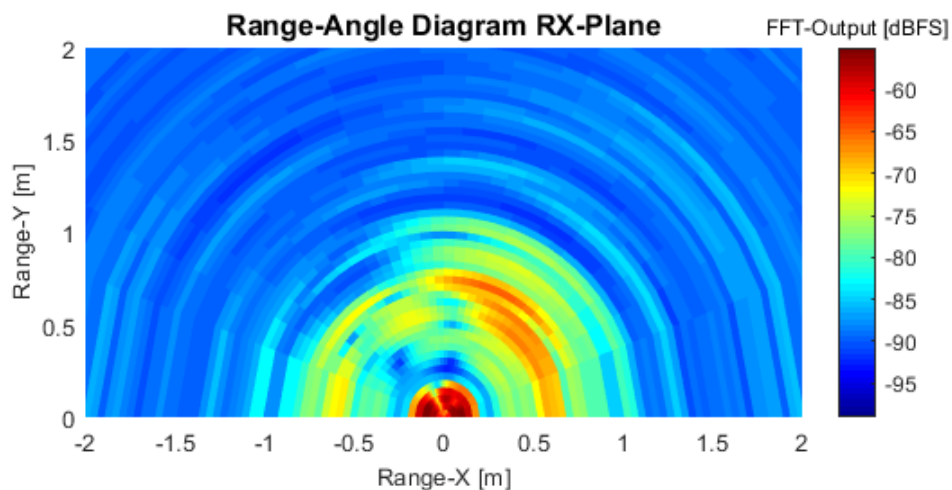


Abb. 3.10: Beispiel eines 2D-Spektrums

Die Abbildung oben (Abb. 3.10) ist ein zweidimensionales Spektrogramm aus der x-y-Ebene. Das tatsächliche Spektrum ist unten (Abb. 3.11) dargestellt. Die x- und y-Achse repräsentieren unterschiedliche Abstände und die z-Achse repräsentiert die Spitze des Spektrums bei unterschiedlichen Abständen. Um die Ergebnisse intuitiver darzustellen, wird später nur das Spektrum aus der x-y-Ebene verwendet.

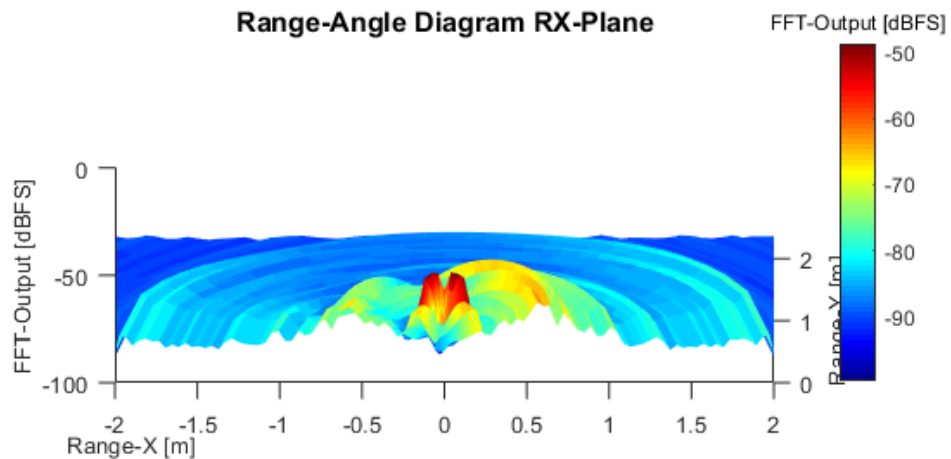


Abb. 3.11: Signalintensität des 2D-Range-Angle -Diagramms

Nachdem man die obigen 1D-FFT- und 2D-FFT-Diagramme erhalten hat, verwendet man CFAR, um die Peaks im Diagramm zu analysieren, die die Entfernung und den Winkel des Hindernisses zum Radar darstellen.

Der Messbereich und die Genauigkeit des Radars hängen mit der Struktur des Radars und der Verteilung der Antennen zusammen. In diesem Radar befinden sich alle Radarantennen in der gleichen Ebene und haben eine feste Position zueinander.

Dies wird im Folgenden genauer betrachtet.

3.2 MIMO und virtuelle Antennen

Im vorherigen Abschnitt werden Methoden zum Messen von Winkeln mit mehreren RX-Antennen erörtert. Und auf dem Radarboard befinden sich 2 TX-Antennen, die parallel zu 2 RX-Antennen stehen:

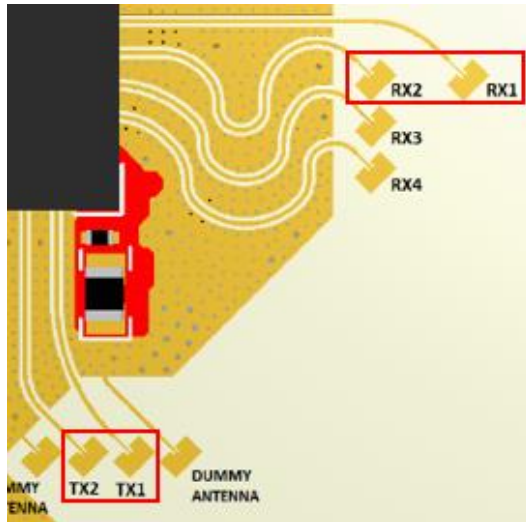


Abb. 3.12: Positionsbeziehung zwischen Radarantennen

Die virtuelle Antennenmethode kann also verwendet werden, um die Messgenauigkeit des Winkels zu verbessern, wie in der folgenden Abbildung (Abb. 3.13) dargestellt:

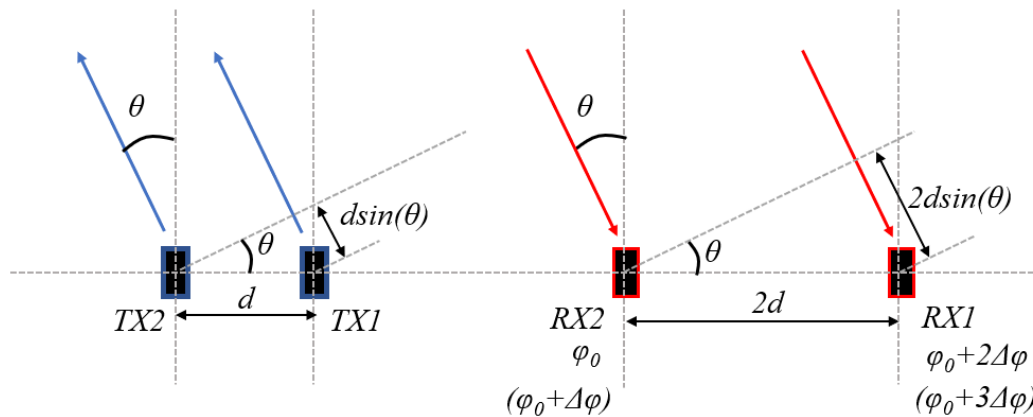


Abb. 3.13: Positionsbeziehung zwischen den Antennen auf dem Modul „AWR1642BOOST-ODS“

Auf der Radarplatine ist der Abstand zwischen 2 RX-Antennen doppelt so groß wie der Abstand zwischen den beiden TX-Antennen. Es wird angenommen, dass die von den beiden TX-Antennen gesendeten Signale dieselbe Anfangsphase haben und das von der TX1-Antenne gesendete Signal die RX2-Antenne mit der Phase φ_0 erreicht. In diesem Fall erreichen gemäß der Abstandsbeziehung in dem Bild das von TX1 gesendete Signal RX1 mit der Phase $2\Delta\varphi$, das von TX2 gesendete Signal RX2 mit der Phase $\Delta\varphi$, das von TX2 gesendete Signal RX1 mit der Phase $3\Delta\varphi$. Diese Phasenbeziehung ist dieselbe wie in dem Fall, in dem ein Signal von einer TX-Antenne gesendet und von vier RX-Antennen empfangen wird. Daher werden die in dieser Kombination entstandenen zusätzlichen Antennen auch als virtuelle Antennen bezeichnet:

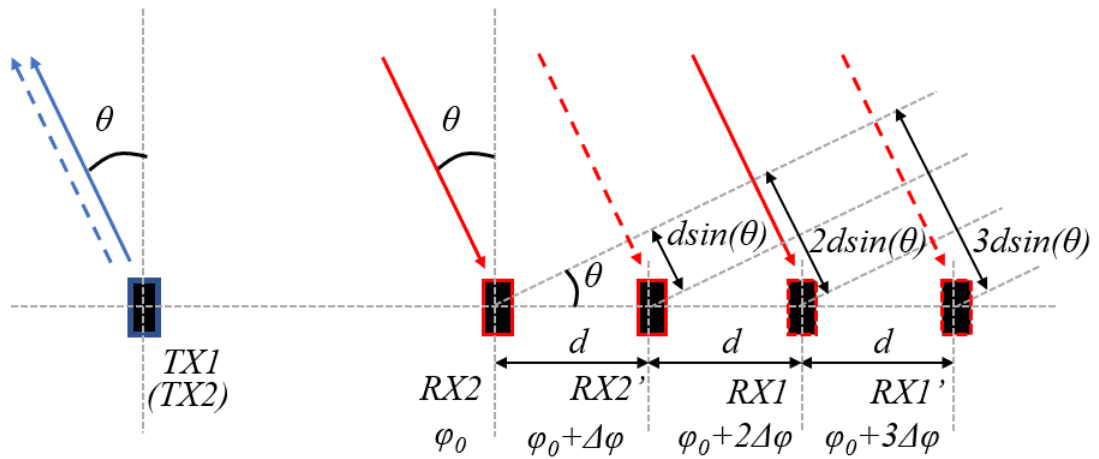


Abb. 3.14: Positionsbeziehung zwischen den virtuellen Antennen auf dem Modul „AWR1642BOOST-ODS“

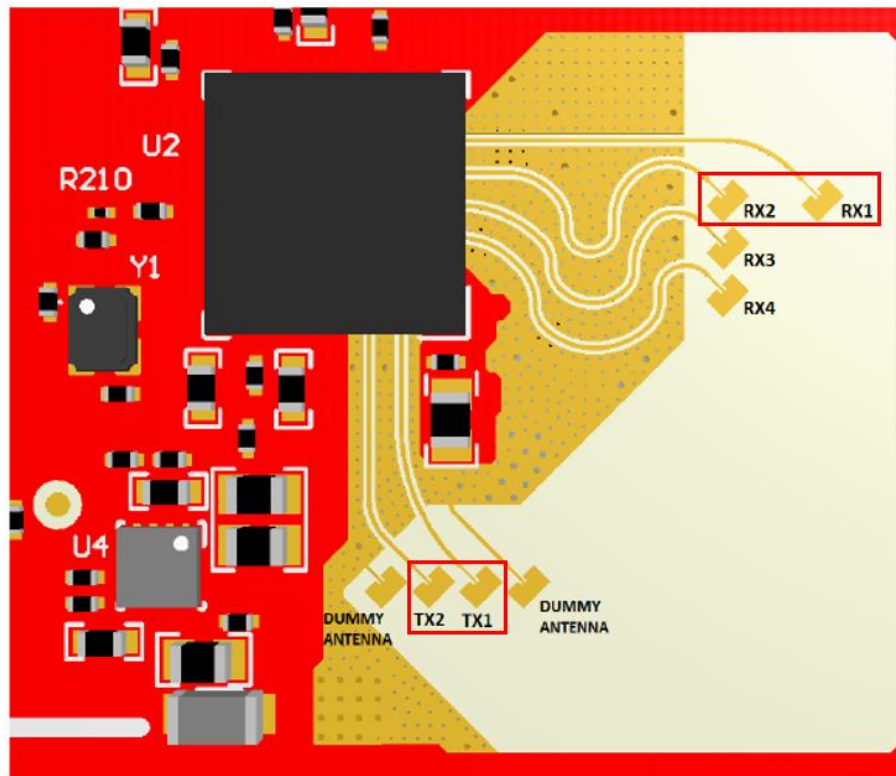


Abb. 3.15: Antennenstruktur des Moduls „AWR1642BOOST-ODS“ und verwendete Antennen für die Berechnung der „TX-Ebene“

Je nach dem Aufbau der Radarplatine liegen vier Antennen TX1, TX2, RX1 und RX2 ungefähr auf derselben geraden Linie. Daher kann man die vier Antennen verwenden, um die virtuellen Antennen zu bilden und den Abstand und Winkel des Hindernisses mit der Linie dieser virtuellen Antennen zu berechnen. Die Ebene, bestehend aus dieser Linie mit TX1, TX2, RX1, RX2 und Hindernissen, wird im Folgenden „TX-Ebene“ genannt.

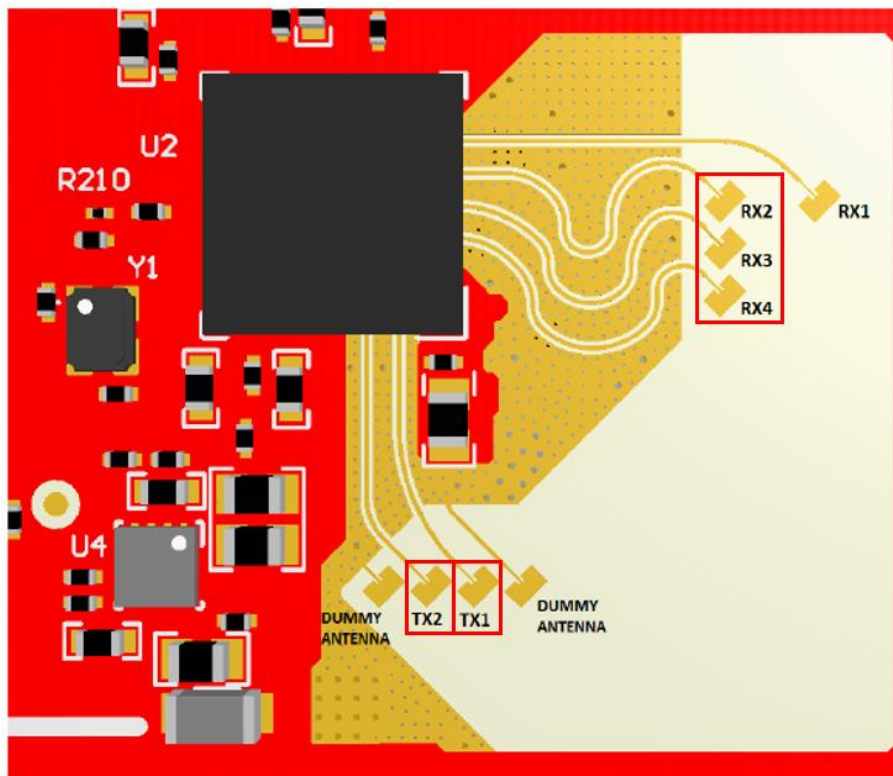


Abb. 3.16: Antennenstruktur des Moduls „AWR1642BOOST-ODS “ und verwendete Antennen für die Berechnung der „RX-Ebene“

Ebenso kann man TX1 beziehungsweise TX2 mit RX2, RX3 und RX4 kombinieren, um den Abstand und Winkel des Hindernisses zur Linie zu berechnen, auf der sich RX2, RX3 und RX4 befinden. Die Ebene, bestehend aus dieser Linie mit RX2, RX3, RX4 und Hindernissen, wird im Folgenden „RX-Ebene“ genannt.

3.3 Die Position des Hindernisses in dem 3D-Raum

Mit den oben erwähnten virtuellen Antennen auf zwei zueinander senkrechten Linien und der TX-Ebene beziehungsweise RX-Ebene kann man die Position des Hindernisses im dreidimensionalen Raum bestimmen. Die folgende Abbildung (Abb. 3.17) zeigt die Positionsbeziehung zwischen einem Hindernis, der TX-Ebene und der RX-Ebene des Radars im Raum:

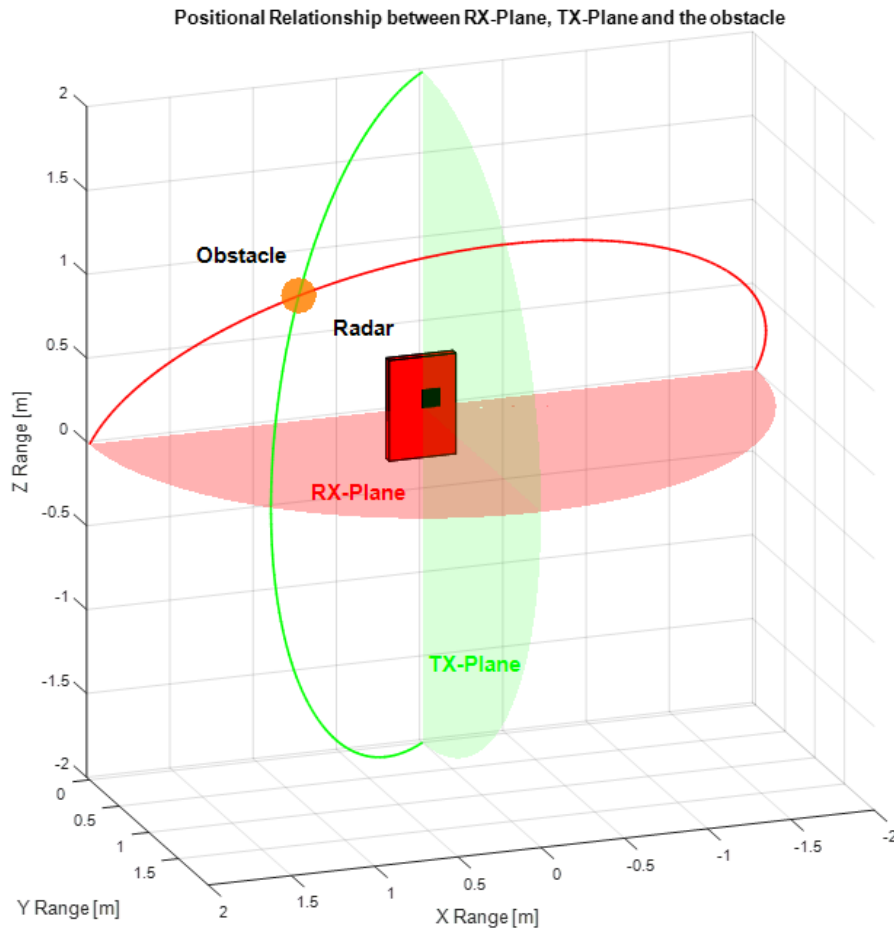


Abb. 3.17: Positionsbeziehung zwischen RX-Ebene, TX-Ebene, Radar und Hindernis

Befindet sich das Hindernis an einer beliebigen Stelle im Raum, spannt es mit den Linien, auf denen sich die Antennen befinden jeweils eine Ebene auf. Diese werden im Folgenden, analog zur „TX-Ebene“ und „RX-Ebene“, „TO-Ebene“ und „RO-Ebene“ genannt. Um die Position des Hindernisses in einem kartesischen Koordinatensystem genau darzustellen, muss man den Winkel zwischen „TO-Ebene“ und „TX-Ebene“, „RO-Ebene“ und „RX-Ebene“ sowie den Abstand vom Hindernis zum Radar kennen. Diese drei Größen werden als α_T , α_R bzw d_O definiert.

Die folgende Abbildung (Abb. 3.18) zeigt die Positionsbeziehung zwischen dem Hindernis und dem Radar aus der Richtung senkrecht zur „RX-Ebene“:

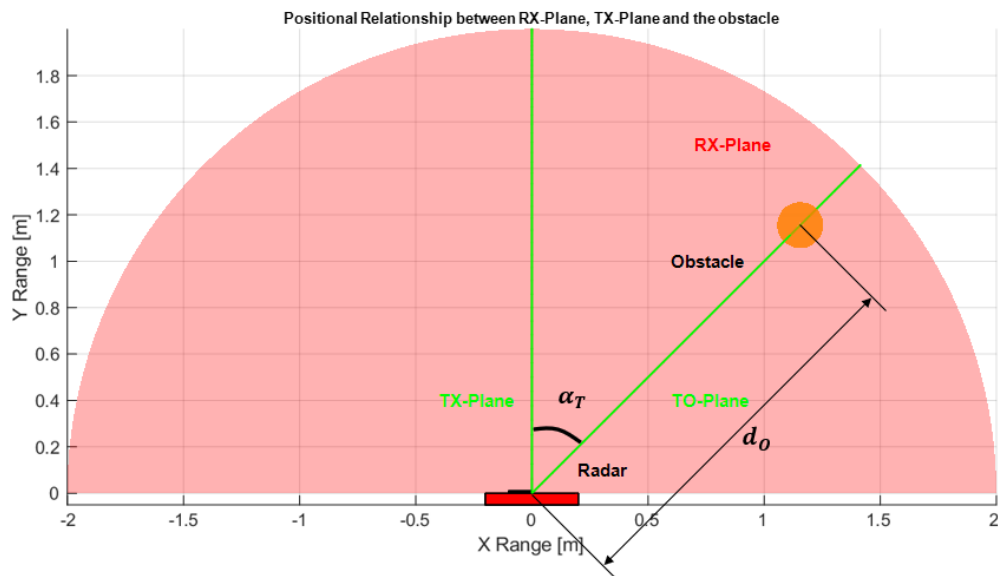


Abb. 3.18: Der Abstand und Winkel des Hindernisses zum Radar in der x-y-Ebene

Die folgende Abbildung (Abb. 3.19) zeigt die Positionsbeziehung zwischen dem Hindernis und dem Radar aus der Richtung senkrecht zur „TX-Ebene“:

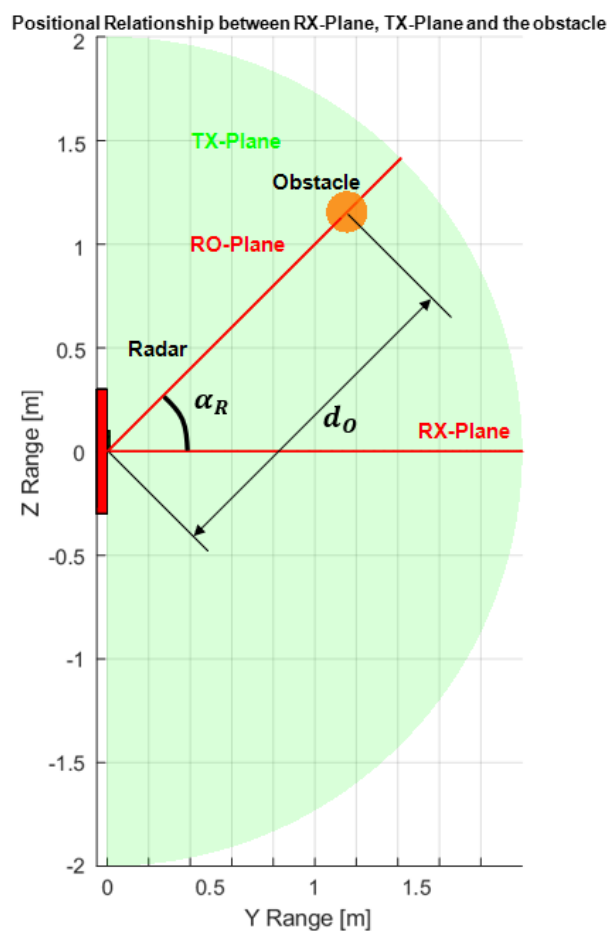


Abb. 3.19: Der Abstand und Winkel des Hindernisses zum Radar in der y-z-Ebene

Entsprechend der geometrischen Beziehung in der Grafik und dem gemessenen Abstand d_o und den gemessenen Winkeln α_T, α_R im kartesischen Koordinatensystem sind die Abstände auf der x-, y- und z-Achse jeweils:

$$x = \frac{\tan \alpha_T}{\sqrt{\tan \alpha_R^2 + \tan \alpha_T^2 + 1}} d_o \quad (3.6)$$

$$y = \frac{1}{\sqrt{\tan \alpha_R^2 + \tan \alpha_T^2 + 1}} d_o \quad (3.7)$$

$$z = \frac{\tan \alpha_R}{\sqrt{\tan \alpha_R^2 + \tan \alpha_T^2 + 1}} d_o \quad (3.8)$$

Auf diese Weise wird die Position des Hindernisses im 3D-Raum bestimmt.

4. Messprogramme für Hinderniserkennung

In diesem Kapitel wird eine Reihe von Messprogrammen vorgestellt, die Radar zur Hinderniserkennung verwenden, um die Messergebnisse von Objekten mit unterschiedlichen Materialien und Formen an unterschiedlichen räumlichen Positionen zu bewerten.

4.1 360°-Messung in der Radarebene

Wenn das Radar nach oben orientiert ist, ist es unmöglich, Objekte unterhalb der Radarebene zu erkennen, d.h. sein theoretischer Messbereich ist eine Hemisphäre, die nach oben gerichtet ist.

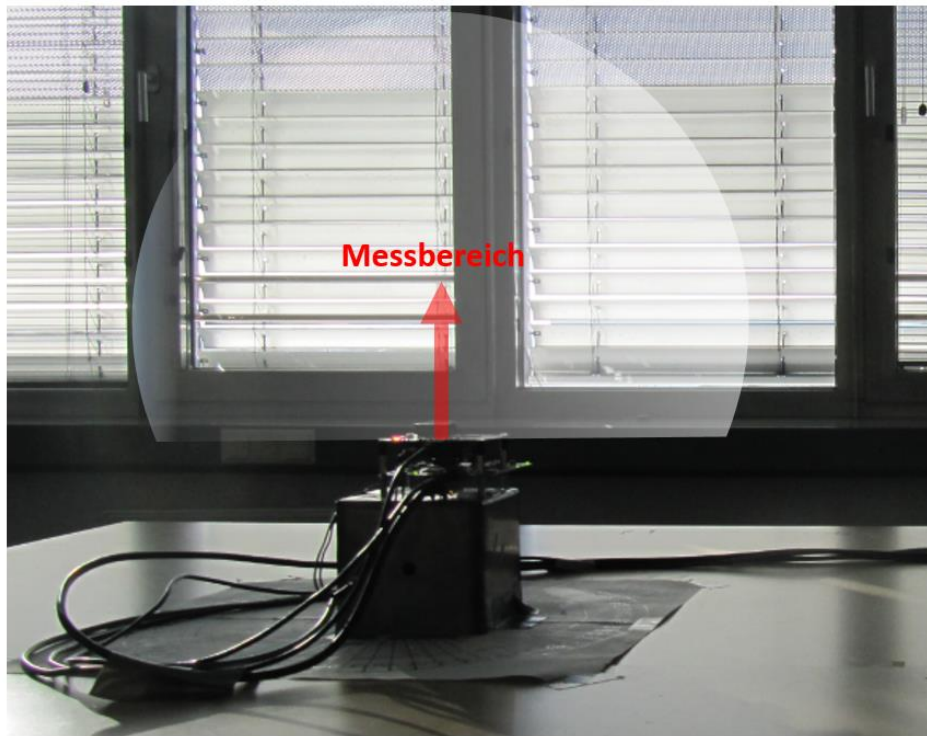


Abb. 4.1: Messbereich des Radars

Daher werden Messungen durchgeführt, um die Messergebnisse im Randbereich des Messbereichs zu bewerten, d.h. die Messergebnisse des Objekts, das sich neben dem Radar und etwas höher als die Radarebene befindet.

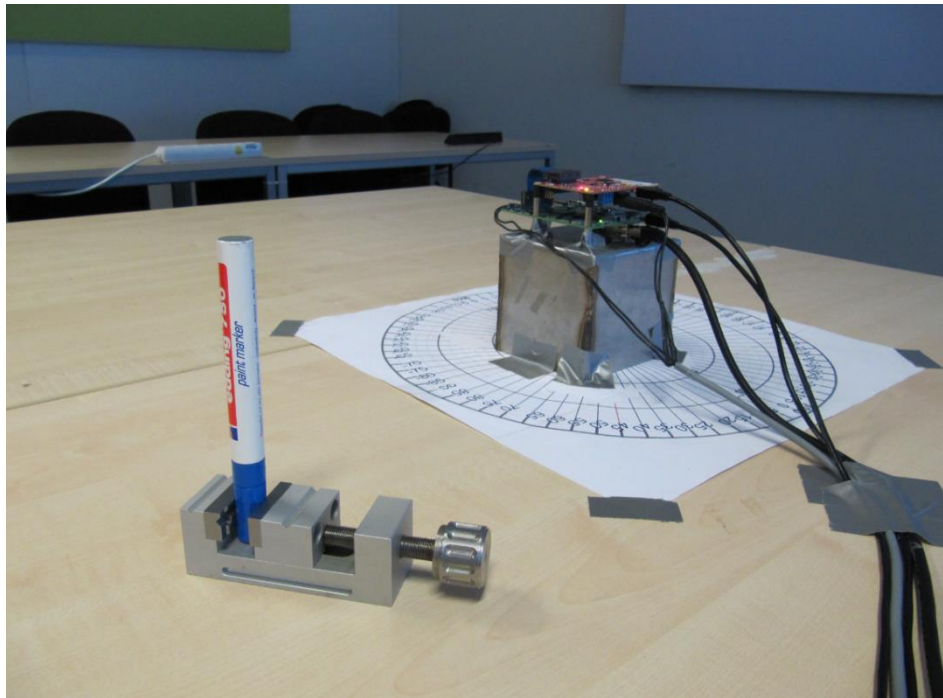


Abb. 4.2: Aufbau der Radarmessung

Die Abbildung oben (Abb. 4.2) zeigt den Aufbau des Messexperiments. Das Radar wird auf einer Metallplattform nach oben platziert, mit Kabeln zur Stromversorgung und Datenübertragung unterhalb der Radarebene. Neben dem Radar befindet sich ein Aluminiumstift, der etwas höher ist als die Radarebene, so dass er korrekt gemessen werden kann. Abhängig vom Winkeldiagramm unterhalb des Radars wird der Metallstift im Bereich von 360° in der Nähe des Radars platziert und gemessen.

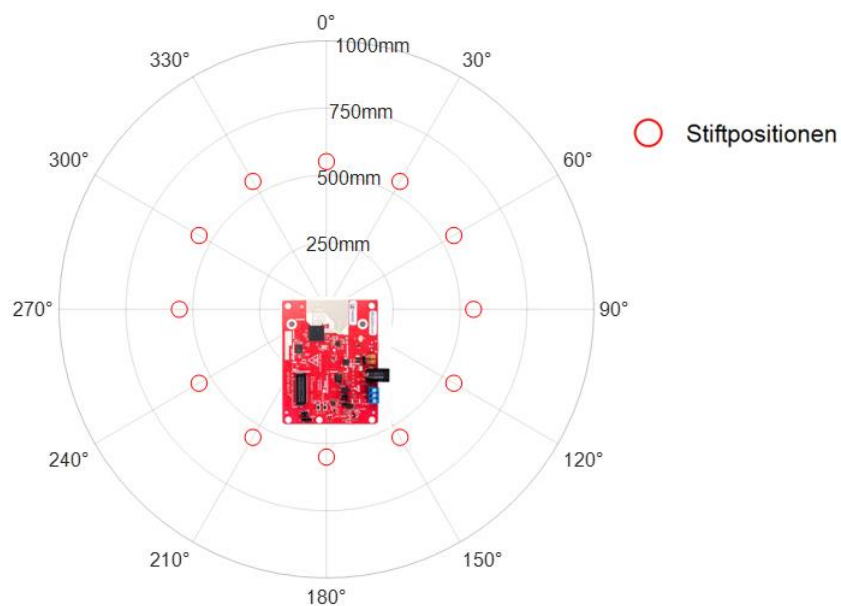


Abb. 4.3: Positionen des Stifts in der Messung

Nach dem Einstellen der Messparameter kann man mit dem Radar messen. Wie bereits beschrieben, überträgt das Radar mit dem DCA1000 die Messdaten über ein Netzkabel an den Host und generiert eine Binärdatei (adc_data.bin), die Rohdaten aus Radarmessungen enthält. Die Algorithmusentwicklung baut auf dieser Binärdatei auf.

Bei Anwendung des im Rahmen dieser Arbeit entwickelten Matlab-Programms zum Lesen und Verarbeiten der Daten in der Binärdatei erhält man das folgende Spektrum:

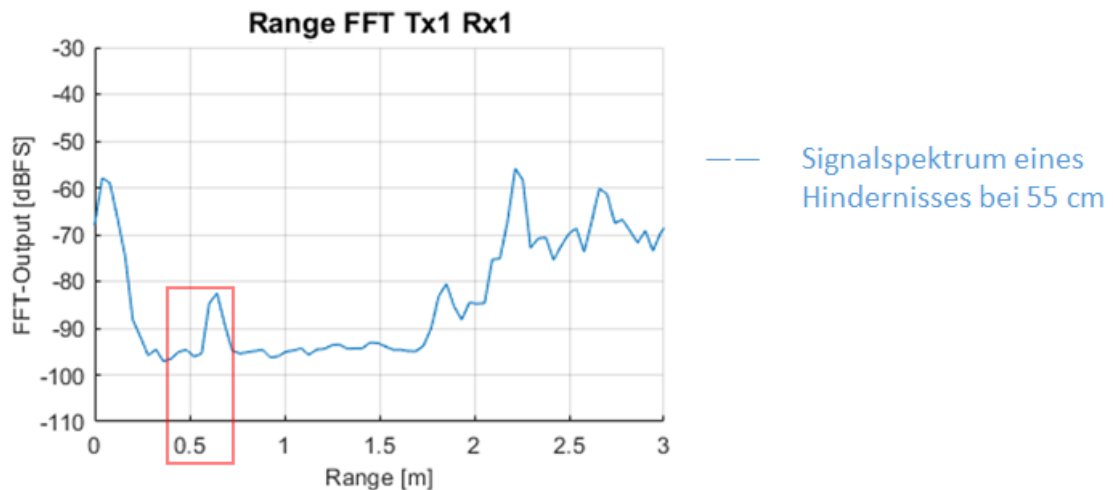


Abb. 4.4: Ein verarbeitetes Signalspektrum

Bei dieser Messung wird das Hindernis 55 cm entfernt platziert, so dass ein Peak des Spektrums in dessen Nähe zu sehen ist. Aufgrund der Leckage des Lokaloszillators an TX-Antennen gibt es auch ein Peak bei 0 m des Spektrums. Dies ist jedoch nicht ein Peak des Hindernisses. Um die Genauigkeit der Messergebnisse zu gewährleisten, befinden sich im Messbereich von 1,5 m keine anderen Objekte wie Stühle, Lampen usw. In größerer Entfernung erscheinen weitere Objekte im Spektrum, wie z. B. die Decke bei etwa 2,2 m. Das folgende Diagramm zeigt den Unterschied zwischen dem Spektrum mit und ohne Hindernis:

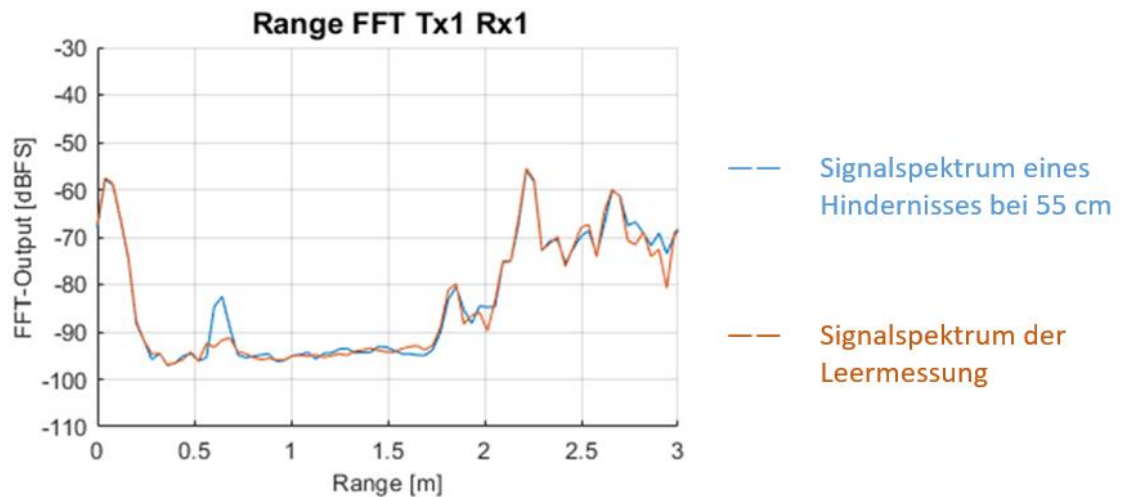


Abb. 4.5: Vergleich zwischen Spektren mit und ohne Hindernisse

Nicht unter jedem Winkel lässt sich das Objekt in einer Entfernung von 55 cm auch erkennen, es gibt blinde Zonen. In der Abbildung unten (Abb. 4.6) sind sowohl eine Position, in der das Hindernis gut erkannt wird, als auch eine Position, in der es nicht erkannt wird, aufgezeigt.

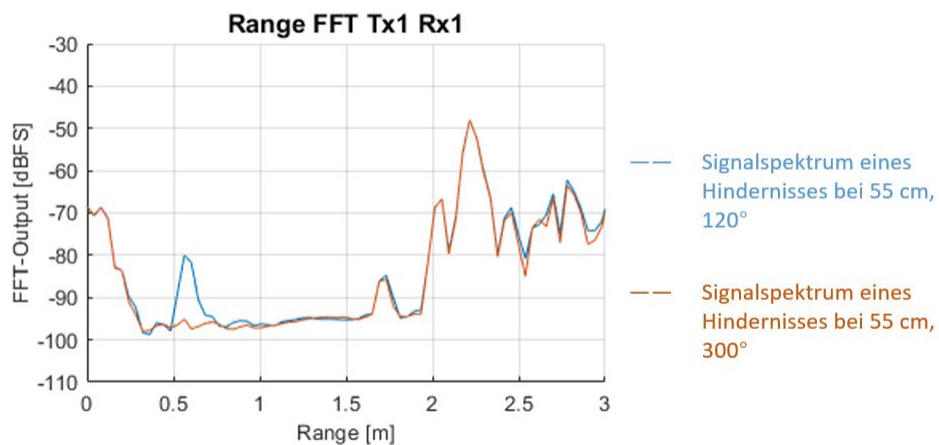


Abb. 4.6: Vergleich zwischen Hindernisspektren bei unterschiedlichen Winkeln

Durch die Analyse des Spektrums kann festgestellt werden, ob das Hindernis durch Radar in der aktuellen Position erkannt werden kann. Aus einer Reihe von Messungen und Vergleichen wird das folgende Bild abgeleitet, das zeigt, wie gut und schlecht der Messbereich ist, wenn das Radar ein Objekt in derselben Ebene erkennen soll:

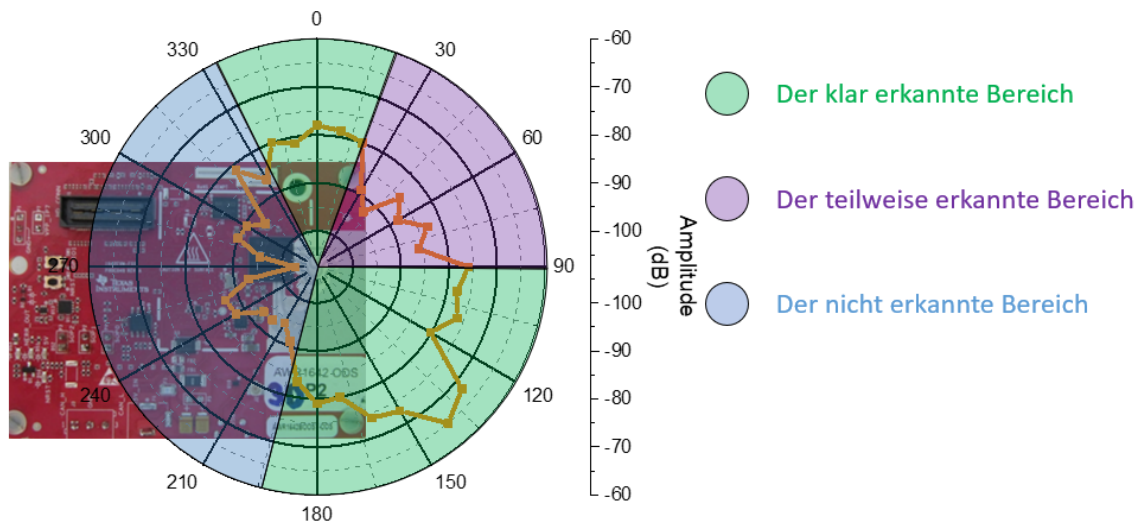


Abb. 4.7: Messergebnisse des Radars in verschiedenen Bereichen in 360°-Messung

Die obige Abbildung (Abb. 4.7) zeigt, dass das Radar im Bereich von 0 bis 180 Grad einen guten Messeffekt auf Hindernisse in derselben Ebene hat, der Messeffekt im Bereich von 180 bis 360 Grad jedoch nicht gut ist. Ein möglicher Grund ist, dass sich die Sende- und Empfangsantennen des Radars in derselben Ebene wie die anderen Komponenten auf der Radarkarte befinden und sich gegenseitig stören können. Dies bedeutet, dass man gewisse Kompensationen in den Algorithmus vornehmen muss, wenn man das Radar in echter Applikation zum Messen von Hindernissen verwendet. Gleichzeitig sind auch einige Hardware-Anpassungen erforderlich, um zu verhindern, dass der Pfad zum Senden und Empfangen von Signalen durch andere Schaltungskomponenten oder andere Antennen blockiert wird.

4.2 „Field of View“-Messungen mit Standard-Hindernissen

In einer Halle werden Radarmessungen mit Normhindernissen unter statischen Bedingungen durch das Einrichten einer geeigneten, störungsarmen Messumgebung durchgeführt:

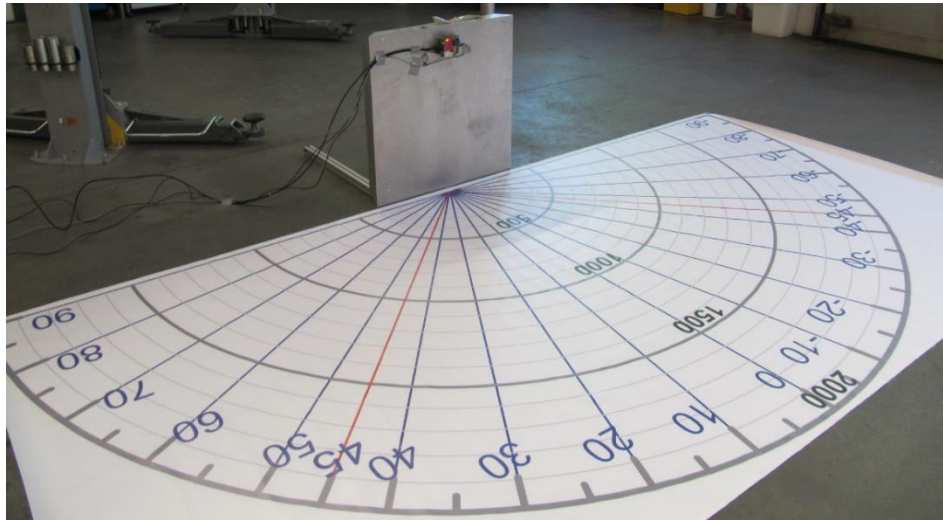


Abb. 4.8: Messaufbau in der Halle

Das Radar wird auf einer Metallplatte platziert, die eine Autotür simuliert. Es ist auf einer bestimmten Höhe vom Boden entfernt. Vor der Metallplatte befindet sich eine Messmatte mit einem Radius von zwei Metern. Auf der Messmatte werden als Normhindernisse Pfosten verschiedener Materialien und Durchmesser gestellt und Radarmessungen durchgeführt. Alle Kabel und andere Gegenstände sind hinter der Radarebene befestigt, um sicherzustellen, dass es keine weiteren Hindernisse im Radardetektionsbereich gibt.

Die bei der Messung verwendeten Hindernisse sind nachstehend aufgeführt. Es gibt Pfosten aus drei Materialien, Aluminium, Holz und Kunststoff. Pfosten aus dem gleichen Material haben unterschiedliche Durchmesser, z. B. 70 mm, 30 mm und 10 mm. Pfosten mit unterschiedlichen Materialien und unterschiedlichen Durchmessern haben unterschiedliche Reflexionsfähigkeiten für Radarsignale:



Abb. 4.9: Standardhindernisse

Der Aufbau einer Messung mit Aluminiumpfosten wird in der folgenden Abbildung (Abb. 4.10) dargestellt. Bei den Messungen beträgt die Höhe der Radarantenne 685 mm, so dass ein Peak an einer Position von ca. 0,7 m im Spektrum zu erwarten ist.

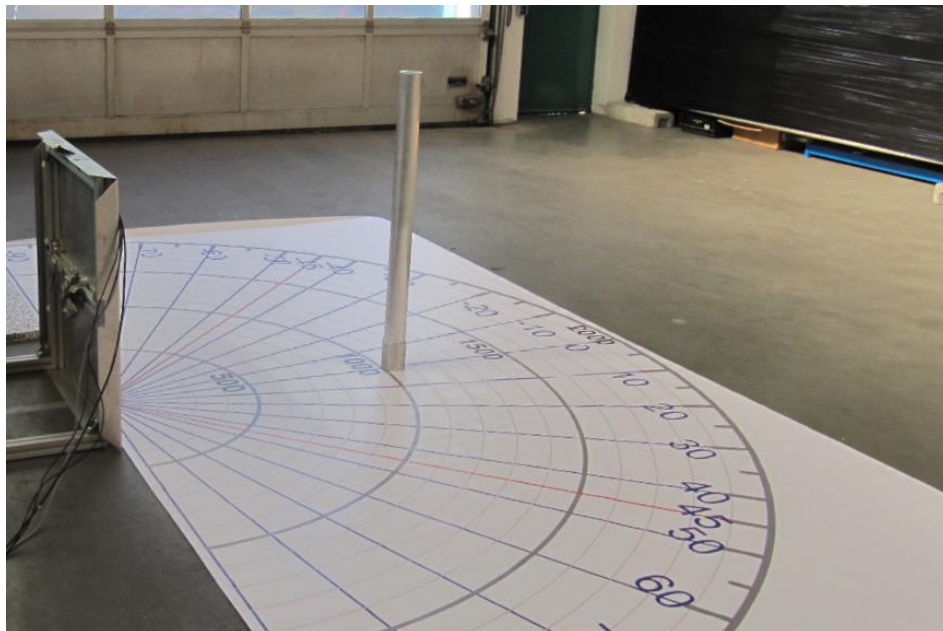


Abb. 4.10: Messaufbau mit Standard-Hindernissen

Bei der Abstandsmessung sind die Einstellung der Parameter und der Prozess der Datenverarbeitung ähnlich den zuvor beschriebenen. Die untere Grafik (Abb. 4.11) zeigt das Messergebnis des Radars, wenn der Aluminiumpfosten mit seiner dem Radar nächsten Seite auf 1 m, 0° der Messmatte gestellt wird:

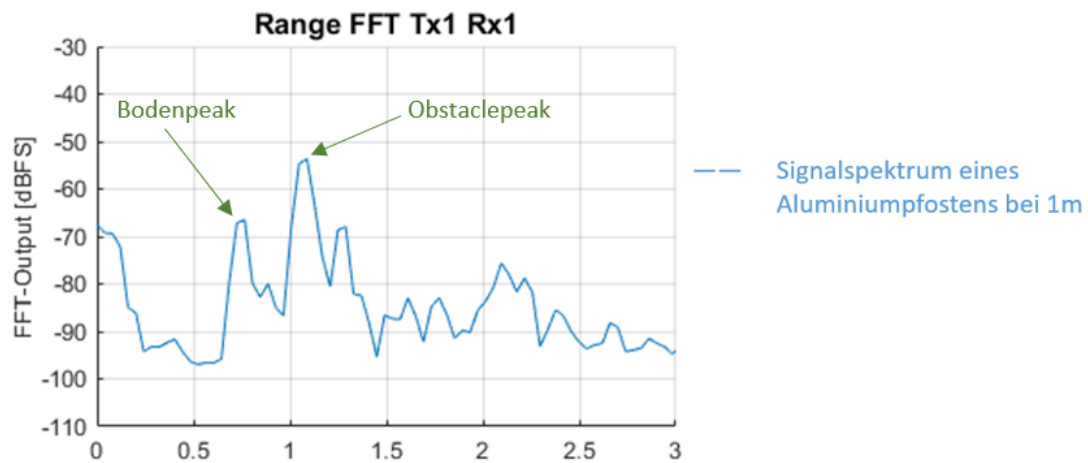


Abb. 4.11: 1D-FFT-Spektrum einer Messung mit Standardhindernissen

Es gibt eine offensichtliche Spitze bei 1 Meter der Grafik, was bedeutet, dass das Hindernis gut erkannt werden kann. Ein Peak bei 0,7 Metern repräsentiert das vom Boden reflektierte Signal. Wenn diese Grafik mit der Grafik ohne Hindernisse verglichen wird, sind die Ergebnisse wie folgt:

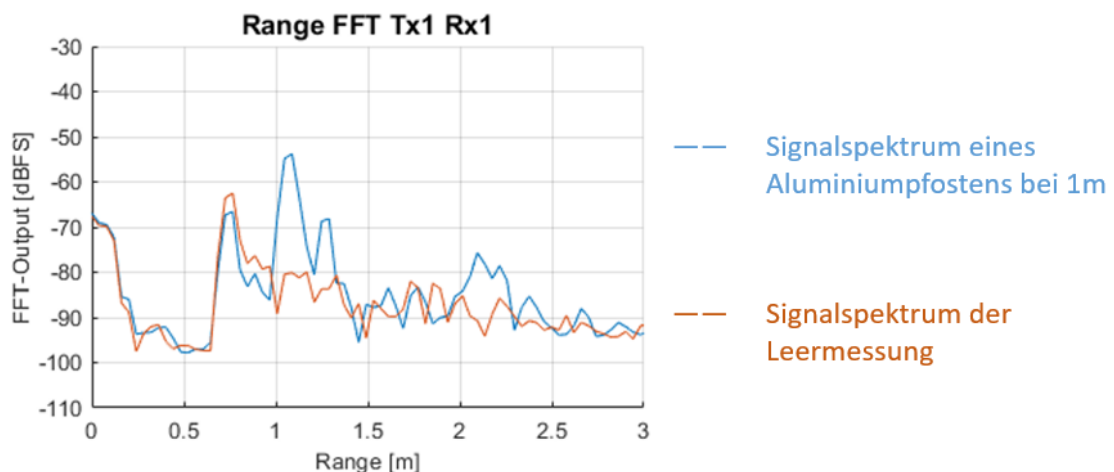


Abb. 4.12: Vergleich zwischen Alupfostenspektrum und Leemessungsspektrum

Man kann erkennen, dass der Spitzenwert bei 1 Meter der Messergebnisse viel höher als der ohne Hindernisse ist. Ein weiterer deutlich sichtbarer Bereich ist das vom Pfostenfuß reflektierte Signal bei 1,25 Metern und eine vermutliche Mehrfachreflexion zwischen Pfosten und Antenne bei 2 Metern.

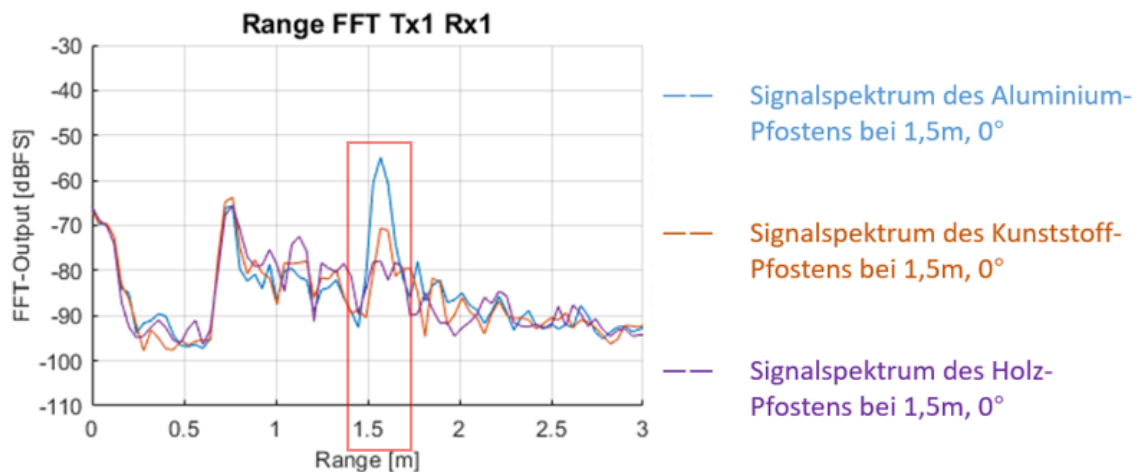


Abb. 4.13: Vergleich zwischen Spektren der unterschiedlichen Pfosten

Verglichen mit der Erkennung von Aluminiumpfosten ist ein Pfosten aus Kunststoff oder Holz jedoch relativ schwer zu erkennen. Wie in der obigen Abbildung (Abb. 4.13) gezeigt, ist der Spitzenwert des Kunststoffpfostens bei 1,5 m und 0 ° niedriger als der Spitzenwert des Aluminiums. Der Spitzenwert des Holzpfeostens ist noch niedriger als der des Kunststoffpfostens und es ist nahezu unmöglich, ihn vom Rauschen zu unterscheiden.

Darüber hinaus ist die Erkennungssicherheit von Hindernissen des Radars stark winkelabhängig. Im Allgemeinen weist das Radar bei 0 ° die beste Signalstärke auf, während das Radar bei 90 ° die schwächste Signalstärke aufweist.

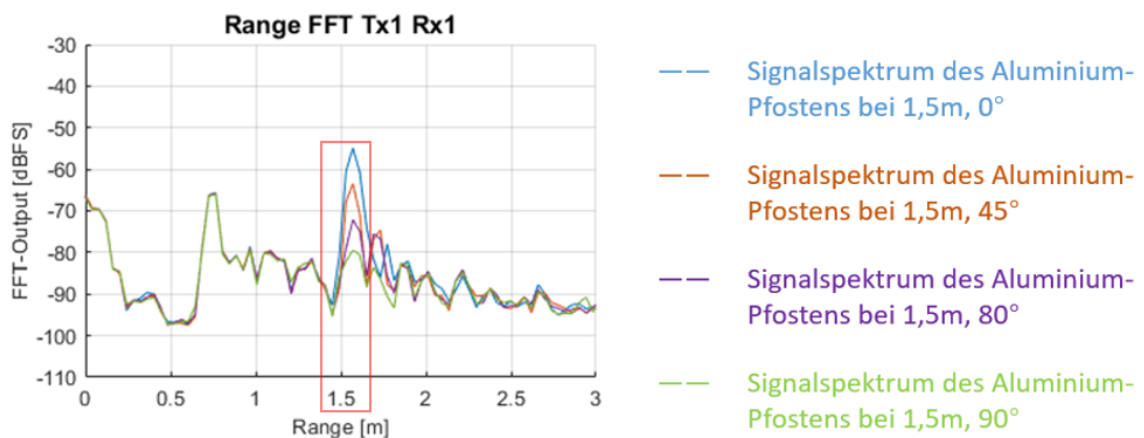


Abb. 4.14: Vergleich zwischen Alupfostenspektren bei unterschiedlichen Winkeln

Aus der obigen Abbildung (Abb. 4.14) erkennt man, dass das Radar den höchsten Peak im Spektrum aufweist, wenn der Aluminiumpfosten bei 0 ° gemessen wird, und den niedrigsten bei 90 °. Dies bedeutet, dass für Hindernisse in 2D-Bereich nur eine 1D-Messung nicht genügt.

Es sollten auch entsprechende Winkelinformationen vorhanden sein, um die Ergebnisse der Messungen korrekt interpretieren zu können.

Daher empfiehlt es sich, eine 2D-FFT zu verwenden, um die Messergebnisse anzuzeigen. Die Methode zur Analyse von Winkeln und Entfernungen mit 2D-FFT wurde in den vorherigen Kapiteln schon beschrieben. Befindet sich der Aluminiumpfosten bei 1,5 m, 0 °, ist das Bild der 2D-FFT wie folgt.

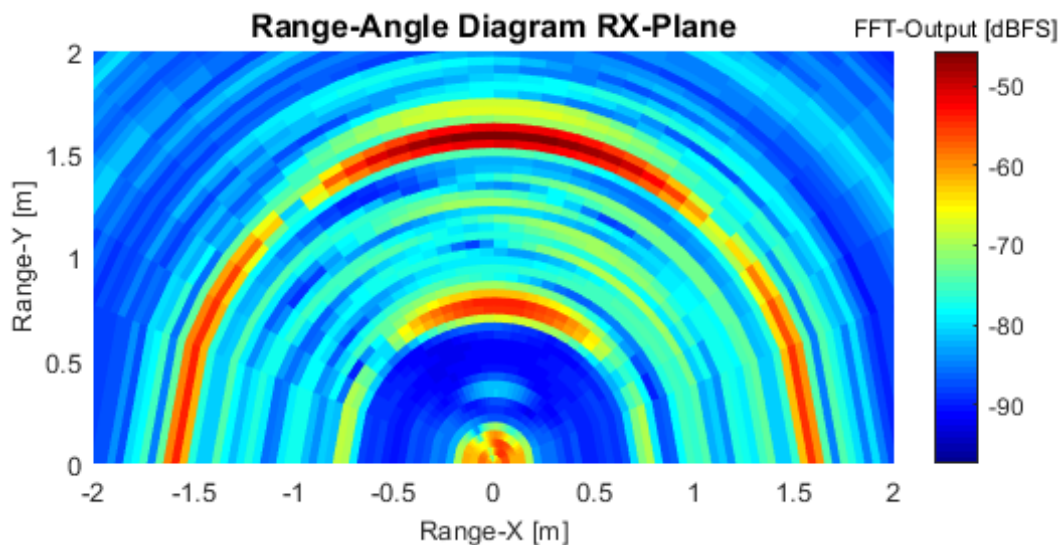


Abb. 4.15: 2D-Range-Angle-Diagramm des Aluminiumpfostens bei 1m, 0°

Die obige Abbildung (Abb. 4.15) zeigt die Positions- und Leistungsinformationen des reflektierten Signals in einer zweidimensionalen Ebene. Man kann sehen, dass das stärkste Signal, das vom Hindernis reflektiert wird, bei etwa 1,5 m, 0 ° liegt. Da Zero-Padding zur Verbesserung der Auflösung des Spektrums angewandt wird, erscheinen zwei falsche Peaks bei größeren Winkeln im etwa gleichen Abstand. Zusätzlich gibt es bei 0,8 m und 0 ° einen starken Peak, der ein vom Boden reflektiertes Signal darstellt.

Wenn sich der Aluminiumpfosten auf 45° befindet, ergeben sich folgende Ergebnisse:

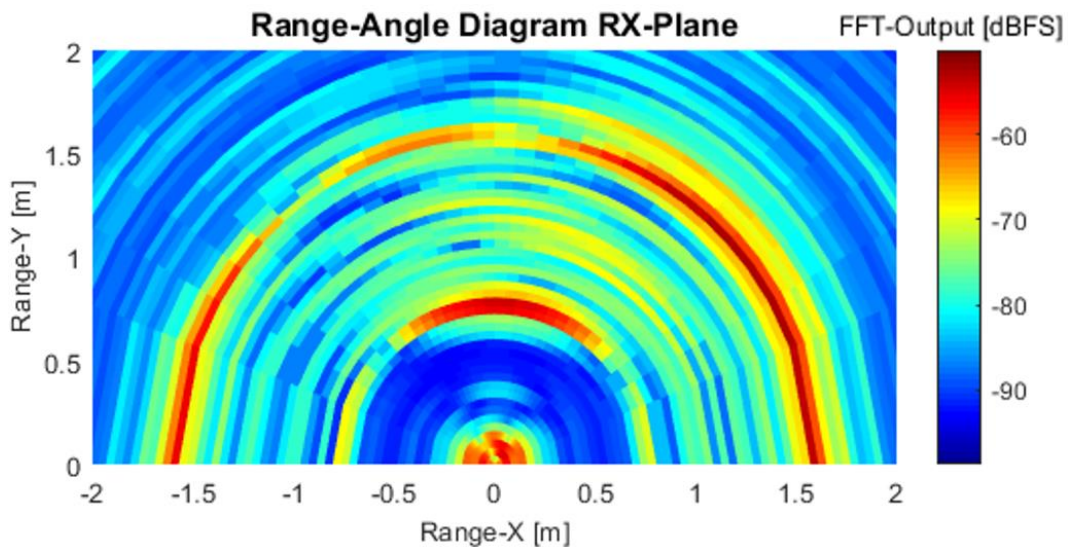


Abb. 4.16: 2D-Range-Angle-Diagramm des Aluminiumpfostens bei 1m, 45°

Es ist zu sehen, dass der höchste Peak nun bei etwa 1,5 m, 45° erscheint. Aus dieser Sicht scheint derselbe Peak auf dem gesamten Bogen von 30 Grad bis 60 Grad in 1,5 Metern Höhe vorhanden zu sein. Tatsächlich kann man jedoch feststellen, dass der höchste Peak bei 45° angezeigt wird, wenn man das Spektrum geringfügig ablenken.

Die Signalreflexionen der Pfosten aus verschiedenen Materialien bei 1,5 m und 45° sind zu vergleichen. Das linke Bild unten zeigt das von der Aluminiumpfosten reflektierte Signal und das rechte Bild zeigt das von einem Holzpfeiler reflektierte Signal. Es ist deutlich zu erkennen, dass das vom Aluminiumpfosten reflektierte Signal viel stärker ist, obwohl in beiden Figuren Peaks bei 1,5 m und 45° vorhanden sind:

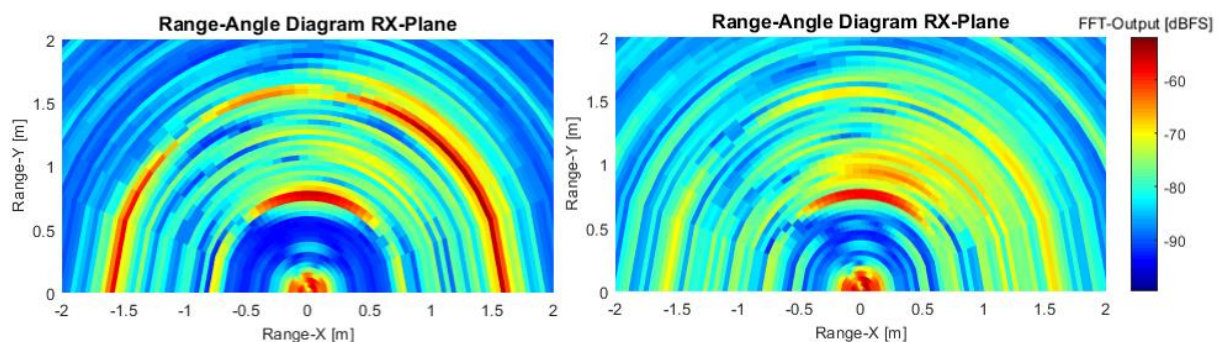


Abb. 4.17: 2D-Range-Angle-Diagramm mit und ohne Aluminiumpfosten

Trotzdem hat die von Holzpfeiler reflektierte Signalleistung noch eine gewisse Intensität. Die folgende Abbildung (Abb. 4.18) gibt einen Vergleich zwischen den Spektren mit und ohne Hindernisse. Obwohl das Signal schwach ist, ist der Unterschied immer noch in den Bildern zu sehen:

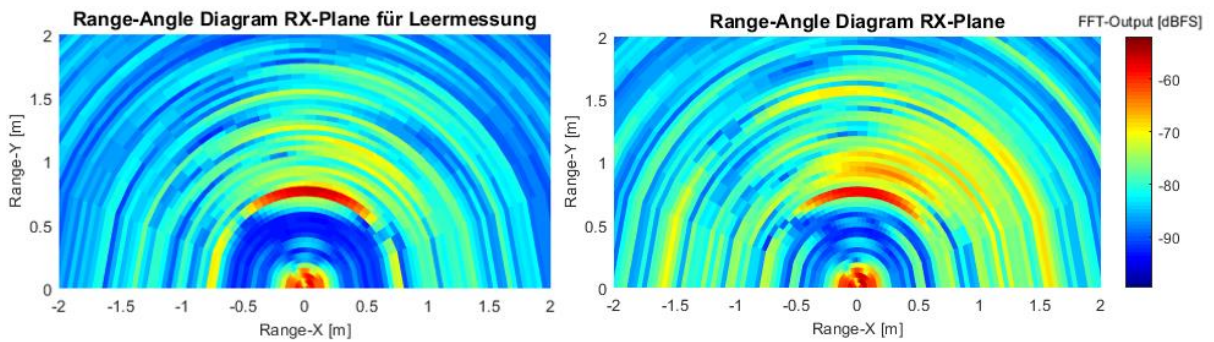


Abb. 4.18: 2D-Range-Angle-Diagramm mit und ohne des Holzpfostens

Für die computergestützte Hinderniserkennung kann man den oben erwähnten CFAR-Algorithmus verwenden, um die Position des effektiven Peaks im Spektrum zu bestimmen. Nach der Verarbeitung der Rohdaten durch 2D-FFT findet man zuerst den Punkt der größten Signalstärke bei jeder Abstandseinheit. Dieser wird dann noch mit einer 1D-Abstands-CFAR abgeglichen. Außerdem wird sichergestellt, dass dieser Punkt den höchsten Peak unter allen Punkten in seiner Nähe aufweist.

Für einen Aluminiumpfosten bei 1,5 m und 0° lautet das Ergebnis der CFAR-Erkennung wie folgt. Die schwarzen Kreise in der Abbildung (Abb. 4.19) zeigen die Positionen der erkannten Hindernisse an:

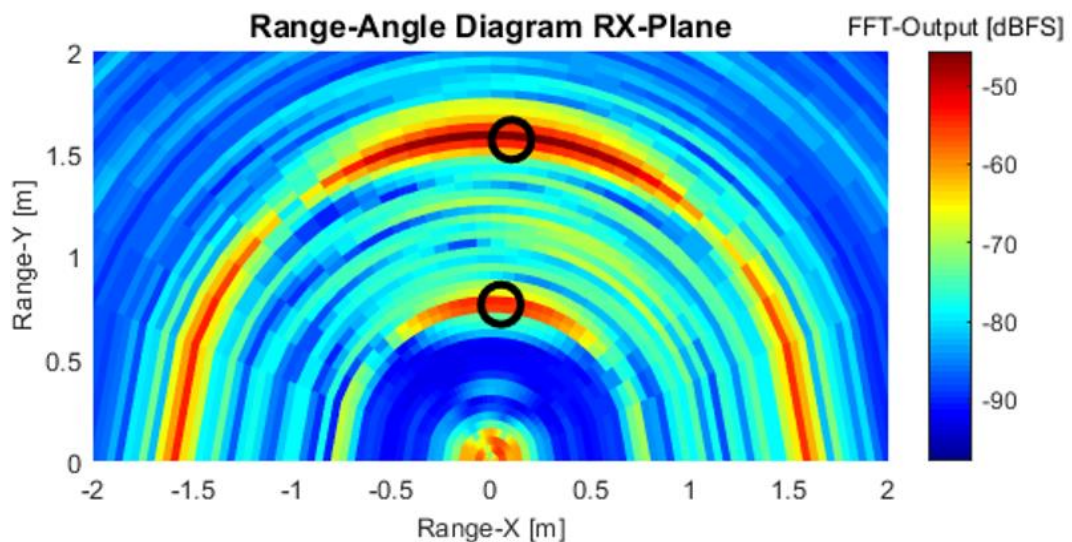


Abb. 4.19: Erkennungsergebnis des Aluminiumpfostens bei 1,5m, 0° durch CFAR

Bei dieser Messung werden sowohl der Aluminiumpfosten als auch der Boden als Hindernisse erkannt. Es ist ein herausforderndes Problem, das vom Boden reflektierte Signal vom Signal eines Hindernisses in ähnlicher Entfernung zu trennen.

Für Holzpfeosten mit schwacher Signalstärke lauten die CFAR-Erkennungsergebnisse, wenn man ihn bei 1,5 m und 45 ° platziert, wie folgt:

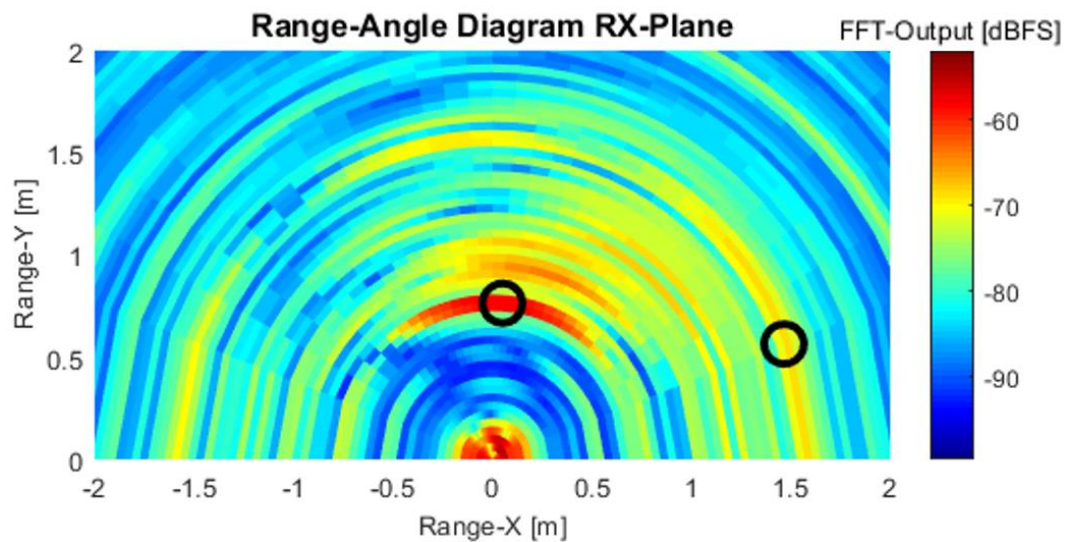


Abb. 4.20: Erkennungsergebnis des Holzpfeostens bei 1,5m, 45° durch CFAR

Abhängig von der Stärke des Signals und der Winkelauflösung ist die Genauigkeit der Winkelerfassung relativ ungenau.

5. Signalverstärkungsgitter im Messbereich

In den vorhergehenden Kapiteln wurde die Theorie der Radarmessung im 3D-Raum vorgestellt. In der Praxis hat die Anwendung theoretischer Messmethoden jedoch gewisse Einschränkungen. In diesem Kapitel wird eine Methode zur Kompensation von Radarsignalen im Messbereich beschrieben, um Radarmessungen besser nutzen zu können.

5.1 Einschränkungen des FMCW Radars in praktischen Anwendungen

Abweichend von perfekten theoretischen Modellen gibt es viele Faktoren, die sich teilweise störend auf Radarmessungen auswirken. Beispielsweise ist, wie oben erwähnt, der Messeffekt des Radars in verschiedenen Messbereichen in seiner Nähe unterschiedlich stark ausgeprägt. Es gibt also gute Messbereiche und schlechtere Messbereiche. Ein weiteres Beispiel: Wenn ein Hindernis aus einem Material besteht, das einen geringen Widerstand gegen elektromagnetische Wellen aufweist, wie z. B. Kunststoff und Holz, ist der Peak im Spektrum im Verhältnis zu einem von Metall reflektierten Signal unbedeutend.

Das erste Beispiel wird in diesem Kapitel unter Verwendung der Verstärkungsgittermethode zur Verbesserung der Messfähigkeit in der Totzone der Radarmessung erörtert.

5.2 Aufbau eines Verstärkungsgitters

Angenommen, der Radarmessbereich ist ein Halbkreis, z. B. eine Fläche mit einem Radius von zwei Metern vor dem Radar. Nach der Vermessung des Hindernisses erhält man ein 2D-Spektrum mit unterschiedlicher Signalverteilung. Idealerweise sollte für dasselbe Hindernis die Signalstärke, die von verschiedenen Positionen zum Radar reflektiert wird, gleich sein. Bei der Radarsignalübertragung treten jedoch Verluste auf, und es gibt eine Winkelabhängigkeit bei der übertragenen und empfangenen Signalstärke.

Wenn man nun Messungen desselben Hindernisses in verschiedenen Winkeln und Entfernungen hat, kann man diese Daten verwenden, um ein Verstärkungsgitter zu erzeugen, um den Radarsignalverlust und die Auswirkungen des Radarmesseffektes zu kompensieren.

Die folgende Abbildung (Abb. 5.1) zeigt den Aufbau eines Verstärkungsgitters:

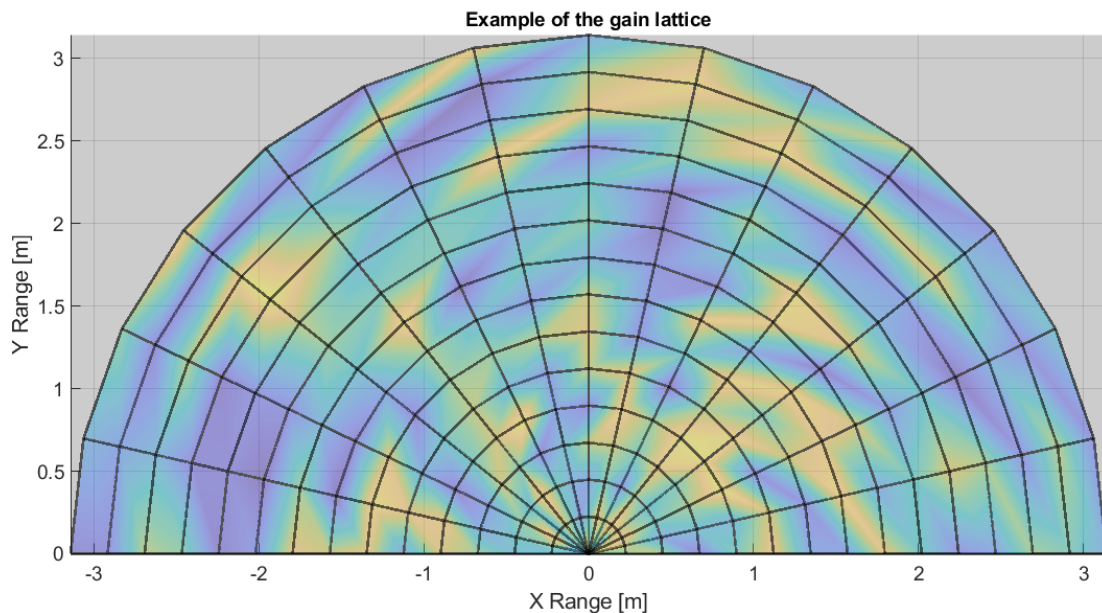
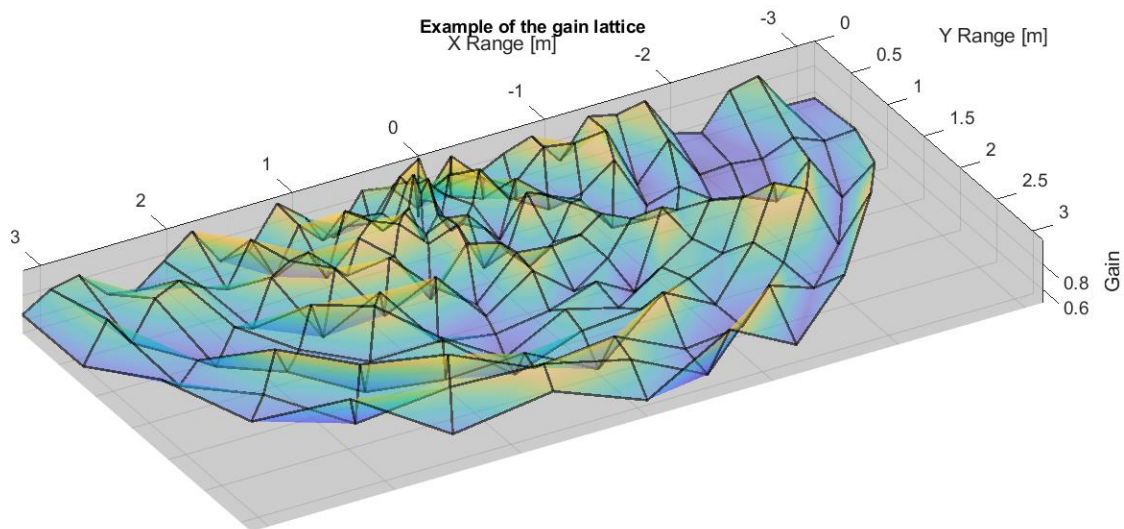


Abb. 5.1: Beispiel eines 15×15 -Verstärkungsgitters in Halbkreisform

Das Signalspektrum nach der 2D-FFT ist eine zweidimensionale Matrix, beispielsweise 1024 Zeilen in Abstandsrichtung und 31 Spalten in Winkelrichtung. Die Abbildung zeigt beispielsweise eine 15×15 -Matrix, die einer Koordinatentransformation unterzogen wird. Jeder Punkt in dieser Matrix hat in x- und y-Richtung einen unterschiedlichen Abstand, und in z-Richtung eine unterschiedliche Signalstärke. Man wählt geeignete Verstärkungsfaktoren und multipliziert den Wert jedes Punktes im Signalspektrum mit einem Verstärkungsfaktor, um ein neues Spektrum zu erzeugen. Das heißt, der neue Wert jedes Punktes im Spektrum ist gegeben durch:

$$Z_{ijnew} = Z_{ij} \times V_{ij} \quad (5.1)$$

Dabei sind (i, j) die Koordinaten eines Punktes im Spektrum, Z_{ij} der ursprüngliche Wert des Punktes, V_{ij} der Verstärkungsfaktor an den Koordinaten dieses Punktes, Z_{ijnew} der neue Wert des Punktes im Spektrum.

Auf diese Weise kann der Verlust des Radarsignals während der Übertragung und der Einfluss unterschiedlicher Messwirkungen des Radars in unterschiedlichen Messbereichen teilweise kompensiert werden.

Um dieses Ziel zu erreichen, muss man zunächst mehrere Messungen mit einem Objekt durchführen, um das entsprechende Verstärkungsgitter zu erstellen.

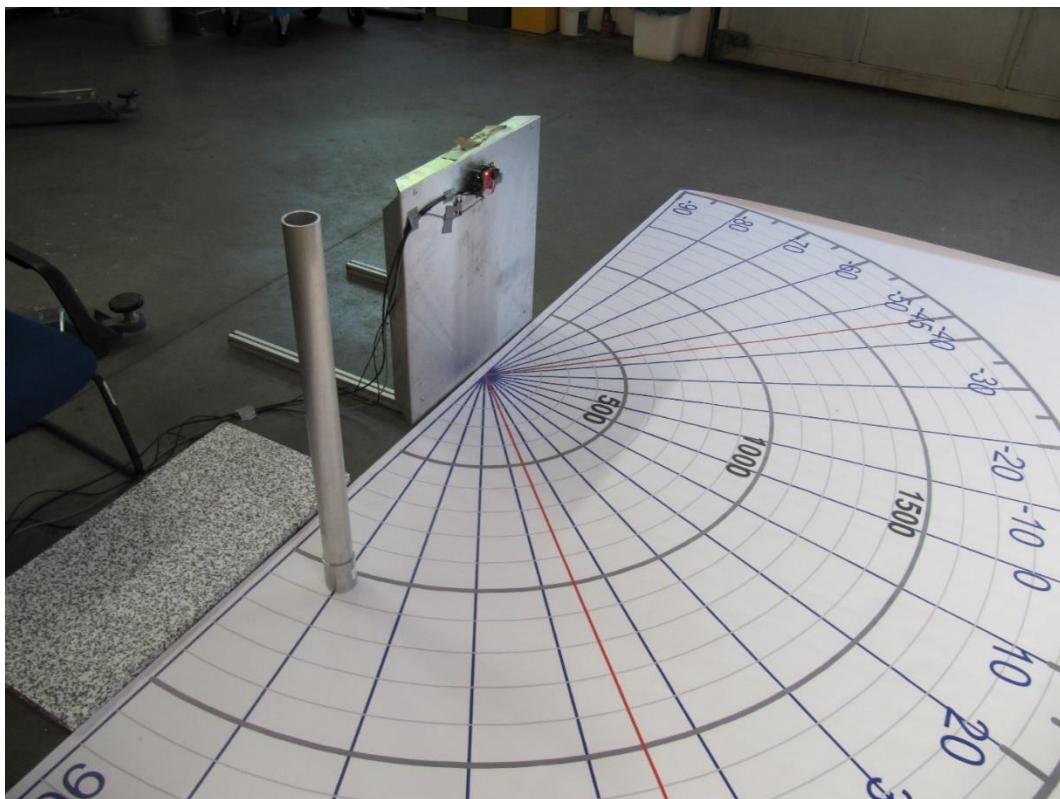


Abb. 5.2: Messungsaufbau für „Field of View“-Messungen

Einige Messungen, hier angefertigt, um das „Field of View“ zu ermitteln („Sichtfeld“-Messungen), werden ausgewählt, um ein Verstärkungsgitter aufzubauen. Zum Beispiel wurde ein Aluminiumpfosten mit einem Durchmesser von 70 mm in einem Messbereich von bis zu 1,5m platziert. Die Positionen dieses Aluminiumpfostens in 39 Messungen (einschließlich einer Messung ohne Hindernisse) sind unten dargestellt:

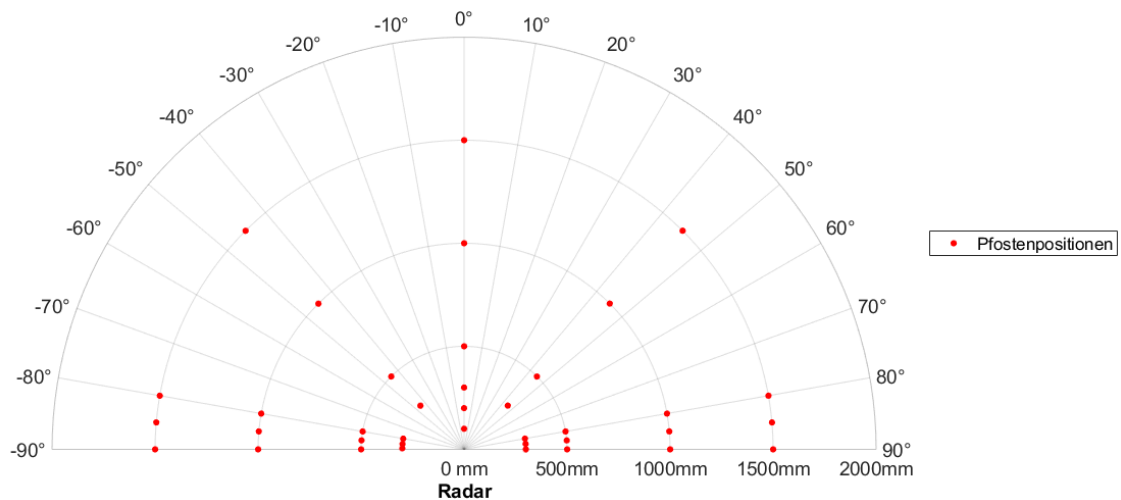


Abb. 5.3: Die Positionen der 70mm-Aluminiumpfosten bei der „Field of View“-Messung

Die ursprüngliche Messergebnismatrix ist eine 31×1024 -Matrix. Die 1024 Abtastpunkte in y-Richtung repräsentieren die unterschiedlichen Entfernungen des Hindernisses vom Radar und die 31 Abtastpunkte in x-Richtung repräsentieren die unterschiedlichen Winkel des Hindernisses zum Radar. Da sich die Berechnung der FFT nur auf den Spitzenwert jedes Abtastpunkts und den relativen Abstand zwischen den beiden Abtastpunkten bezieht und vom absoluten Abstand zwischen den Abtastpunkten auf der Koordinatenachse unabhängig ist, können die physikalischen Bedeutungen der x-Achse und der y-Achse vorübergehend ignoriert werden. Man berücksichtigt nur die relativen Koordinaten der einzelnen Punkte in der 31×1024 -Matrix, z. B. (15,512), (28, 768) usw. Jetzt muss man die Koordinate und Amplitude des höchsten Peaks der 38 Messungen auf der x- Richtung und der y- Richtung, d.h. die Positionen des Hindernisses, aufzeichnen. Hierbei beziehen sich x und y nur auf die Position der Koordinatenpunkte in der Matrix, nicht auf die spezifischen Abstandseinheiten. Obwohl es in Abstandsrichtung 1024 Abtastpunkte gibt, sind nur 50 Abtastpunkte nützlich, da nur Messungen innerhalb von 2 Metern berücksichtigt werden. So nimmt man eine 31×50 -Matrix, die den effektiven Messbereich aller Messungen darstellt. Die folgende Tabelle zeigt die Koordinaten und Amplituden der Spektralspitzen, wenn sich die Hindernisse an verschiedenen Positionen befinden.

Measurement	Distance / mm	Angle / °	x	y	Amplitude
1	100	0	13	5	9.3035e+04
2	200	0	14	8	8.6422e+04
3	300	-90	31	10	9.9332e+03
4	300	-85	31	10	1.1401e+04
5	300	-80	30	10	1.7194e+04
6	300	-45	26	10	6.7034e+04
7	300	0	14	10	9.6179e+04
8	300	45	3	11	4.3328e+04
9	300	80	1	10	2.0532e+04
10	300	85	1	10	1.8171e+04
11	300	90	1	11	1.2669e+04
12	500	-85	30	15	8.2586e+03
13	500	-80	30	15	1.5555e+04
14	500	-45	26	15	6.3275e+04
15	500	0	14	15	9.3196e+04
16	500	45	3	15	4.2874e+04
17	500	80	1	15	2.0861e+04
18	500	85	1	15	9.7980e+03
19	500	90	1	15	1.5675e+04
20	1000	-90	30	27	6.2751e+03
21	1000	-85	30	27	1.3189e+04
22	1000	-80	29	27	1.1518e+04
23	1000	-45	26	27	5.0806e+04
24	1000	0	15	28	6.9770e+04
25	1000	45	3	28	3.1901e+04
26	1000	80	1	27	6.9818e+03
27	1000	85	1	28	1.0995e+04
28	1000	90	2	27	7.2579e+03
29	1500	-90	31	40	4.2392e+03
30	1500	-85	31	40	7.5392e+03
31	1500	-80	31	40	7.1535e+03
32	1500	-45	26	39	3.7529e+04
33	1500	0	15	40	6.1958e+04
34	1500	45	4	40	2.7458e+04
35	1500	80	1	40	1.9942e+04
36	1500	85	1	40	1.3539e+04
37	1500	90	1	40	7.0285e+03

Tabelle: Die Position und Amplitude des höchsten Punktes jedes 2D-Spektrums in den Messungen

Die Amplitude der Messpunkte in dB ist in der Abbildung (Abb. 5.4) unten dargestellt:

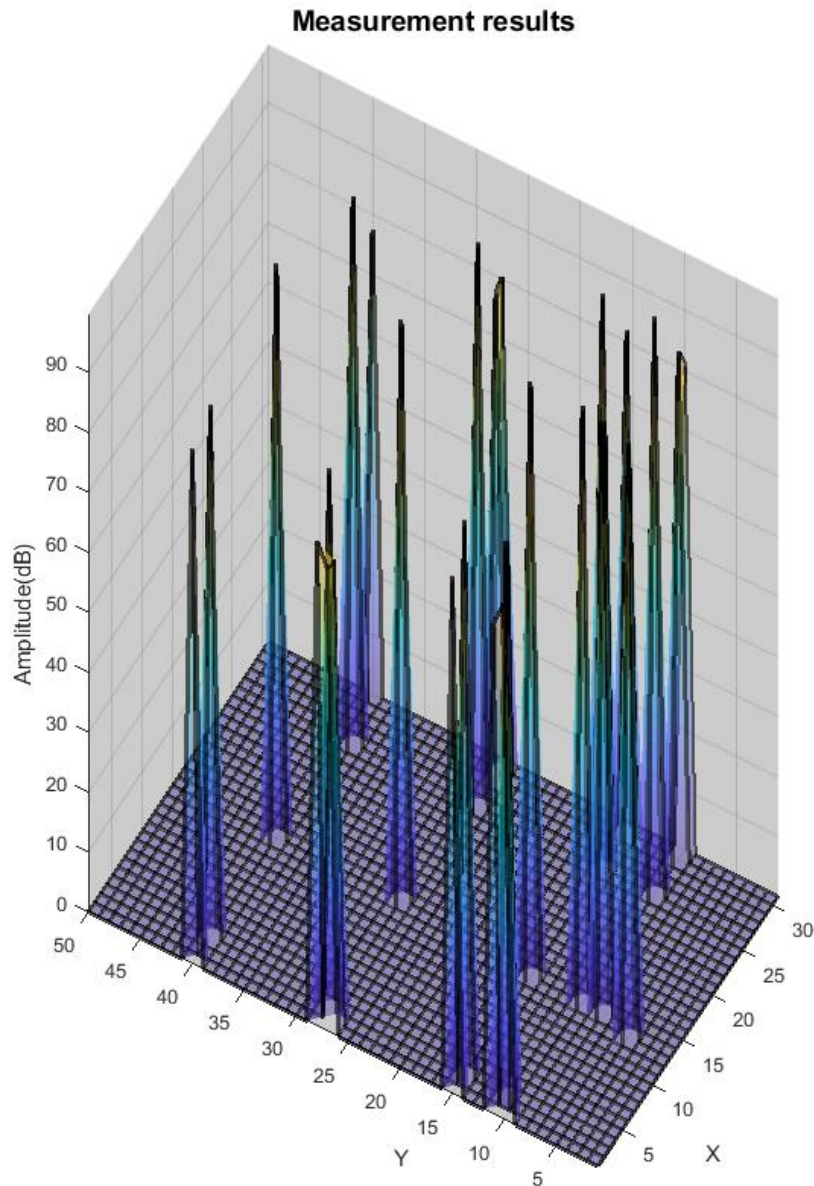


Abb. 5.4: Messergebnisse desselben Hindernisses an verschiedenen Positionen unter denselben Messbedingungen

Um das Verstärkungsgitter zu erhalten, muss man die Werte dieser Punkte auf den gesamten Messbereich ausweiten, um eine Oberfläche zu bilden. Matlab bietet eine Methode, Oberflächen mit mehreren Punkten im Raum zu konstruieren. Aufgrund der unterschiedlichen Parameter und Ordnungen der Polynome weisen die Oberflächenanpassungsergebnisse von Matlab jedoch große Unterschiede auf. Um das Anpassungsergebnis so genau wie möglich zu machen, wird in dieser Arbeit auch eine Methode zum Konstruieren der Oberfläche unter Verwendung mehrerer Punkte im Raum angegeben und mit den Ergebnissen von Matlab verglichen.

Zuerst findet man einen nächstgelegenen Datenpunkt für den Mittelpunkt jeder Kante im Messbereich und den Punkt an jeder Ecke und weist man den Wert dieses Datenpunkts dem Mittelpunkt oder dem Eckpunkt daneben zu, und das Ergebnis in der x-y-Ebene wird unten angezeigt:

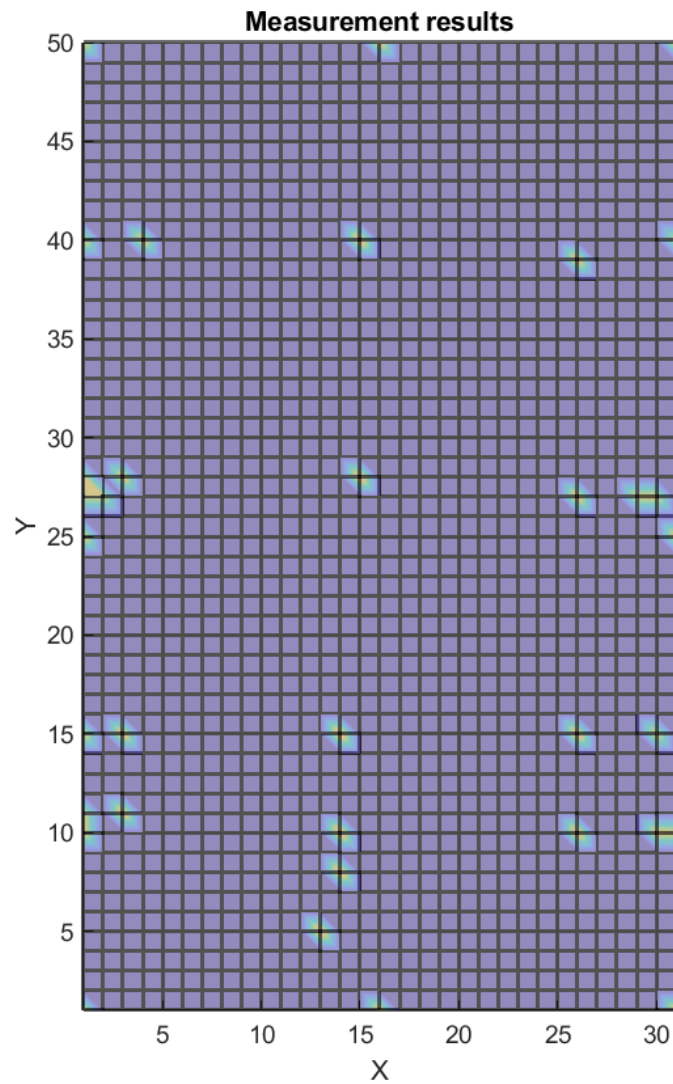


Abb. 5.5: Messergebnisse in der x-y-Ebene, nachdem den vier Ecken des Messbereichs und den Mittelpunkten der vier Seiten Werte zugewiesen werden

Als nächstes nimmt man drei beliebige Datenpunkte im Raum als Beispiel, um den Konstruktionsprozess der Oberfläche zu zeigen. Angenommen, die Koordinaten der drei Punkte lauten $P_1(25,25,1)$, $P_2(15,2,2)$, $P_3(1,28,4)$, und der Durchschnittswert aller Punkte im gesamten Raum beträgt 3. Die folgende Abbildung (Abb. 5.6) zeigt die relative Position und der Wert drei Punkte im Raum:

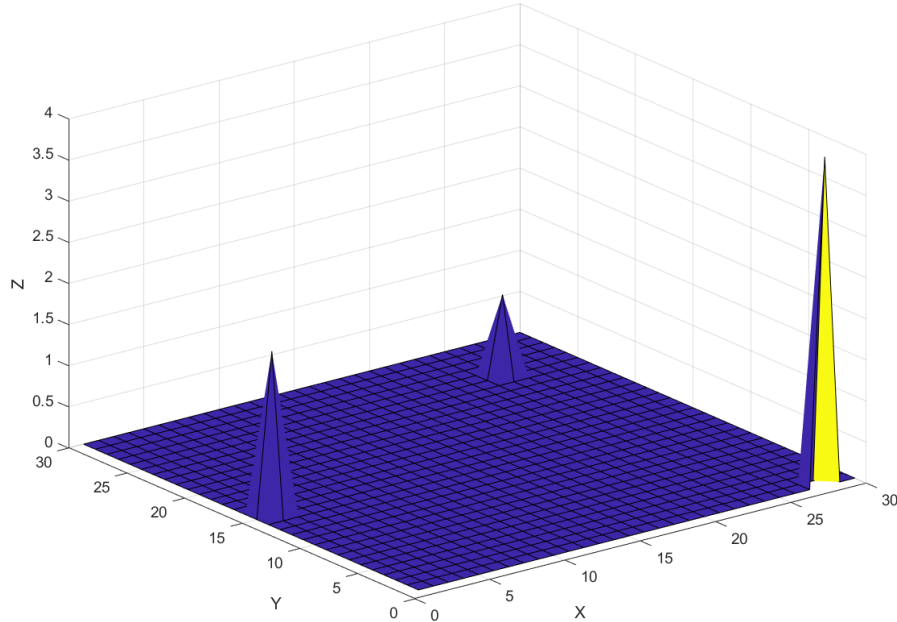


Abb. 5.6: Drei Datenpunkte im Raum ohne Oberflächenanpassung

Diese drei Punkte bilden ein Dreieck in der x-y-Ebene. Der Schwerpunkt des Dreiecks wird zunächst berechnet, nämlich $x_m = \frac{x_1+x_2+x_3}{3}$, $y_m = \frac{y_1+y_2+y_3}{3}$. Der angenommene Durchschnitt aller Punkte wird dann dem Schwerpunkt des Dreiecks zugeordnet, der in diesem Fall 3 ist. Danach werden die Werte der drei Eckpunkte und des Schwerpunkts des Dreiecks verwendet, um anderen Punkten im Dreieck Werte zuzuweisen. Von jedem Eckpunkt bis zum Schwerpunkt des Dreiecks wird mit zunehmendem Abstand jedem Punkt ein Wert zugewiesen, der über die logistische Funktion $\frac{1}{1+e^{-x}}$ in der Sigmoidfunktion festgelegt wird. In diesem Beispiel stellen diese drei Punkte wie die Abtastpunkte in der 2D-FFT den Spitzenwert des Rücklaufsignals dar, wenn der Aluminiumpfosten an einer bestimmten Position im Raum gemessen wird. Die durch dieses Verfahren gebildete Oberfläche ist nachstehend gezeigt:

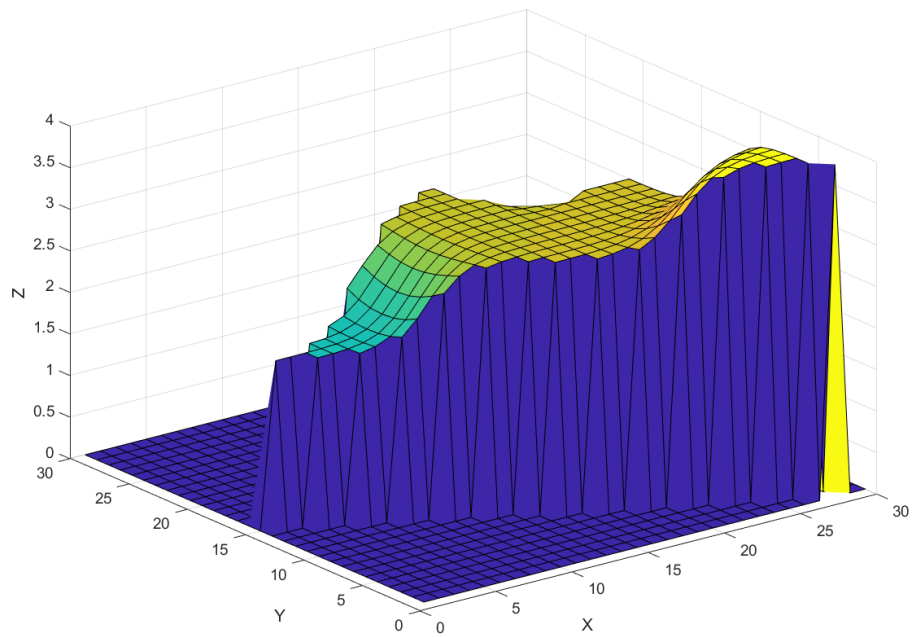


Abb. 5.7: Oberflächenanpassung mit drei Datenpunkten im Raum

In der anderen Richtung sieht die Oberfläche entsprechend so aus:

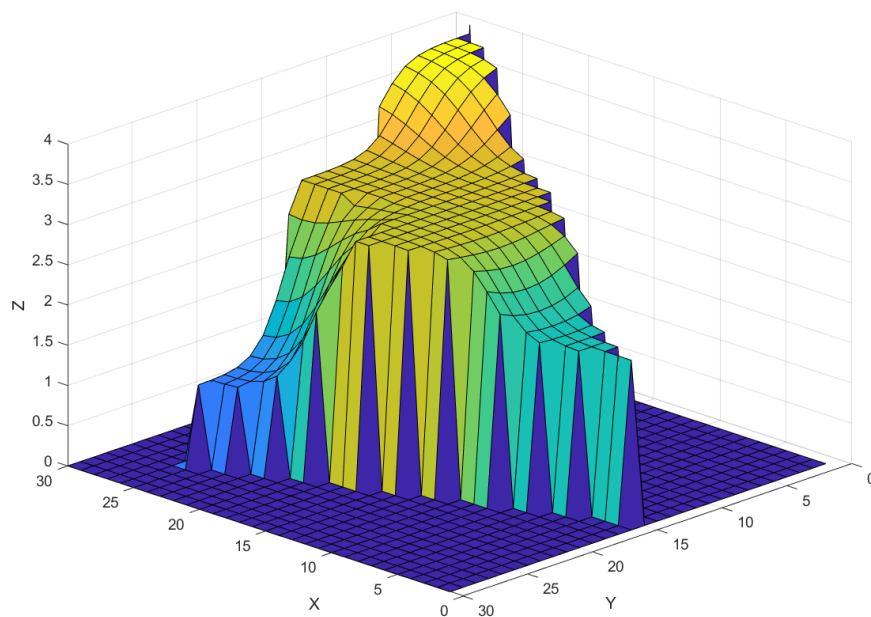


Abb. 5.8: Oberflächenanpassung mit 3 Datenpunkten im Raum

Das obige Beispiel gibt einen Algorithmus zum Konstruieren einer Oberfläche unter Verwendung von drei beliebigen Punkten im Raum an. Als nächstes möchten wir innerhalb des gesamten Messbereichs alle bekannten Punkte verwenden, um mehrere Dreiecke zu bilden, um eine ganze Oberfläche unter Verwendung der obigen Methode zu konstruieren.

Dazu wurde eine Methode entwickelt, mithilfe der man mit vielen Punkten an den Ecken, an den Kanten und im Inneren eines Rechteckes eine Parkettierung mit Dreiecken durchführt. Diese Methode kann als „Webalgorithmus“ bezeichnet werden.

Die Idee dieses Algorithmus ist es, einen kontinuierlich expandierenden Kreis mit einem Eckpunkt des Rechtecks als Mittelpunkt zu erzeugen. Mit zunehmender Ausdehnung des Kreises befinden sich immer mehr bekannte Punkte im Kreis, und diese bekannten Punkte können Dreiecke bilden, die sich nicht überlappen. Wenn ein neuer bekannter Punkt den Rand des Kreises berührt, beginnt die Beurteilung, ob dieser Punkt mit anderen Punkten im Kreis ein neues Dreieck bilden kann, das sich nicht vorhandenen Dreiecken im Kreis überschneidet. Wenn ja, wird dieses Dreieck ein neues Dreieck innerhalb des Kreises. Die spezifische Berechnungsmethode des „Webalgorithmus“ wird anhand des folgenden Beispiels erläutert.

Angenommen, es gibt ein rechteckiges 9×9 -Array. An den Ecken, Seiten und innerhalb dieses Arrays befinden sich die folgenden Datenpunkte:

P1(1,1,2), P2(1,9,4), P3(9,1,2), P4(9,9,1), P5(1,5,3), P6(6,5,5), P7(4,6,2), P8(8,8,7)

In der x-y-Ebene ist die Verteilung dieser Punkte wie folgt:

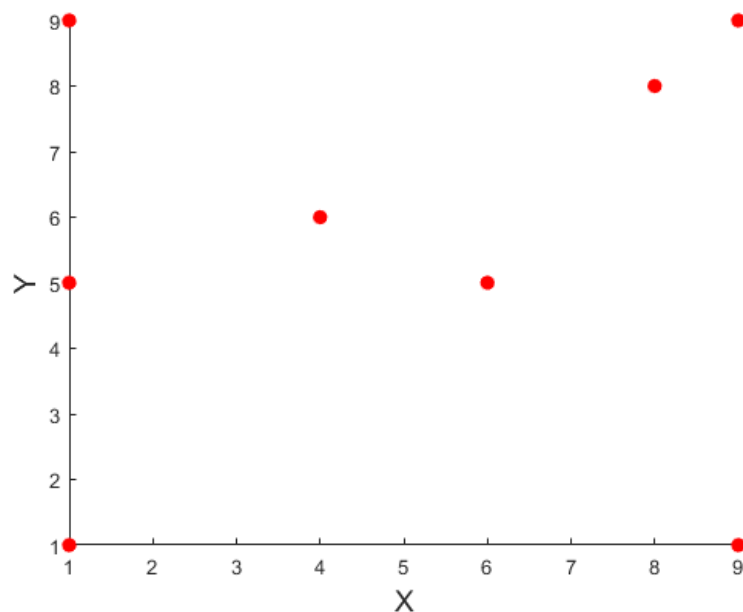


Abb. 5.9: Einige Punkte in der Ebene, die zur Parkettierung der Ebene mit Dreiecken verwendet werden

Unter Verwendung dieser Punkte wird der „Webalgorithmus“ verwendet, um eine Dreiecksegmentierung durchzuführen. Zunächst werden der Punkt bei (1,1) und die zwei ihm am nächsten liegenden Punkte (nämlich (1,5), (4,6)) genutzt, um ein Dreieck zu bilden. Gesucht wird dann der nächste Punkt, der dem (1,1)-Punkt am nächsten liegt (in diesem Fall (6,5)), und

damit und mit allen anderen bekannten Punkten werden Dreiecke gebildet. Es ist jedoch erforderlich, dass sich das gebildete Dreieck nicht mit einem schon vorhandenen Dreieck überlappt und der maximale Winkel des Dreiecks so klein wie möglich ist (um eine gleichmäßigere Form zu gewährleisten). Unter solchen Bedingungen wird ein zweites Dreieck gezeichnet. Danach wird der Vorgang wiederholt, bis die Ebene von Dreiecken bedeckt ist. Die folgende Abbildung (Abb. 5.10) zeigt die ersten drei Schritte des „Webalgorithmus“ und das Endergebnis.

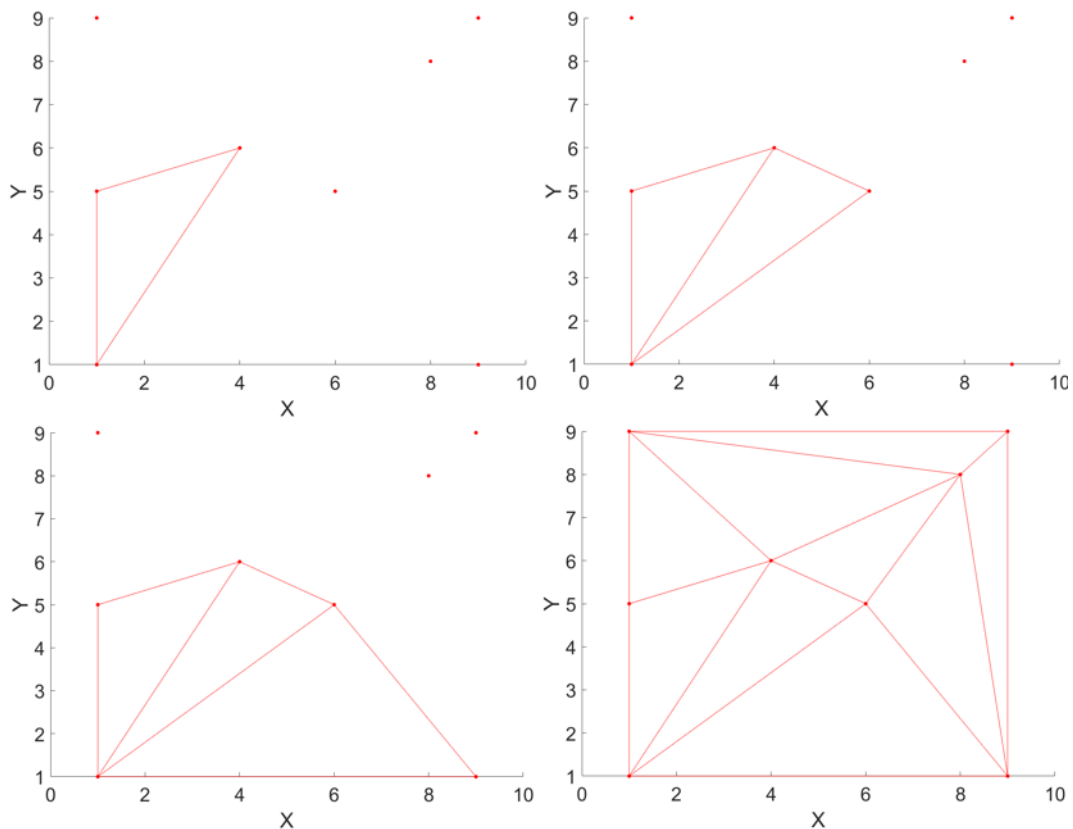


Abb. 5.10: Die ersten drei Schritte und Endergebnisse des „Webalgorithmus“

Jetzt kann man den „Webalgorithmus“ verwenden, um den Messbereich mit Dreiecken zu parkettieren.

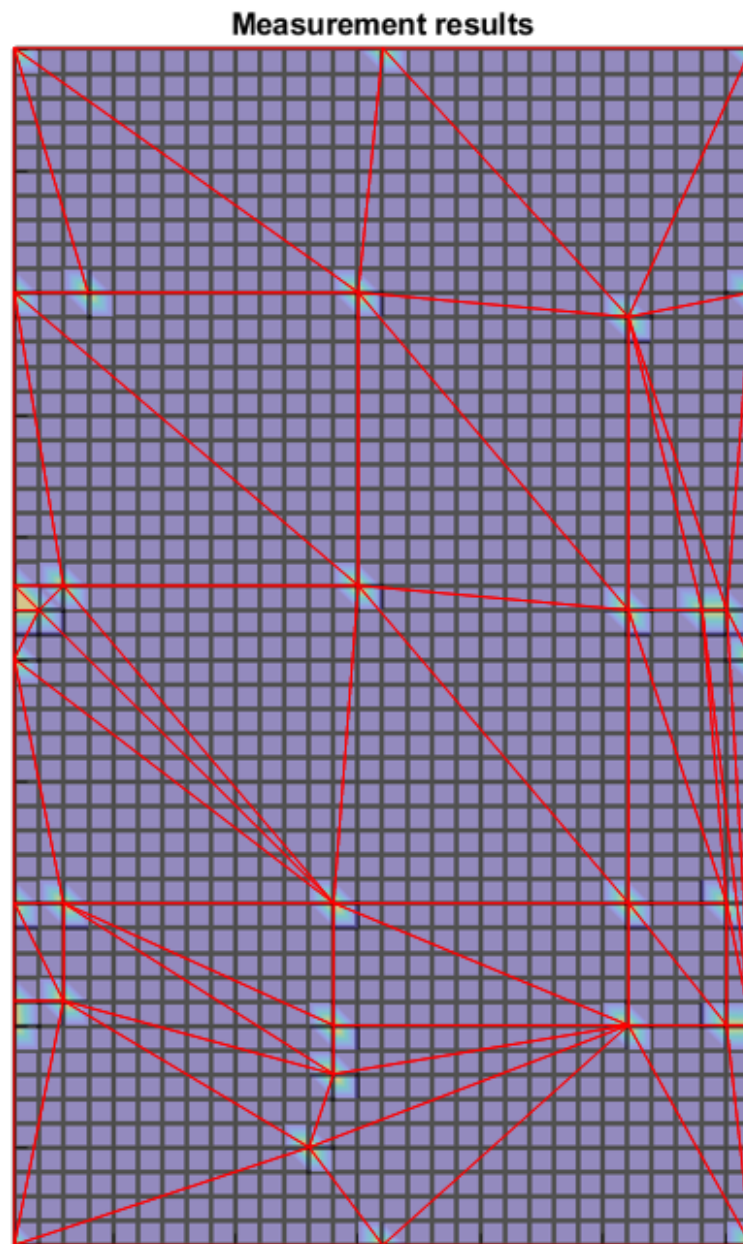


Abb. 5.11: Die x-y-Ebene des Messergebnisses, die durch den „Webalgorithmus“ in mehrere Dreiecke unterteilt wird

Die Oberfläche, die mit den obigen Dreiecken ausgestattet ist, wird unten gezeigt:

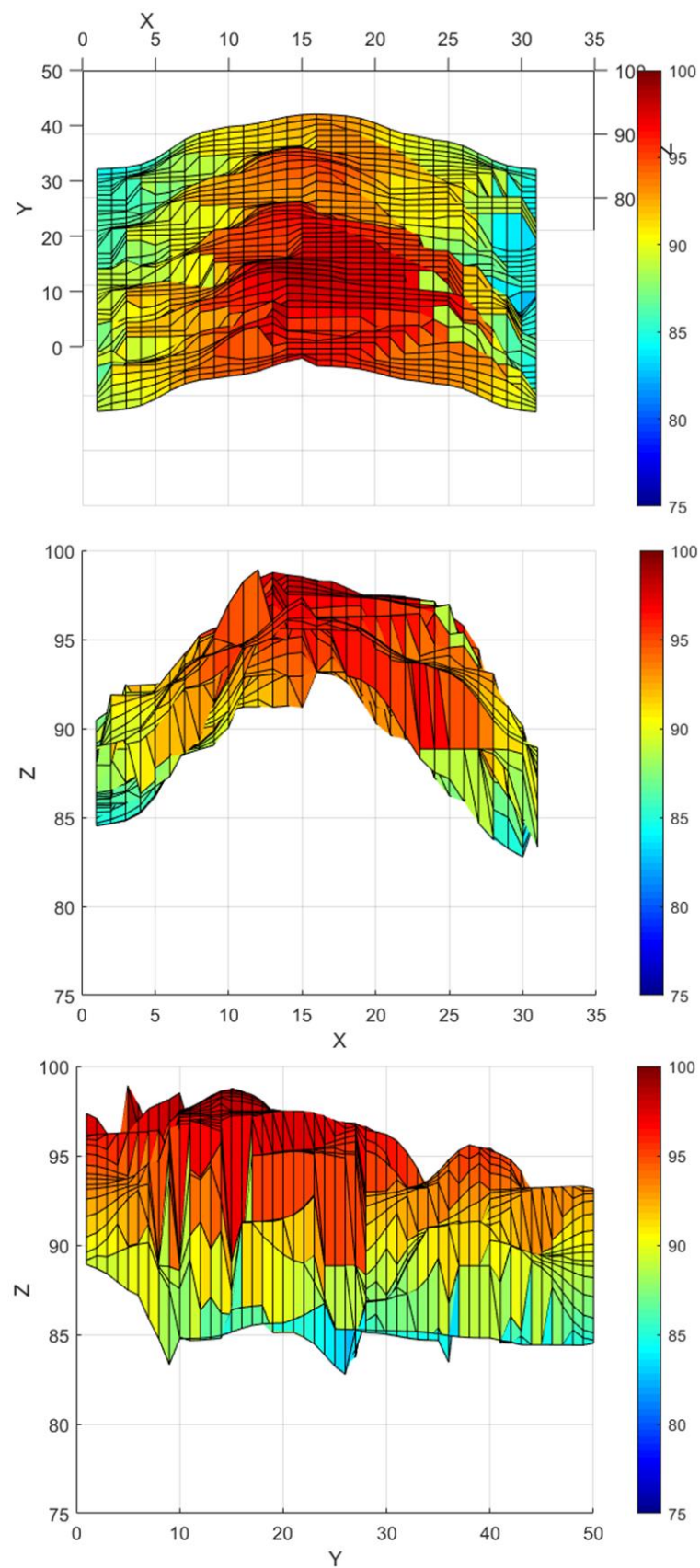


Abb. 5.12: Eine mit dem „Webalgorithmus“ erzeugte Oberfläche, die sich aus verschiedenen Messergebnissen zusammensetzt

Aus der obigen Abbildung (Abb. 5.12) ist ersichtlich, dass mit zunehmender Entfernung oder zunehmendem Winkel die vom Radar bei der Messung desselben Hindernisses zurückgesendete Signalstärke allmählich abnimmt und der Effekt der Winkelerhöhung stärker beeinflusst wird als die Entfernungserhöhung.

Als nächstes wird die polynomielle Anpassung der Oberfläche mit den internen Tools von Matlab durchgeführt. Da man die allgemeine Form der Oberfläche bereits kennt, muss man die Parameter nur so anpassen, dass die Ergebnisse der Matlab-Oberflächenanpassung den obigen Ergebnissen ähneln. Das Ergebnis ist unten dargestellt:

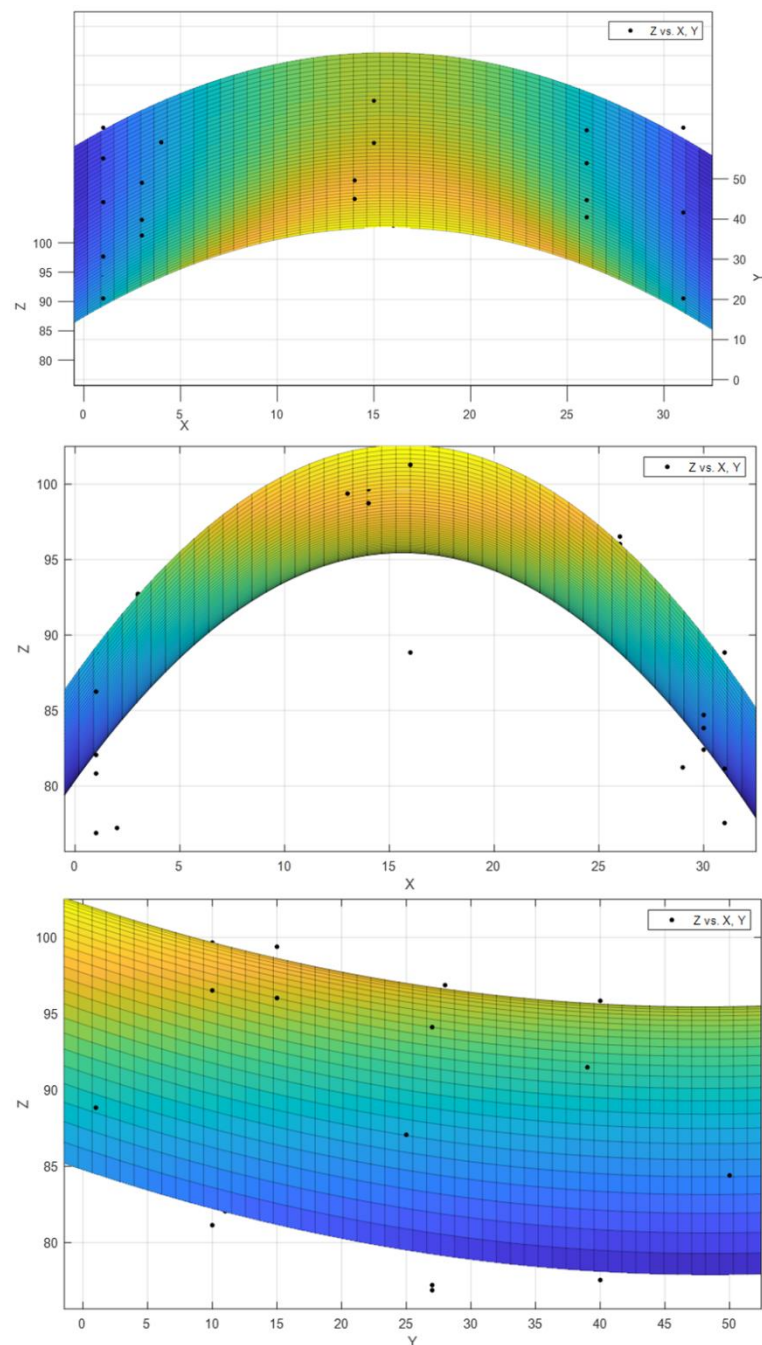


Abb. 5.13: Eine mit der „Curve Fitting Toolbox“ in Matlab erstellte Oberfläche

Eine geeignete Oberflächengleichung kann sein:

$$z(x, y) = 86.96 + 1.939x - 0.2773y - 0.06174x^2 - 0.0002019xy + 0.002925y^2 \quad (5.2)$$

Setzt man abschließend den Wert des Punktes mit dem größten Wert auf dieser Fläche auf 1, dividiert man den Wert der anderen Punkte durch den Maximalwert und nimmt man den Kehrwert, nämlich mit der folgenden Formel berechnet:

$$V_{ij} = \frac{z_{max}}{z_{ij}} \quad (5.3)$$

Dabei sind z_{max} der nach der obigen Formel berechnete Maximalwert von z , z_{ij} der Wert am Punkt (i, j) . V_{ij} ist der Wert des Verstärkungsgitters am Punkt (i, j) .

Dann ergibt sich ein Verstärkungsgitter, wie in der folgenden Abbildung (Abb. 5.14) gezeigt:

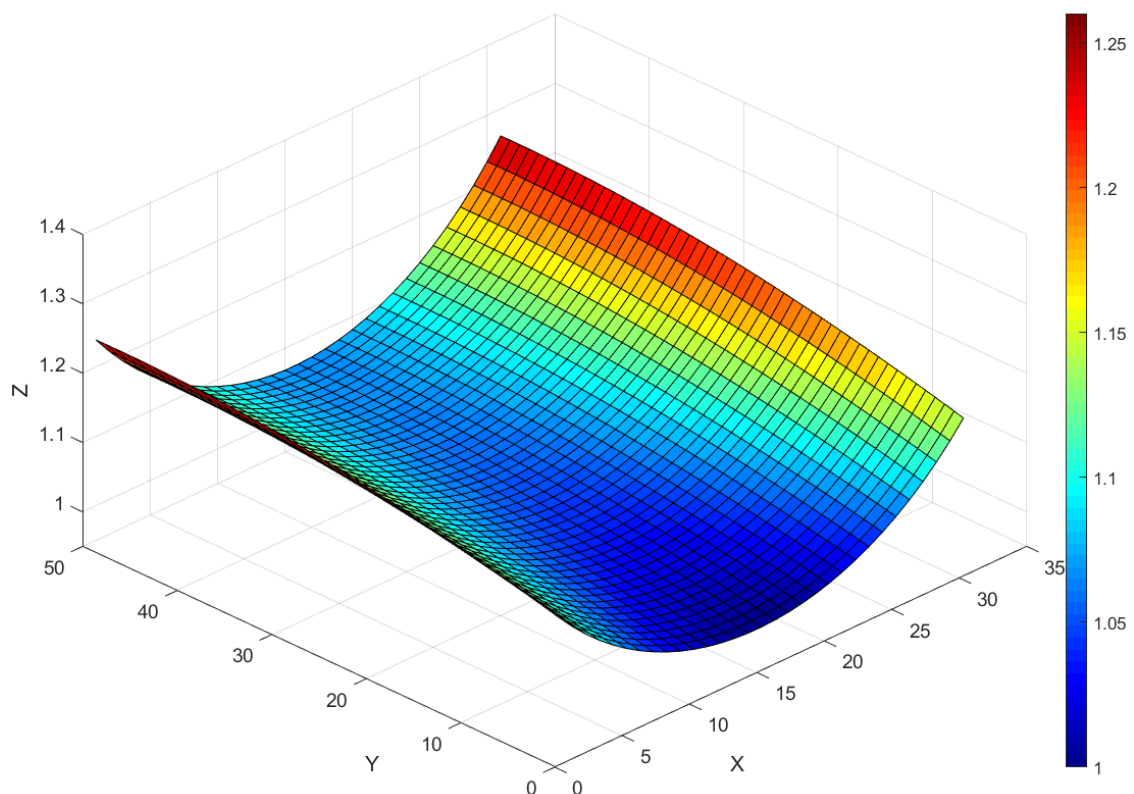


Abb. 5.14: Das 50 × 31-Verstärkungsgitter

5.3 Beispiel für die Verwendung des Verstärkungsgitters

Beispielsweise nimmt man das Signal eines 10 mm dicken Aluminiumpfostens bei 1500 mm und 80 °, um die Messergebnisse nach dem Anwenden vom Verstärkungsgitter zu zeigen. Wenn das Verstärkungsgitter nicht verwendet wird, ist das Ergebnis der 2D-FFT wie folgt. Es ist ersichtlich, dass der Aluminiumpfosten nicht als Hindernis erkannt wird:

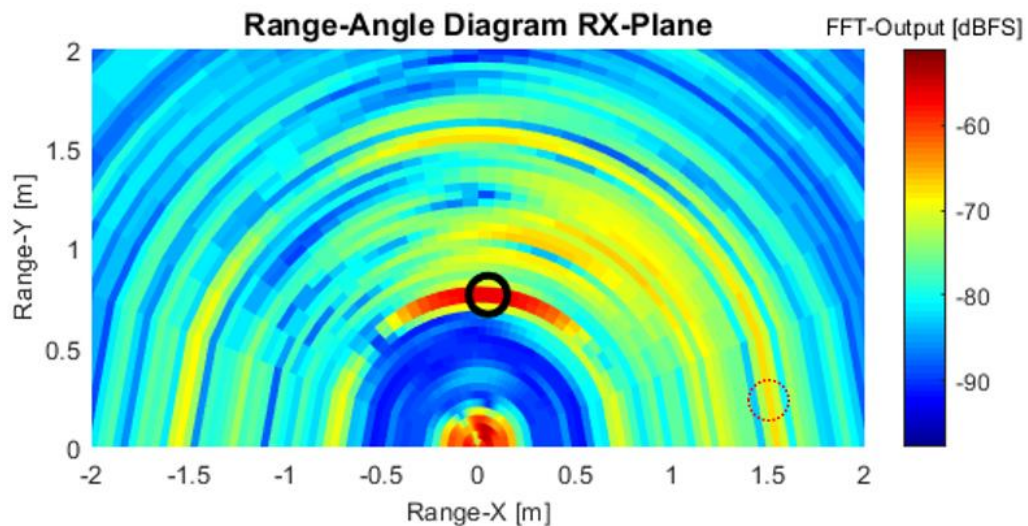


Abb. 5.15: Eine nicht durch das Verstärkungsgitter verarbeitete Messung (Das Hindernis an der Position des gepunkteten Kreises wird nicht erkannt.)

Wenn das Verstärkungsgitter zur Verarbeitung der 2D-FFT verwendet wird, kann man folgendes Ergebnis mit Aluminiumpfostenerkennung sehen:

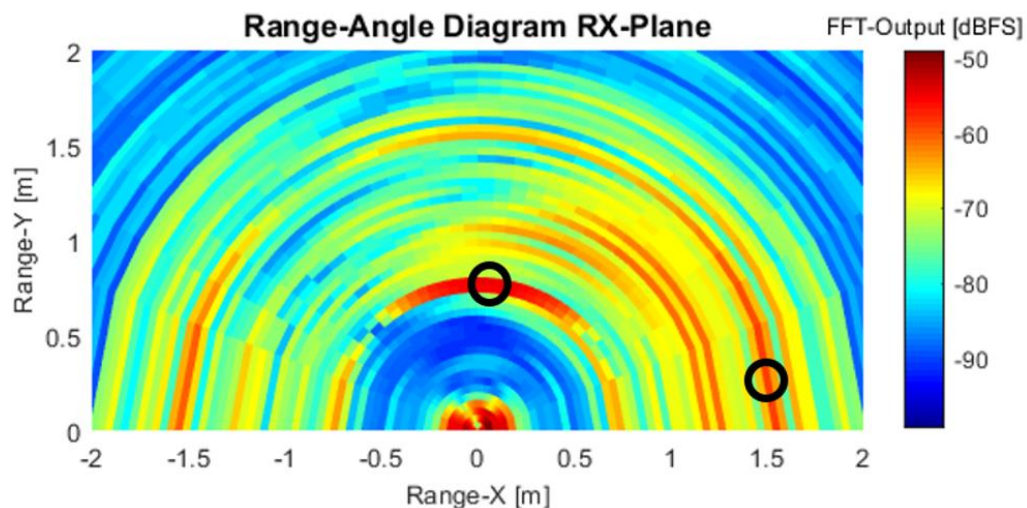


Abb. 5.16: Die durch das Verstärkungsgitter verarbeitete Messung

Die Verwendung des Verstärkungsgitters muss mit der Anpassung der CFAR-Parameter kombiniert werden, um die besten Ergebnisse zu erzielen.

6. Algorithmus für Leermessung

Im letzten Kapitel werden zwei Herausforderungen bei der Radarmessung in der Realität erwähnt. Eines davon ist, dass die Signalintensität in verschiedenen Bereichen unterschiedlich sind. Beispielsweise werden Radarsignale, die aus einem größeren Winkel und in größerer Entfernung zurückgeworfen werden, schwächer. Die Verstärkungsgitter-Methode aus dem vorherigen Kapitel wird verwendet, um dieses Problem zu lösen.

Ein weiteres Problem, das im vorherigen Kapitel erwähnt wird, ist, dass das von einigen Hindernissen reflektierte Signal zu schwach ist, z. B. das von dem Holzpfeiler reflektierte Signal. Dies führt zu einer unzureichenden Stärke des Hindernissignals, um einen deutlichen Peak im Spektrum zu bilden. Eine sehr effektive Lösung besteht darin, das Signalspektrum ohne Hindernisse vom Signalspektrum mit Hindernissen zu subtrahieren, damit die Differenz zwischen beiden gut dargestellt werden kann. Bei tatsächlichen Messungen kann man jedoch nicht immer durch eine einzige Messung feststellen, ob sich im Radarmessbereich Hindernisse befinden. In diesem Kapitel wird eine Methode vorgestellt, bei der Hindernisse aus verschiedenen Materialien und Formen an verschiedenen Stellen innerhalb des Radarmessbereichs platziert werden, während die Radarposition unverändert bleibt. Durch Vergleichen der Unterschiede zwischen diesen Messungen kann ein Leermessungsspektrum abgeleitet und verwendet werden, um zu vergleichen, ob sich bei einer weiteren Messung in derselben Umgebung ein Hindernis im Sichtbereich befindet.

6.1 Generierung des Spektrums ohne Hindernisse

Auch hier werden die in Kapitel 4.2 beschriebenen "Field of View" -Messungen verwendet, um diesen Prozess zu demonstrieren. Aus allen Messungen werden zufällig einige Messungen ausgewählt, beispielsweise drei Messungen des Aluminiumpfostens mit 70 mm Durchmesser bei 1500 mm und -45° , 1000 mm und 80° beziehungsweise 500 mm und 0° .

Man nehme die Messung bei 1500 mm und -45° als die erste Messung. Der exakt gleiche Prozess wie in Kapitel 4.2 beschrieben wird durchgeführt, bis die 2D-FFT dieser Messung erhalten wird. Die Ergebnisse sind in der folgenden Abbildung (Abb. 6.1) dargestellt. In der Abbildung sieht man, dass es ein Maximum bei 1,5 m und -45° gibt, das das Hindernis darstellt. Die andere Spitze erscheint bei 0,7 Metern, 0° , was das zum Boden korrespondierende Maximum ist und kein Hindernis darstellt. Um sie von den Signalpeaks anderer Hindernisse zu unterscheiden, ist es notwendig, diesen Signalpeak im erzeugten Leermessungsspektrum durch Vergleich verschiedener Messungen zu erhalten.

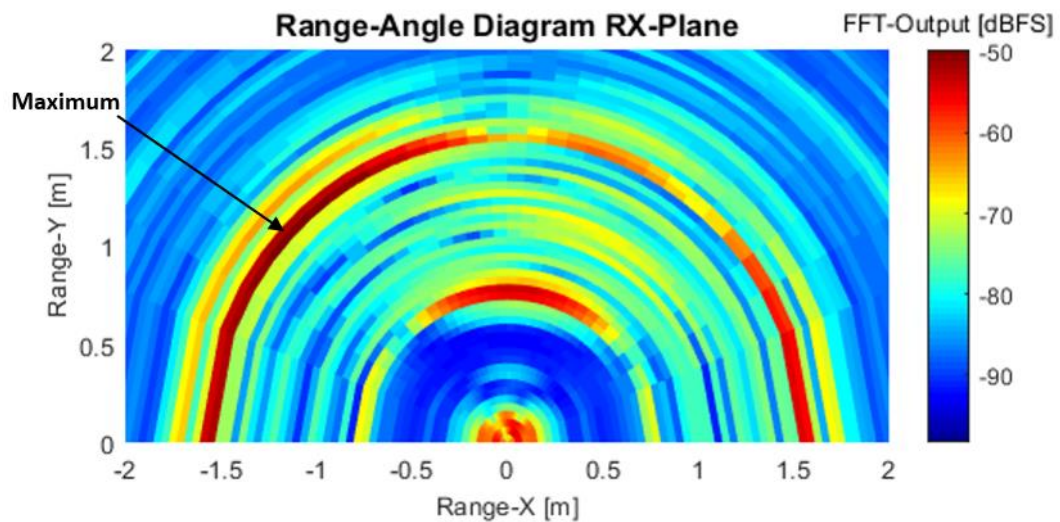


Abb. 6.1: Signalspektrum des Aluminiumpfostens bei 1500 mm, -45°

Wenn im Radarmessbereich ein neues Hindernis vorhanden ist, erscheint im Messspektrum ein neuer Peak. Es ist bei ähnlicher Umgebung nur notwendig, diesen neuen Peak mit dem Rauschen an derselben Position in anderen Messungen zu vergleichen, um festzustellen, ob der neue Peak ein neues Hindernis ist. Das an jedem Punkt im Messbereich zurückgegebene Rauschsignal ist jedoch bei jeder Messung unterschiedlich, genauso wie die Gesamtleistung. Deswegen ist im zweidimensionalen Spektrum nur der Wert jedes Peaks von Bedeutung, und die Punkte deutlich unterhalb des Peaks sind vernachlässigbar. Um die Auswirkungen des unterschiedlichen Rauschens bei jeder Messung zu beseitigen und den Vergleich später zu vereinfachen, kann man den Durchschnittswert des gesamten Spektrums innerhalb des Messbereichs ermitteln und den Wert aller Punkte unter dem 0,8-fachen des Durchschnittswerts auf das 0,8-fache des Durchschnittswerts setzen, wie in der folgenden Abbildung (Abb. 6.2) dargestellt:

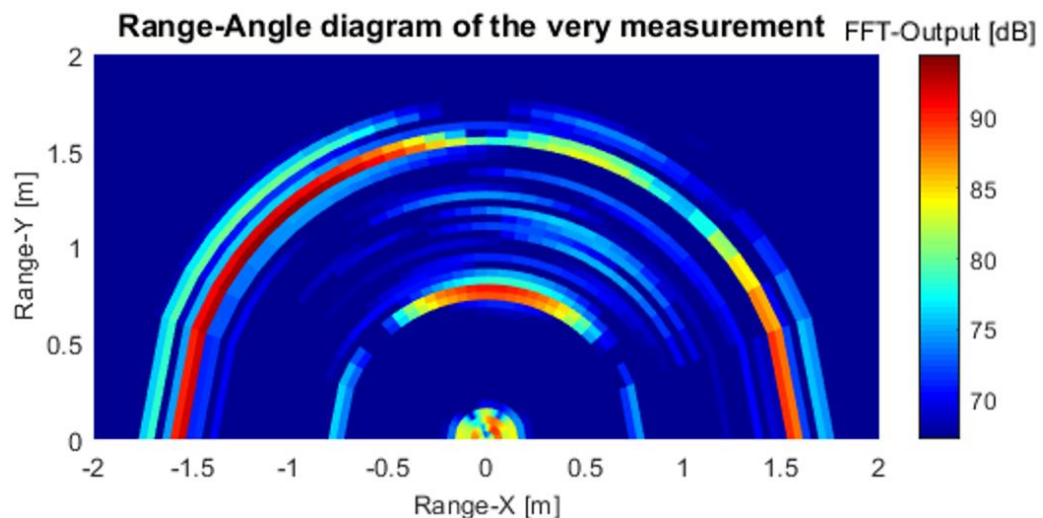


Abb. 6.2: Die Signalpunkte unter dem 0,8-fachen der Durchschnittsleistung werden auf das 0,8-fache der Durchschnittsleistung gesetzt

0,8 ist ein Parameter, der durch mehrere Versuche bestimmt wird. Auf diese Weise werden die Rauschinformationen im Spektrum entfernt. Die Peakinformationen jedes vorhandenen Hindernisses, wie z. B. die vom Boden reflektierten Signalinformationen, bleiben erhalten.

Da die absolute Signalleistung nicht aussagekräftig ist und nur die Leistungsdifferenz zwischen dem Peak und dem Rauschen eine Rolle spielt, wird die Leistungseinheit (dB) nach der FFT-Berechnung verwendet. Diese Messung wird als die erste „Leermessung“ verwendet, und daraufhin die Verarbeitung der zweiten Messung mit Hindernis bei 1000mm und 80° wird durchgeführt. Analog zur ersten Messung erhält man die folgende Abbildung (Abb. 6.3):

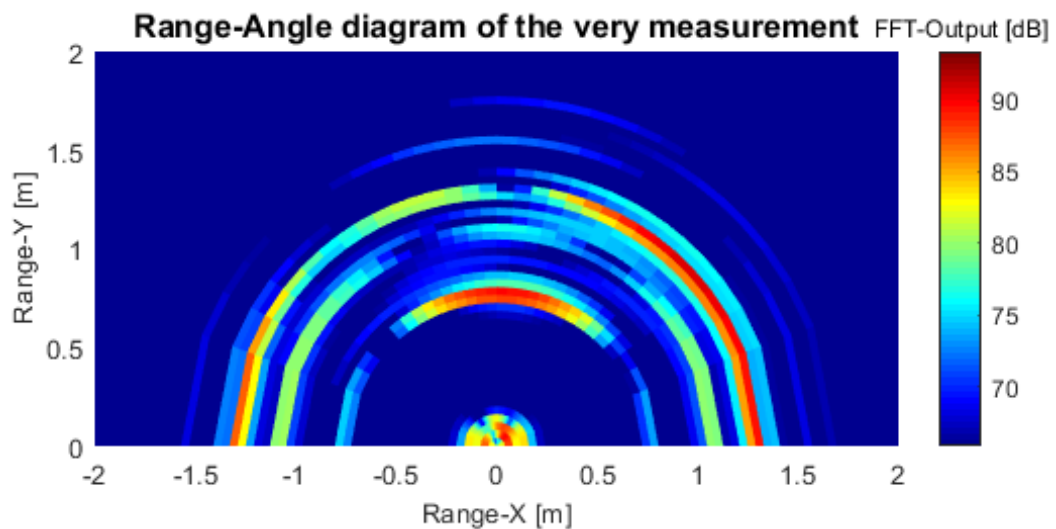


Abb. 6.3: Die zweite Messung nach der Verarbeitung analog zu Abb. 6.2

Die Durchschnitte der zwei Messungen werden subtrahiert, und die Differenz wird zu jedem Punkt der Messung addiert, die einen kleinen Durchschnitt besitzt, so dass die beiden Messungen auf derselben Basis verglichen werden können. Die durchschnittliche Leistung aller Punkten bei der ersten Messung, sei P_1 . Die durchschnittliche Leistung aller Punkten bei der zweiten Messung, sei P_2 . Und angenommen, dass $P_1 > P_2$ ist. Um dann die relativen Höhen jedes Peaks in den beiden Messungen zu vergleichen, ist die neue Leistung P_{2ijnew} bei einem Punkt (i, j) in der zweiten Messung:

$$P_{2ijnew} = P_{2ij} + P_1 - P_2 \quad (6.1)$$

Vergleicht man nun die Werte der Punkte der beiden Messungen und nimmt man an jedem Punkt den kleineren Wert, um eine neue Leermessung zu erhalten, ergibt sich dieses Spektrum:

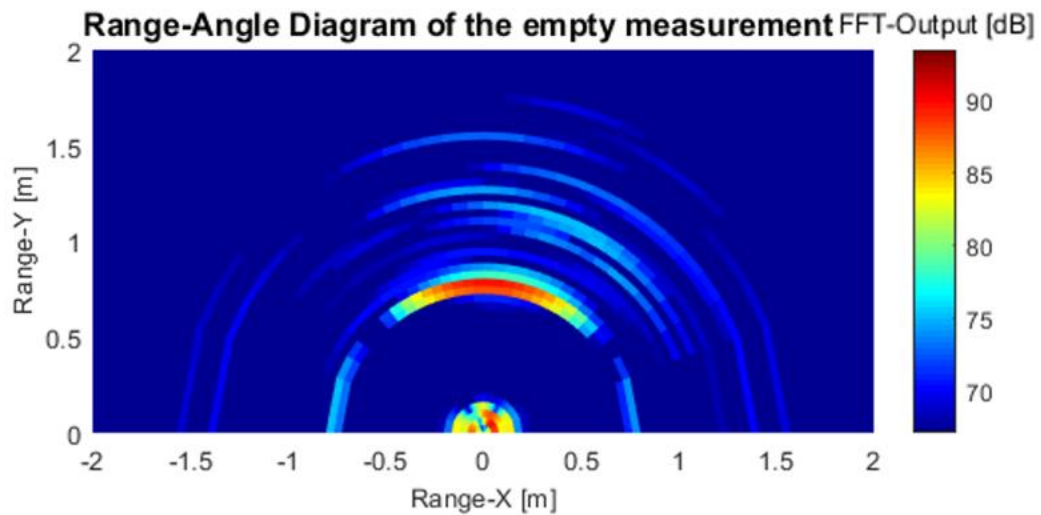


Abb. 6.4: Leermessung generiert aus zwei Messungen

Es ist zu sehen, dass die neue Leermessung nur einen Peak enthält, da der niedrigere Wert bei jedem Punkt der beiden Messungen genutzt wird. Dieser Peak ist der Bodenreflexion geschuldet.

Die Leermessung aus nur zwei Messungen ist noch relativ ungenau. Es werden weitere Messungen hinzugefügt, um die Leermessung genauer zu machen, z. B. eine dritte Messung bei 500 mm, 0 °. Danach sieht das Ergebnis der Leermessung wie folgt aus:

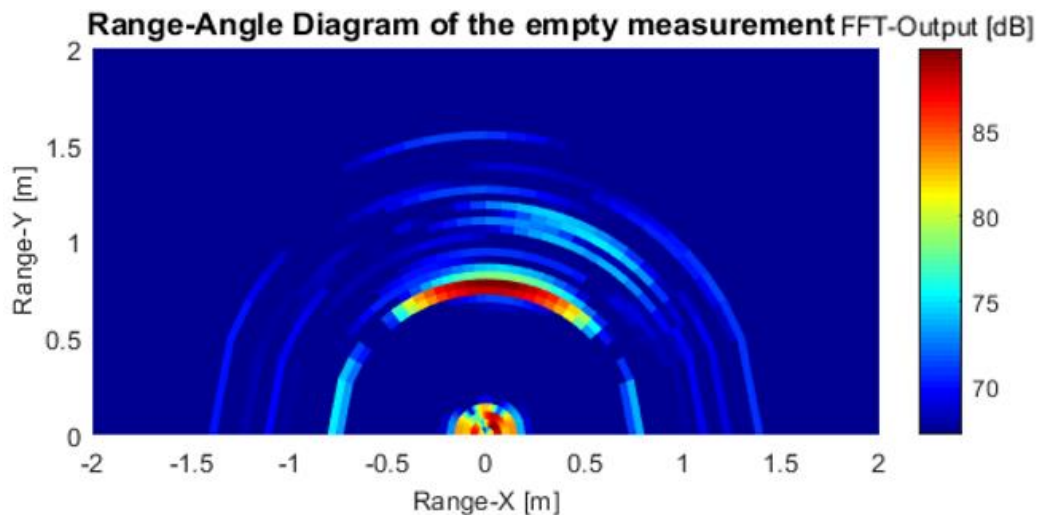


Abb. 6.5: Leermessung generiert aus drei Messungen

Wenn man dann das Ergebnis einer Messung durch das Spektrum der Leermessung subtrahiert, kann man das Hindernis genau erkennen, wie z. B. das Hindernis bei 1,5m und 45°:

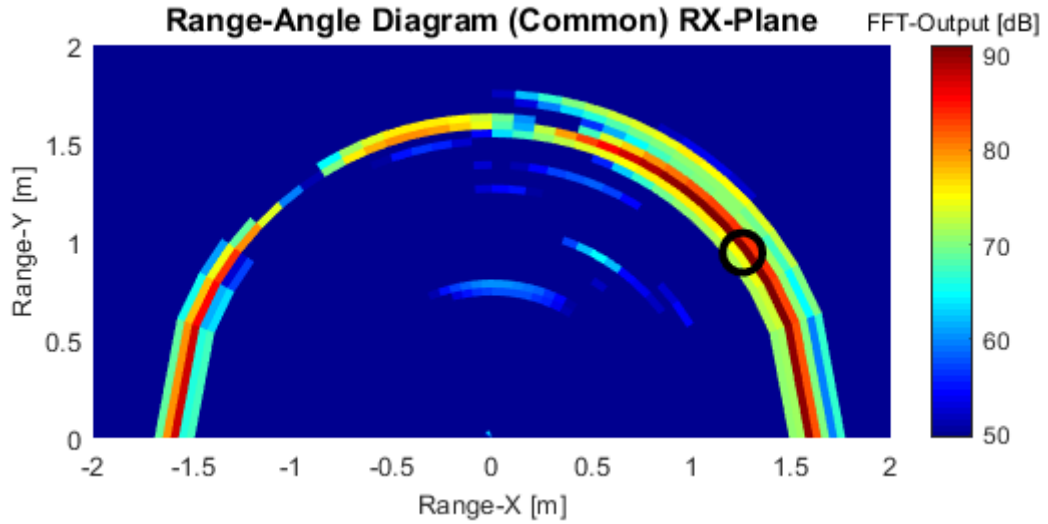


Abb. 6.6: Vergleich der neuen Messung mit Leermessung

Der Peak des Bodens im Spektrum verschwindet und es bleibt nur der Peak des Hindernisses.

6.2 Kompensation für das Ergebnis der Leermessung

Beim Vergleich der Ergebnisse mehrerer Messungen stellt man fest, dass an einem Punkt, an dem es eigentlich kein Hindernis gibt, zwischen zwei Messungen ein Leistungsunterschied von einigen dB bestehen kann. Mit Ausnahme der Position des Hindernisses sind die Messbedingungen tatsächlich unverändert. Um das Leerspektrum so genau wie möglich zu generieren, ist es daher vorzuziehen, die Durchschnittsleistung der Leermessung auf die Durchschnittsleistung aller Messungen abzustimmen. Das heißt, die theoretische durchschnittliche Leistung der n-ten Leermessung ist gegeben durch:

$$P_{D_n} = \frac{(n-1)P_{D_{n-1}} + P_n}{n} \quad (6.2)$$

$$P_{D_1} = P_1 \quad (6.3)$$

Dabei sind P_{D_n} die theoretische Durchschnittsleistung der n-ten Leermessung, $P_{D_{n-1}}$ die theoretische Durchschnittsleistung der n-1-ten Leermessung, P_n die Durchschnittsleistung der n-ten Messung und n die Anzahl der Messungen.

Unter der Annahme, dass die tatsächliche Durchschnittsleistung der x-ten Leermessung P_x ist, sollte jeder Punkt der x-ten Leermessung mit dem Kompensationswert der Leistung addiert werden. Das ist:

$$P_{Comp} = P_{D_n} - P_x \quad (6.4)$$

$$P_{ij_{new}} = P_{ij} + P_{Comp} \quad (6.5)$$

Dabei ist P_{Comp} die Kompensationsleistung, die aus der Durchschnittsleistung berechnet wird, P_{ij} die Leistung eines beliebigen Punktes im Leermessungsspektrum und $P_{ij_{new}}$ die kompensierte Punktleistung.

Ein weiteres Problem beim Ermitteln der Leermessung besteht darin, dass es einen Fall geben kann, in dem der Wert an einem bestimmten Punkt auf einen Fehler zurückzuführen ist, wenn der niedrigste Wert jeder Messung an dem Punkt genommen wird. Dieser Punkt erhält also einen zu niedrigen Wert. Zu niedrig, sodass dieser Punkt bei jedem Vergleich als Hindernis erkannt wird. Um den Fall zu korrigieren, in dem der Wert eines Punktes plötzlich zu niedrig wird und einen Fehler verursacht, kann man die folgende Formel verwenden, um den Wert jedes Punktes der Leermessung zu kompensieren.

$$X_{L_{nk}} = X_{L_n} + (X_n - X_{L_{n-1}}) \times \varepsilon_1(t) \times \alpha + (X_{L_{n-1}} - X_n) \times \varepsilon_2(t) \times \beta \quad (6.6)$$

Dabei sind $X_{L_{nk}}$ der Wert eines Punktes der Leermessung nach der Kompensation, X_{L_n} der Wert eines Punktes der Leermessung vor der Kompensation, α eine Zufallszahl von 0,05 bis 0,2 und β eine Zufallszahl von 0,2 bis 0,5. Die Werte von $\varepsilon_1(t)$ und $\varepsilon_2(t)$ werden durch die folgenden Funktionen bestimmt:

$$\varepsilon_1(t) \begin{cases} = 1, t = X_n - X_{L_{n-1}} > 0 \\ = 0, t = X_n - X_{L_{n-1}} \leq 0 \end{cases} \quad (6.7)$$

$$\varepsilon_2(t) \begin{cases} = 1, t = X_n - X_{L_{n-1}} < 0 \\ = 0, t = X_n - X_{L_{n-1}} \geq 0 \end{cases} \quad (6.8)$$

Der erste Term der obigen Formel kompensiert die ursprüngliche Leermessung mit der neuen Messung, um die eventuell aufgetretenen zu kleinen Werte auszugleichen. Im zweiten Term der obigen Formel wird Leermessung verwendet, um die neue Messung zu kompensieren, um die kleinen Werte zu kompensieren, die bei der neuen Messung auftreten können. Beide Kompensationen werden mit einer Zufallszahl als Gewicht multipliziert, um die Fehlertoleranz des Kompensationsalgorithmus zu verbessern.

6.3 Erkennung von schwach reflektierenden Materialien mit Algorithmus der Leermessung

Mit der Methode der Leermessung kann man Hindernisse mit schwachem Reflexionsvermögen erkennen. Wenn beispielsweise ein Kunststoffpfosten mit einem Durchmesser von nur 10 mm bei 1000 mm und 45 ° platziert wird, ist es schwierig, diesen in einer allgemeinen 2D-FFT zu erfassen:

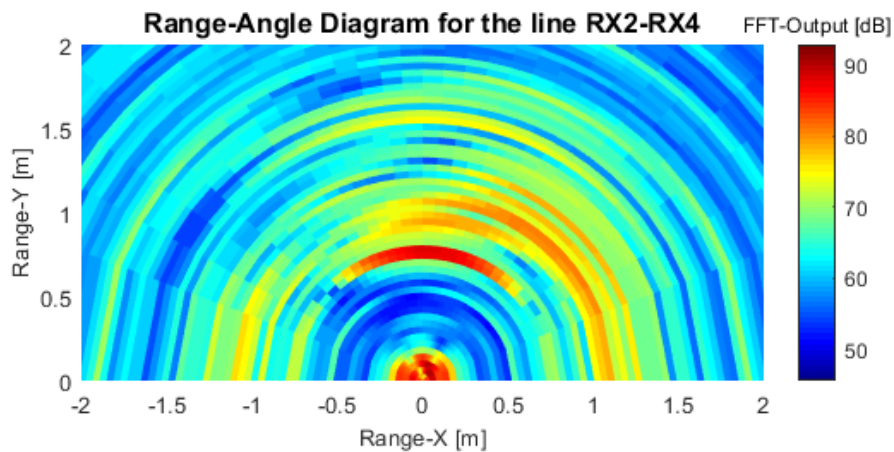


Abb. 6.7: Messergebnisse des 10mm-Kunststoffpfostens bei 1000 mm, 45°

Im Vergleich mit einer Leermessung ist seine Position jedoch klar zu ermitteln.

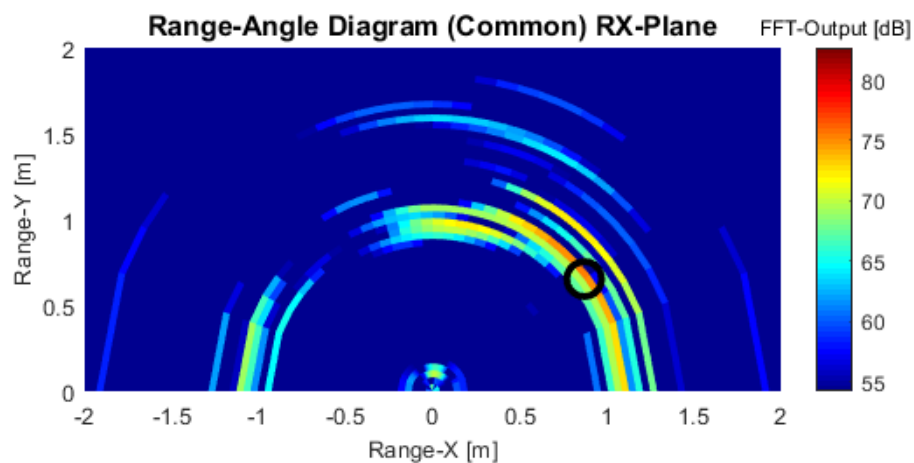


Abb. 6.8: Kunststoffpfostenmessung im Vergleich zur Leermessung

Im Allgemeinen wird die resultierende Leermessung mit zunehmender Anzahl von Messungen präziser.

6.4 Praktische Anwendung des Leermessung-Algorithmus

Wenn sich die Messumgebung, in der sich das Radar befindet, ständig ändert, ist die Leermessung-Methode nicht anwendbar. Weil die Leermessung-Methode stationäre Hindernisse in der Umgebung von neu hinzugekommenen Hindernissen unterscheidet, eignet sie sich für Situationen, in denen das Radar stationär ist oder sich mit geringer Geschwindigkeit bewegt. Beispielsweise kann dieses Verfahren in einem automatischen Parksystem für eine Garage verwendet werden. Das Radar kann an der Wand vor der Parklücke installiert werden. Wenn plötzlich ein Hindernis auf der Parklücke auftaucht (z. B. eine Kiste, die von anderen Fahrzeugen abgeworfen wurde), kann das Radar die Kiste durch Vergleichen der zuvor gespeicherten Leermessung schnell finden. Ein weiteres Beispiel ist das im Kapitel 1 erwähnte Projekt "Power Door". Da sich die Bodenbedingungen des Fahrzeugs vor dem Anhalten im Allgemeinen nicht ändern, kann das Radar den Spitzenwert der Bodenreflexion durch Vergleichen der Messungen der letzten fünf Sekunden der Fahrt bestimmen, um in Kombination mit neuen Messungen andere Hindernisse zu finden.

7. Zusammenfassung und Ausblick

In diesem Kapitel werden die in der Arbeit erledigten Hauptaufgaben zusammengefasst und die Aufgaben vorgeschlagen, die im nächsten Schritt ausgeführt werden können.

7.1 Zusammenfassung

Wie in Kapitel 1 beschrieben, besteht der Hauptzweck dieser Arbeit darin, herauszufinden, wie Hindernisse mithilfe eines Radars an der Autotür erkannt und lokalisiert werden, und entsprechende Algorithmen zu schreiben, um die Erkennungsergebnisse zu verbessern. In Kapitel 2 dieser Arbeit werden einige wichtige digitale Signalverarbeitungsalgorithmen vorgestellt und das Messprinzip des FMCW-Radars erläutert. In Kapitel 3 werden der Aufbau und Messergebnisse des Radarmoduls „AWR1642BOOST-ODS“, das Konzept virtueller Antennen und die Verwendung von Radarmessergebnissen zur Bestimmung der Position von Objekten im 3D-Raum beschrieben.

Als nächstes werden in Kapitel 4 die mit dem Radarmodul „das Modul AWR1642BOOST-ODS“ durchgeführten Messversuche beschrieben, um die Leistungen des Radars und den Effekt der Hinderniserkennung zu analysieren. Durch die Analyse der Messergebnisse weiß man, dass das Modul AWR1642BOOST-ODS eine gute Erkennungsfähigkeit für Hindernisse mit sehr geringem Abstand aufweist. Der Radar hat jedoch auch seine Fehleranfälligkeiten, zum Beispiel hat er Schwierigkeiten, Hindernisse bei großen Entfernungen, großen Winkeln und Hindernisse mit schwacher Reflexionsfähigkeit des Radarsignals zu erkennen.

Für das Problem, dass der Radar in verschiedenen Messbereichen unterschiedliche Messwirkungen hat, wird in Kapitel 5 der Algorithmus „Verstärkungsgitter“ vorgeschlagen, um die Radarmessung von Objekten bei großem Abstand und Winkel zu verbessern. Zusätzliche Algorithmen zum Segmentieren von Ebenen mit Dreiecken und Bilden von Oberflächen unter Verwendung mehrerer Punkte im Raum werden auch vorgeschlagen. Die in Kapitel 5 vorgeschlagene Methode verbessert die Fähigkeit des Radars, im sonst blinden Bereich zu messen, indem die Verstärkung des Radars an verschiedenen Punkten innerhalb des Messbereichs geändert wird.

Für das Problem, dass der Radar unterschiedliche Messwirkungen auf Hindernisse mit unterschiedlichen Materialien und Formen hat, wird in Kapitel 6 der Algorithmus "Leermessungsmethode" vorgeschlagen, um das statische Leermessungsspektrum durch Verwendung mehrerer Messungen zu bilden und die Leermessung mit anderen Messungen zu vergleichen. Diese Methode unterscheidet die neuen Hindernisse effektiv von der

ursprünglichen Umgebung, indem eine Leermessung von vielen Messungen erhalten wird, selbst wenn die Hindernisse ein schwaches Reflexionsvermögen aufweisen.

Die beiden Methoden in Kapitel 5 und Kapitel 6 verbessern die Fähigkeit des Radars, Objekte im Ultranahebereich zu messen, und können effektiv für die Messungen in Parkhäusern, Autotüren und unter anderen Bedingungen eingesetzt werden.

7.2 Ausblick

Die Verwendung des FMCW-MIMO-Radars zur Hinderniserkennung beinhaltet eine Vielzahl von Theorien, Techniken und Anwendungsfällen. Basierend auf den Entwicklungen im Rahmen dieser Abschlussarbeit, ist es möglich, die Technologie zur Erkennung von Hindernissen mithilfe von FMCW-MIMO-Radar in Zukunft weiterzuentwickeln und zu verbessern. Beispielsweise können einige DOA (Direction of Arrival)-Algorithmen wie MUSIC oder CAPON-Beamforming zur Verarbeitung von Radarmesssignalen verwendet werden. Diese Algorithmen können die Hinderniserkennung verbessern, indem sie mit den in diesem Artikel beschriebenen Algorithmen „Verstärkungsgitter“ und „Leermessung“ kombiniert werden. Darüber hinaus wurde bisher die Messleistung des Radars unter anderen Wetterbedingungen nicht untersucht. Einige Algorithmen könnten entwickelt werden, die die Auswirkungen unterschiedlicher Wetterbedingungen auf Radarmessungen kompensieren.

Alle oben genannten Verbesserungen sind nur auf die Algorithmik bezogen, allerdings wären einige Hardware- und Systemverbesserungen ebenfalls von Bedeutung. Beispielsweise ist die Interaktion zwischen den Radaren mehrerer Fahrzeuge, die ein Netzwerk bilden, ein Forschungsthema. Der Schwerpunkt des Problems ist, wie ein Radar die von den Radaren auf anderen Fahrzeugen übertragenen Signale unterscheiden und darauf reagieren kann. Darüber hinaus ist die Kombination und Datenfusion mehrerer Sensoren, wie beispielsweise die Kombination von Radar und Kamera, Radar und Ultraschallsensoren, ein wichtiges Forschungsthema. Außerdem ist es ein gutes Forschungsthema, wie die Größe von Hindernissen geschätzt und die Arten von Hindernissen erkannt werden können. Dies stellt nicht nur höhere Anforderungen an Hinderniserkennungsalgorithmen, sondern erfordert auch den Abgleich und die Kommunikation verschiedener Hardware.

Literaturverzeichnis

- [1] Jonah Gamba, *Fundamentals of Radar Systems*, Radar Signal Processing for Autonomous Driving, Springer Nature Singapore Pte Ltd. 2020: S. 1

- [2] M. Vorderderfler, M. E. Gadringer, H. Schreiber, A. Gruber, W. Bösch, S. Metzner, H. Pflügl, M. Paulweber, *Frequency dividers in radar target stimulator application*, Elektrotechnik & Informationstechnik 2018, S. 345

- [3] M. Tröscher, R. Weigel, *Ein KFZ-Radarsensor als Kollisionsfrühwarnsystem*, e & i Elektrotechnik und Informationstechnik, 2001: S. 49

- [4] Jens Gayko, *Lane Departure Warning*, Handbuch Fahrerassistenzsysteme, Vieweg+Teubner 2009: S. 543

- [5] Additi Mrinal Singh, Soumyasree Bera and Rabindranath Bera, *Review on Vehicular Radar for Road Safety*, Advances in Communication, Cloud, and Big Data, Springer Nature Singapore Pte Ltd. 2019: SS. 41-42

- [6] Hermann Winner, *Radarsensorik*, Handbuch Fahrerassistenzsysteme, Vieweg+Teubner 2015: S. 260

- [7] Karthik Ramasubramanian, Kishore Ramaiah, Artem Aginskiy, *Moving from legacy 24 GHz to state-of-the-art 77 GHz radar*, Texas Instruments Incorporated, 2017: S. 3

- [8] S. K. Bose, *Fast Fourier Transform*, Numerical Methods of Mathematics Implemented in Fortran, Springer Nature Switzerland AG, 2019: S. 444

- [9] K. Iizuka, *The Fast Fourier Transform (FFT)*, Springer Nature Switzerland, 2019: S. 197

- [10] Mark A. Richards, *Fundamentals of Radar Signal Processing*, McGraw Hill Electronic Engineering, 2005: S. 348

- [11] E. Hyun, Y. S. Jin, J. H. Lee, *Design and Development of Automotive Blind Spot Detection Radar System based on ROI Pre-processing Scheme*, International Journal of Automotive Technology, Vol. 18, No. 1: S. 167
- [12] Shannon Hilbert, *FFT Zero Padding*, Digital Signal Processing, 04.22.2013, <http://www.bitweenie.com/listings/fft-zero-padding/>
- [13] Prof. Dr.-Ing. Peter Knoll, *Sensorik für Fahrzeugrundumsicht, Fahrstabilisierungssysteme und Fahrerassistenzsysteme*, Vieweg+Teubner 2010: S. 130
- [14] Ian Sharp, Kegen Yu, *Signaling Techniques*, Wireless Positioning: Principles and Practice, Springer Nature Singapore Pte Ltd. 2019: S. 45
- [15] Sandeep Rao, *Introduction to mmwave Sensing: FMCW Radars*, Texas Instruments Incorporated, <https://training.ti.com/mmwave-training-series>
- [16] Texas Instruments, *AWR1642 Obstacle Detection Sensor*, Texas Instruments Incorporated, 2018: S. 3
- [17] Texas Instruments, *DCA1000EVM Data Capture Card*, Texas Instruments Incorporated, 2018: S. 3-8
- [18] Texas Instruments, *RadarStudio_matlab_scaling*, Texas Instruments Incorporated, https://e2e.ti.com/cfs-file/__key/communityserver-discussions-components-files/1023/RadarStudio_5F00_matlab_5F00_scaling.pdf

Danksagung

Frau Dipl.-Ing. Katharina Burger von der CPT Group GmbH half mir viel beim Verfassen dieser Arbeit. Sie hat meine Bachelorarbeit mit absoluter Geduld, Sorgfalt und Professionalität überarbeitet. Ich möchte Frau Burger für Ihre Hilfe aufrichtig danken.

Anhang

1. Webalgorithmus

```
clc;
clear all force;
close all

Z=xlsread('gitterwithcorner.xls');
numPoints = sum(sum(Z~=0));
coordinate = zeros(2,numPoints);
coordinate(1,1) = 1;
coordinate(2,1) = 1;

[row,col]=find(Z~=0);

Z1 = zeros(length(Z(:,1)),length(Z(1,:)));
Z1(coordinate(1,1),coordinate(2,1)) = 1;

currentPoint=2;
for radius = 1:60
    for y = 1:31
        for x = 1:50
            if (Z(x,y)~=0)&&(sqrt((x-1)^2+(y-1)^2)<radius)&&(Z1(x,y)~=1)
                coordinate(1,currentPoint) = x;
                coordinate(2,currentPoint) = y;
                Z1(coordinate(1,currentPoint),coordinate(2,currentPoint)) =
1;
                currentPoint = currentPoint+1;
            end
        end
    end
end

numTriangle = 2*(numPoints-4)+2;
coordinateTriangle = cell(3,numTriangle);
Trianglematrix = zeros(6,numTriangle);
currTria=1;
Z2 = zeros(100*length(Z(:,1)),100*length(Z(1,:)));

for y = 1:31
    for x = 1:50
        if Z1(x,y) == 1
            Z2(round(100*x),round(100*y))=1;
        end
    end
end

for cPoints = 3:numPoints
    for iindex=1:(cPoints-2)
        for jindex=(iindex+1):(cPoints-1)
            kindex=cPoints;
            currCoor1= [coordinate(1,iindex),coordinate(2,iindex)];
            currCoor2= [coordinate(1,jindex),coordinate(2,jindex)];
            currCoor3= [coordinate(1,kindex),coordinate(2,kindex)];
```

```

x1 = currCoor1(1);
y1 = currCoor1(2);
x2 = currCoor2(1);
y2 = currCoor2(2);
x3 = currCoor3(1);
y3 = currCoor3(2);

Angle1 = acos(((x2-x1)*(x3-x1)+(y2-y1)*(y3-y1))/sqrt(((x2-x1)^2+(y2-y1)^2)*((x3-x1)^2+(y3-y1)^2)));
Angle2 = acos(((x1-x2)*(x3-x2)+(y1-y2)*(y3-y2))/sqrt(((x1-x2)^2+(y1-y2)^2)*((x3-x2)^2+(y3-y2)^2)));
Angle3 = acos(((x1-x3)*(x2-x3)+(y1-y3)*(y2-y3))/sqrt(((x1-x3)^2+(y1-y3)^2)*((x2-x3)^2+(y2-y3)^2)));

S = abs((x1*y2+x2*y3+x3*y1-x2*y1-x3*y2-x1*y3) / 2);

if (S>0) && (Angle1<2) && (Angle2<2) && (Angle3<2)
    count=0;
    isTrangle=1;
    for x =1:0.01:51
        for y=1:0.01:51
            S1 = abs((x1*y+x2*y1+x*y2-x2*y-x*y1-x1*y2)/2);
            S2 = abs((x1*y+x3*y1+x*y3-x3*y-x*y1-x1*y3)/2);
            S3 = abs((x3*y+x2*y3+x*y2-x2*y-x*y3-x3*y2)/2);
            if (S1+S2+S3==S) && (Z2(round(100*x),round(100*y)) ==
1) && (S1>1e-6) && (S2>1e-6) && (S3>1e-6)
                isTrangle=0;
                break;
            end
            %if (S1+S2+S3==S) && (Z2(round(100*x),round(100*y))
== 0) && (S1~=0) && (S2~=0) && (S3~=0) && (rem(x,1)==0) && (rem(y,1)==0)
                if (S1+S2+S3==S) && (Z2(round(100*x),round(100*y)) ==
0) && (S1>1e-6) && (S2>1e-6) && (S3>1e-6)
                    count=count+1;
                end
            end
            if (S1+S2+S3==S) && (Z2(round(100*x),round(100*y)) ==
1) && (S1>1e-6) && (S2>1e-6) && (S3>1e-6)
                isTrangle=0;
                break;
            end
        end
    end

if (isTrangle==1) && (count>0)
    coordinateTriangle(1,currTria)=[x1,y1];
    coordinateTriangle(2,currTria)=[x2,y2];
    coordinateTriangle(3,currTria)=[x3,y3];

    Trianglematrix(1,currTria)=x1;
    Trianglematrix(2,currTria)=y1;
    Trianglematrix(3,currTria)=x2;
    Trianglematrix(4,currTria)=y2;
    Trianglematrix(5,currTria)=x3;
    Trianglematrix(6,currTria)=y3;

    currTria = currTria+1;
    figure(1);
    subplot(1,1,1);
    ax = gca;
    ax.Color = 'none';
    hold on;

```

```

        plot([x1,x2],[y1,y2],'r');
        plot([x1,x3],[y1,y3],'r');
        plot([x2,x3],[y2,y3],'r');

        for x =1:0.01:51
            for y=1:0.01:51
                S1 = abs((x1*y+x2*y1+x*y2-x2*y-x*y1-x1*y2)/2);
                S2 = abs((x1*y+x3*y1+x*y3-x3*y-x*y1-x1*y3)/2);
                S3 = abs((x3*y+x2*y3+x*y2-x2*y-x*y3-x3*y2)/2);
                if
(S1+S2+S3==S) && (Z2(round(100*x),round(100*y)) ~= 1)
                    Z2(round(100*x),round(100*y))=1;
                end
            end
        end
    end
end

for cPoints = 3:numPoints
    for iindex=1:(cPoints-2)
        for jindex=(iindex+1):(cPoints-1)
            kindex=cPoints;
            currCoor1= [coordinate(1,iindex),coordinate(2,iindex)];
            currCoor2= [coordinate(1,jindex),coordinate(2,jindex)];
            currCoor3= [coordinate(1,kindex),coordinate(2,kindex)];

            x1 = currCoor1(1);
            y1 = currCoor1(2);
            x2 = currCoor2(1);
            y2 = currCoor2(2);
            x3 = currCoor3(1);
            y3 = currCoor3(2);

            Angle1 = acos(((x2-x1)*(x3-x1)+(y2-y1)*(y3-y1))/sqrt(((x2-
x1)^2+(y2-y1)^2)*((x3-x1)^2+(y3-y1)^2)));
            Angle2 = acos(((x1-x2)*(x3-x2)+(y1-y2)*(y3-y2))/sqrt(((x1-
x2)^2+(y1-y2)^2)*((x3-x2)^2+(y3-y2)^2)));
            Angle3 = acos(((x1-x3)*(x2-x3)+(y1-y3)*(y2-y3))/sqrt(((x1-
x3)^2+(y1-y3)^2)*((x2-x3)^2+(y2-y3)^2)));

            S = abs((x1*y2+x2*y3+x3*y1-x2*y1-x3*y2-x1*y3) / 2);

            if (S>0)
                count=0;
                isTrangle=1;
                for x =1:0.01:51
                    for y=1:0.01:51
                        S1 = abs((x1*y+x2*y1+x*y2-x2*y-x*y1-x1*y2)/2);
                        S2 = abs((x1*y+x3*y1+x*y3-x3*y-x*y1-x1*y3)/2);
                        S3 = abs((x3*y+x2*y3+x*y2-x2*y-x*y3-x3*y2)/2);
                        if (S1+S2+S3==S) && (Z2(round(100*x),round(100*y)) ==
1) && (S1>1e-6) && (S2>1e-6) && (S3>1e-6)
                            isTrangle=0;
                            break;
                        end
                    %if (S1+S2+S3==S) && (Z2(round(100*x),round(100*y))
== 0) && (S1~=0) && (S2~=0) && (S3~=0) && (rem(x,1)==0) && (rem(y,1)==0)

```


2. Algorithmus zum Konstruieren von Oberflächen mit drei Punkten im Raum

```

clear all force;
close all;
clc;

x1 = 25;
y1 = 25;
z1 = 1;

x2 = 15;
y2 = 2;
z2 = 2;

x3 = 1;
y3 = 28;
z3 = 4;

%Durchschnitt = (z1+z2+z3)/3;
Durchschnitt = 3;

Z = zeros(30,30);

xo = 1:30;
yo = 1:30;

Z(x1,y1)=1;
Z(x2,y2)=2;
Z(x3,y3)=4;

Xm = (x1+x2+x3)/3;
Ym = (y1+y2+y3)/3;

S = abs((x1*y2+x2*y3+x3*y1-x2*y1-x3*y2-x1*y3) / 2);

d1 = sqrt((Xm-x1)^2+(Ym-y1)^2);
d2 = sqrt((Xm-x2)^2+(Ym-y2)^2);
d3 = sqrt((Xm-x3)^2+(Ym-y3)^2);

%{
for x =-50:50
    for y=-50:50
        S1 = abs((x1*y+x2*y1+x*y2-x2*y-x*y1-x1*y2)/2);
        S2 = abs((x1*y+x3*y1+x*y3-x3*y-x*y1-x1*y3)/2);
        S3 = abs((x3*y+x2*y3+x*y2-x2*y-x*y3-x3*y2)/2);
        if (S1+S2+S3==S) && (sqrt((x-x1)^2+(y-
y1)^2)<d1) && (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
            Z(x,y)=(d1-sqrt((x-x1)^2+(y-y1)^2))/d1*(Z(x1,y1)-
Durchschnitt)+Durchschnitt;
        end
        if (S1+S2+S3==S) && (sqrt((x-x2)^2+(y-
y2)^2)<d2) && (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
            Z(x,y)=(d2-sqrt((x-x2)^2+(y-y2)^2))/d2*(Z(x2,y2)-
Durchschnitt)+Durchschnitt;
        end
    end
end

```

```

        if (S1+S2+S3==S) && (sqrt((x-x3)^2+(y-
y3)^2)<d3) && (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
        Z(x,y)=(d3-sqrt((x-x3)^2+(y-y3)^2))/d3*(Z(x3,y3)-
Durchschnitt)+Durchschnitt;
    end

end

end

%}
%{
&& (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
&& (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
&& (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
%}

for x =-50:50
    for y=-50:50
        S1 = abs((x1*y+x2*y1+x*y2-x2*y-x*y1-x1*y2)/2);
        S2 = abs((x1*y+x3*y1+x*y3-x3*y-x*y1-x1*y3)/2);
        S3 = abs((x3*y+x2*y3+x*y2-x2*y-x*y3-x3*y2)/2);
        if (S1+S2+S3==S) && (sqrt((x-x1)^2+(y-y1)^2)<d1)
            Z(x,y)=(1/(1+exp(1)^(sqrt((x-x1)^2+(y-y1)^2)-d1/2)))*(Z(x1,y1)-
Durchschnitt)+Durchschnitt;
        end
        if (S1+S2+S3==S) && (sqrt((x-x2)^2+(y-y2)^2)<d2)
            Z(x,y)=(1/(1+exp(1)^(sqrt((x-x2)^2+(y-y2)^2)-d2/2)))*(Z(x2,y2)-
Durchschnitt)+Durchschnitt;
        end
        if (S1+S2+S3==S) && (sqrt((x-x3)^2+(y-y3)^2)<d3)
            Z(x,y)=(1/(1+exp(1)^(sqrt((x-x3)^2+(y-y3)^2)-d3/2)))*(Z(x3,y3)-
Durchschnitt)+Durchschnitt;
        end
    end
end

[X,Y] = meshgrid(xo,yo);
gitter = surf(X,Y,Z);
%view(0,120);
xlabel('X','FontSize',11);
ylabel('Y','FontSize',11);
zlabel('Z','FontSize',11);
set(gcf,'color','w');
ax = gca;
ax.FontSize = 14;

```

3. Algorithmus für Verstärkungsgitter

```
clc;
close all;
clear all force;

figure(1);
set(gcf, 'color', 'w');
Z = zeros(50,31);
Z(5,13)=20*log10(9.3035e+04);
Z(8,14)=20*log10(8.6422e+04);
Z(10,31)=20*log10(1.1401e+04);
Z(10,30)=20*log10(1.7194e+04);
Z(10,26)=20*log10(6.7034e+04);
Z(10,14)=20*log10(9.6179e+04);
Z(11,3)=20*log10(4.3328e+04);
Z(10,1)=20*log10(2.0532e+04);
Z(11,1)=20*log10(1.2669e+04);
Z(15,30)=20*log10(1.5555e+04);
Z(15,26)=20*log10(6.3275e+04);
Z(15,14)=20*log10(9.3196e+04);
Z(15,3)=20*log10(4.2874e+04);
Z(15,1)=20*log10(2.0861e+04);
Z(27,30)=20*log10(1.3189e+04);
Z(27,29)=20*log10(1.1518e+04);
Z(27,26)=20*log10(5.0806e+04);
Z(28,15)=20*log10(6.9770e+04);
Z(28,3)=20*log10(3.1901e+04);
Z(27,1)=20*log10(6.9818e+03);
Z(28,1)=20*log10(1.0995e+04);
Z(27,2)=20*log10(7.2579e+03);
Z(40,31)=20*log10(7.5392e+03);
Z(39,26)=20*log10(3.7529e+04);
Z(40,15)=20*log10(6.1958e+04);
Z(40,4)=20*log10(2.7458e+04);
Z(40,1)=20*log10(1.9942e+04);
nx = 31;
ny = 50;

t=0;
n=0;

for x=1:31
    for y=1:50
        if Z(y,x)~=0
            t = t+Z(y,x);
            n = n+1;
        end
    end
end

%{
for x=1:31
```

```

for y=1:50
    if Z(y,x)==0
        Z(y,x)=t/n;
    end
end
end
%}
Durchschnittall = t/n;
Z(1,1)=t/n;
Z(50,1)=0.95*t/n;
Z(1,31)=t/n;
Z(50,31)=0.95*t/n;
Z(1,round(nx/2))=1.10*t/n;
Z(50,round(nx/2))=1.05*t/n;
Z(round(ny/2),1)=t/n;
Z(round(ny/2),31)=0.98*t/n;

Trianglematrix=xlsread('Trianglematrix.xls');
Trianglematrix(:,find(sum(abs(Trianglematrix),1)==0))=[];
numTriangle=length(Trianglematrix(1,:));

for index=1:numTriangle
    x1=Trianglematrix(1,index);
    y1=Trianglematrix(2,index);
    x2=Trianglematrix(3,index);
    y2=Trianglematrix(4,index);
    x3=Trianglematrix(5,index);
    y3=Trianglematrix(6,index);
    Xm = (x1+x2+x3)/3;
    Ym = (y1+y2+y3)/3;

    Durchschnitt=(Z(x1,y1)+Z(x2,y2)+Z(x3,y3))/3;

    S = abs((x1*y2+x2*y3+x3*y1-x2*y1-x3*y2-x1*y3) / 2);

    d1 = sqrt((Xm-x1)^2+(Ym-y1)^2);
    d2 = sqrt((Xm-x2)^2+(Ym-y2)^2);
    d3 = sqrt((Xm-x3)^2+(Ym-y3)^2);

    %{

for x =-50:50
    for y=-50:50
        S1 = abs((x1*y+x2*y1+x*y2-x2*y-x*y1-x1*y2)/2);
        S2 = abs((x1*y+x3*y1+x*y3-x3*y-x*y1-x1*y3)/2);
        S3 = abs((x3*y+x2*y3+x*y2-x2*y-x*y3-x3*y2)/2);
        if (S1+S2+S3==S) && (sqrt((x-x1)^2+(y-
y1)^2)<d1) && (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
            Z(x,y)=(d1-sqrt((x-x1)^2+(y-y1)^2))/d1*(Z(x1,y1)-
Durchschnitt)+Durchschnitt;
        end
        if (S1+S2+S3==S) && (sqrt((x-x2)^2+(y-
y2)^2)<d2) && (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
            Z(x,y)=(d2-sqrt((x-x2)^2+(y-y2)^2))/d2*(Z(x2,y2)-
Durchschnitt)+Durchschnitt;
        end
        if (S1+S2+S3==S) && (sqrt((x-x3)^2+(y-
y3)^2)<d3) && (x<length(xo)) && (y<length(yo)) && (x>0) && (y>0)
            Z(x,y)=(d3-sqrt((x-x3)^2+(y-y3)^2))/d3*(Z(x3,y3)-
Durchschnitt)+Durchschnitt;

```

```

        end

    end

end

    %}
    %{
    &&(x<length(xo)) &&(y<length(yo)) &&(x>0) &&(y>0)
    &&(x<length(xo)) &&(y<length(yo)) &&(x>0) &&(y>0)
    &&(x<length(xo)) &&(y<length(yo)) &&(x>0) &&(y>0)
    %}

    for x =-50:50
        for y=-50:50
            S1 = abs((x1*y+x2*y1+x*y2-x2*y-x*y1-x1*y2)/2);
            S2 = abs((x1*y+x3*y1+x*y3-x3*y-x*y1-x1*y3)/2);
            S3 = abs((x3*y+x2*y3+x*y2-x2*y-x*y3-x3*y2)/2);
            if (S1+S2+S3==S) &&(sqrt((x-x1)^2+(y-y1)^2)<d1)
                Z(x,y)=(1/(1+exp(1)^(sqrt((x-x1)^2+(y-y1)^2)-
d1/2)))*(Z(x1,y1)-Durchschnitt)+Durchschnitt;
            end
            if (S1+S2+S3==S) &&(sqrt((x-x2)^2+(y-y2)^2)<d2)
                Z(x,y)=(1/(1+exp(1)^(sqrt((x-x2)^2+(y-y2)^2)-
d2/2)))*(Z(x2,y2)-Durchschnitt)+Durchschnitt;
            end
            if (S1+S2+S3==S) &&(sqrt((x-x3)^2+(y-y3)^2)<d3)
                Z(x,y)=(1/(1+exp(1)^(sqrt((x-x3)^2+(y-y3)^2)-
d3/2)))*(Z(x3,y3)-Durchschnitt)+Durchschnitt;
            end
        end
    end

end

    for x =1:50
        for y=1:31
            if Z(x,y)<1e-6
                Z(x,y)=Z(x,y-1);
            end
        end
    end

end

    for x =1:50
        for y=1:30
            if abs((Z(x,y)-Z(x,y+1)))>5
                Z(x,y)=Durchschnittall;
                Z(x,y+1)=Z(x,y);
            end
        end
    end

end

    for x =1:50
        for y=2:31
            if abs((Z(x,y)-Z(x,y-1)))>5

```

```

        Z(x,y)=Durchschnittall;
        Z(x,y-1)=Z(x,y);
    end

    end

end

for x =1:50
    for y=1:30
        if abs((Z(x,y)-Z(x,y+1)))>5
            Z(x,y)=Durchschnittall;
            Z(x,y+1)=Z(x,y);
        end
    end

    end

end

xo=1:31;
yo=1:50;

[X,Y] = meshgrid(xo,yo);
gitter = surf(X,Y,Z);
view(180,120);
xlabel('X','FontSize',11);
ylabel('Y','FontSize',11);
zlabel('Z','FontSize',11);
zlim([75,100]);
set(gcf,'color','w');
colormap jet;
colorbar;
ax = gca;
ax.FontSize = 12;
caxis([75 100]);

```

4. Algorithmus für Leermessung

```
temp1 = mean(Z_Range(1/2*size(Z_Range(:,1))+4:2/3*size(Z_Range(:,1)),1));
output_matrix1D = Z_Range(:,1);
output_matrix1D(output_matrix1D < 0.8*temp1) = 0.8*temp1;
Z_Range_Mean = output_matrix1D;

effective_range =
Z_Range_Angle_XY(1/2*size(Z_Range_Angle_XY(:,1))+4:2/3*size(Z_Range_Angle_X
Y(:,1)),:);

temp2 = mean(mean(effective_range));
output_matrix = Z_Range_Angle_XY;
output_matrix(output_matrix < 0.8*temp2) = 0.8*temp2;
Z_Range_Angle_XY_Mean = output_matrix;

if first_measurement == 1
    Mean_Range_Angle = Z_Range_Angle_XY_Mean;
    Mean_Range = Z_Range_Mean;
    Empty_Range_Angle = Z_Range_Angle_XY_Mean;
    Empty_Range = Z_Range_Mean;
    Mode_Empty_Range = mode(Empty_Range(:));
    Mode_Empty_Range_Angle = mode(Empty_Range_Angle(:));
    Temp_Empty_Range_Angle = Empty_Range_Angle;
    Temp_Empty_Range = Empty_Range;
    first_measurement = 0;
else
    Mode_Z_Range_Angle = mode(Z_Range_Angle_XY_Mean(:));
    Mode_Z_Range = mode(Z_Range_Mean(:));

    Temp_Empty_Range_Angle = Empty_Range_Angle;

    Temp_Empty_Range = Empty_Range;

    Empty_Range_Angle = (Empty_Range_Angle.*(Z_Range_Angle_XY_Mean >
Empty_Range_Angle))+ ...

    Z_Range_Angle_XY_Mean.*(Empty_Range_Angle>=Z_Range_Angle_XY_Mean));

    %           Empty_Range_Angle = Empty_Range_Angle.*(1-exp(-
measurement_num))+ ...
    %
    Empty_Range_Angle.*(Mode_Z_Range_Angle/Mode_Empty_Range_Angle)*exp(-
measurement_num);
    Mode_Empty_Range_Angle = mode(Empty_Range_Angle(:));

    Empty_Range_Angle = Empty_Range_Angle+((measurement_num-
1)*Mode_Empty_Range_Angle+Mode_Z_Range_Angle)/measurement_num-
Mode_Empty_Range_Angle;

    Empty_Range = (Empty_Range.*(Z_Range_Mean > Empty_Range)+ ...
    Z_Range_Mean.*(Empty_Range >= Z_Range_Mean));
```

```

%      Empty_Range = Empty_Range.*(1-exp(-measurement_num))+ ...
%      Empty_Range.*(Mode_Z_Range/Mode_Empty_Range)*exp(-
measurement_num);
Mode_Empty_Range = mode(Empty_Range(:));

Empty_Range = Empty_Range+((measurement_num-
1)*Mode_Empty_Range+Mode_Z_Range)/measurement_num-Mode_Empty_Range;

Mean_Range_Angle =
log(measurement_num)/log(measurement_num+1)*Mean_Range_Angle + ...
(1-
log(measurement_num)/log(measurement_num+1))*Z_Range_Angle_XY_Mean;

Mean_Range = log(measurement_num)/log(measurement_num+1)*Mean_Range
+ ...
(1-log(measurement_num)/log(measurement_num+1))*Z_Range_Mean;

Z_Range_Angle_XY_Mean = Z_Range_Angle_XY_Mean -
mode(Z_Range_Angle_XY_Mean(:)) + Mode_Empty_Range_Angle
Z_Range_Mean = Z_Range_Mean - mode(Z_Range_Mean(:)) +
Mode_Empty_Range;

end

display(mean(Empty_Range(:)));
display(mean(Z_Range_Mean(:)));

%      Z_Range_Angle_XY_Mean = Z_Range_Angle_XY_Mean -
mode(Z_Range_Angle_XY_Mean(:)) + Mode_Empty_Range_Angle
%      Z_Range_Mean = Z_Range_Mean - mode(Z_Range_Mean(:)) +
Mode_Empty_Range;

measurement_num = measurement_num + 1;

```

5. Algorithmus für CFAR

```
function [detected_points] = CFAR_Detection(range_angle_heatmap)

max_range_bin=length(range_angle_heatmap(:,1));
max_angle_bin = length(range_angle_heatmap(1,:));
max_angle = zeros(1,max_angle_bin);
max_angle_pos = zeros(1,max_angle_bin);

[max_angle, max_angle_pos] = max(range_angle_heatmap,[],2);
num_guard_cells = round(max_range_bin/50);
num_training_cells = round(max_range_bin/20);
rangeThresh = 2*num_training_cells*0.2;
detectedObjects = zeros(max_range_bin,max_angle_bin);
current_ang=0;

count = 0;

for range_bin=1:max_range_bin
    current_ang = max_angle_pos(range_bin);
    num_cells = 2*num_guard_cells + 2*num_training_cells + 1;
    mid_cell = num_guard_cells+num_training_cells+1;
    CFAR_element = zeros(1,num_cells);
    begining_pos = range_bin - num_guard_cells - num_training_cells;
    ending_pos = range_bin + num_guard_cells + num_training_cells;
    if begining_pos < 1
        num_repeat_cells = 1 - begining_pos;
        repeat_cell = range_angle_heatmap(1,current_ang);
        CFAR_element(1:num_repeat_cells) = repeat_cell;
        CFAR_element(num_repeat_cells+1:num_cells)= ...

range_angle_heatmap(begining_pos+num_repeat_cells:ending_pos,current_ang);
    elseif ending_pos > max_range_bin
        num_repeat_cells = ending_pos - max_range_bin;
        repeat_cell = range_angle_heatmap(max_range_bin,current_ang);
        CFAR_element(1:num_cells-num_repeat_cells) = ...
            range_angle_heatmap(begining_pos:max_range_bin,current_ang);
        CFAR_element(num_cells-num_repeat_cells+1:num_cells)=repeat_cell;
    else
        CFAR_element(1:num_cells) = ...
            range_angle_heatmap(begining_pos:ending_pos);
    end

    for index=1:num_guard_cells
        if (CFAR_element(mid_cell) > ...
            CFAR_element(mid_cell + index)) && ...
            (CFAR_element(mid_cell) >...
            CFAR_element(mid_cell - index))
            peakFlag = 1;
        else
            peakFlag = 0;
            break;
        end
    end

end
```

```

noiseLeft=0;
noiseRight=0;
for index = 1:num_training_cells
    noiseLeft = noiseLeft + CFAR_element(index);
    noiseRight = noiseRight + CFAR_element(num_cells+1-index);
end
noiseSum=noiseLeft+noiseRight;
if (CFAR_element(mid_cell)*rangeThresh>noiseSum) && (peakFlag==1) && ...
    (range_bin>3)
    %&& (current_ang>1) && (current_ang< max_angle_bin)
    detectedObjects(range_bin,current_ang)=1;
    count=count+1;
end
end

detected_points = detectedObjects;
end

```